

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn chân thành nhất tới TS. Đỗ Bá Lâm. Trong suốt quá trình thực hiện đề tài, thầy đã luôn dành thời gian định hướng khoa học, tận tình hướng dẫn và hỗ trợ em vượt qua những khó khăn từ giai đoạn bắt đầu cho đến khi hoàn thành đồ án tốt nghiệp này.

Đồng thời, em xin trân trọng cảm ơn tập thể giảng viên Đại học Bách khoa Hà Nội nói chung và các thầy cô giáo tại Trường Công nghệ Thông tin và Truyền thông nói riêng. Những kiến thức chuyên môn sâu sắc cùng phương pháp luận nghiên cứu khoa học mà các thầy cô truyền đạt trong suốt quá trình học tập là hành trang vô cùng quý giá cho em.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình và bạn bè. Đây là nguồn động viên tinh thần to lớn, luôn chia sẻ, đồng hành và tạo mọi điều kiện thuận lợi nhất để em yên tâm tập trung học tập và hoàn thành chặng đường nghiên cứu này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh rủi ro an ninh mạng ngày càng phức tạp, việc bảo vệ các hệ thống liên lạc nội bộ đang trở thành ưu tiên hàng đầu, thúc đẩy sự phổ biến của tiêu chuẩn mã hóa đầu cuối (E2EE) như một giải pháp bảo mật tất yếu. Hiện nay, hầu hết các ứng dụng nhắn tin E2EE hàng đầu vận hành dựa trên giao thức Double Ratchet, một tiêu chuẩn bộc lộ nhiều hạn chế khi mở rộng quy mô nhóm. Nhằm giải quyết bài toán này, Messaging Layer Security (MLS) ra đời như một giao thức tiên phong, tái định nghĩa toàn diện kiến trúc mã hóa nhóm. Tuy nhiên, tiêu chuẩn MLS hiện tại được thiết kế độc quyền cho kiến trúc tập trung và bắt buộc phụ thuộc vào một máy chủ điều phối để đảm bảo thứ tự. Việc triển khai MLS trên môi trường mạng ngang hàng (P2P) phi tập trung vẫn là một thách thức chưa được giải quyết.

Từ thực tiễn đó, em quyết định theo đuổi hướng nghiên cứu tích hợp trực tiếp giao thức MLS lên kiến trúc mạng ngang hàng, tước bỏ hoàn toàn máy chủ trung tâm. Hướng đi này được lựa chọn nhằm chứng minh khả năng dung hợp giữa tiêu chuẩn mã hóa nhóm tiên tiến nhất hiện nay và triết lý mạng phi tập trung, từ đó vạch ra mô hình lý thuyết cho một hệ thống truyền thông không tồn tại điểm lỗi duy nhất.

Tổng quan giải pháp của nghiên cứu là đề xuất một lớp điều phối phi tập trung. Kiến trúc này thay thế hoàn toàn vai trò của máy chủ trong việc quản lý cập nhật trạng thái nhóm, đồng thời cung cấp khả năng tự động duy trì trạng thái mã hóa nhóm, nhận diện và Fork Healing khi mạng xảy ra đứt gãy. Bên cạnh phần thiết kế giao thức, đồ án còn xây dựng một ứng dụng desktop nhằm triển khai các luồng khởi tạo định danh, cấp bundle, tham gia tổ chức, trao đổi thông điệp mã hóa đầu cuối và quản trị thành viên. Sự hiện diện của ứng dụng này cho phép đối chiếu trực tiếp giữa mô hình giao thức và hành vi vận hành ở mức ứng dụng, qua đó củng cố tính khả thi của hướng tiếp cận được đề xuất.

Sinh viên thực hiện

Lê Tiến Dũng

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện tại và hạn chế	2
1.3 Mục tiêu và định hướng giải pháp	3
1.4 Đóng góp của đề án	4
1.5 Bố cục đề án	4
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	6
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán	6
2.2 Các kết quả nghiên cứu tương tự	8
2.2.1 Các cơ chế mã hóa nhóm trước MLS.....	8
2.2.2 MLS với Delivery Service	9
2.2.3 Các hướng phi tập trung hóa MLS	9
2.3 Nền tảng MLS và TreeKEM	10
2.3.1 Các khái niệm cốt lõi: client, group, member và epoch	10
2.3.2 Ratchet Tree và TreeKEM.....	11
2.3.3 Proposal, Commit và tiến trình chuyển epoch	11
2.3.4 Các dấu hiệu nhận biết trạng thái: tree hash, transcript hash và epoch authenticator	12
2.3.5 Application messages, delayed messages và External Commit.....	12
2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán	14
2.4.1 Đặc tính của mạng ngang hàng.....	14
2.4.2 go-libp2p và GossipSub.....	14
2.4.3 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng.....	14
2.4.4 Khoảng trống cần giải quyết.....	15

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	16
3.1 Tổng quan bài toán và hướng tiếp cận	16
3.1.1 Bài toán cần giải quyết	16
3.1.2 Kiến trúc của giải pháp.....	17
3.1.3 Các cơ chế chính của phương pháp	17
3.1.4 Mối liên hệ giữa thách thức, câu hỏi nghiên cứu và cơ chế	18
3.1.5 Phạm vi bảo mật và Mô hình mối đe dọa (Threat Model).....	18
3.2 Giao thức Single-Writer theo epoch.....	19
3.2.1 Mục tiêu thiết kế	19
3.2.2 Tập ứng viên hợp lệ	20
3.2.3 Hàm bầu chọn Token Holder	20
3.2.4 Tạo Proposal và Commit.....	21
3.2.5 Chuyển quyền và failover	22
3.3 Kiểm tra nhất quán Epoch	23
3.3.1 Mục tiêu thiết kế	23
3.3.2 Cấu trúc bản tin điều phối	23
3.3.3 Phân loại thông điệp theo Epoch.....	23
3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai	24
3.3.5 Xử lý thao tác bị bỏ lỡ	25
3.4 Cơ chế Fork Healing trạng thái	25
3.4.1 Bối cảnh phân vùng mạng.....	25
3.4.2 Phát hiện nghi vấn phân nhánh và đối chiếu trạng thái	25
3.4.3 Hàm trọng số nhánh	26
3.4.4 Quy trình Fork Healing.....	27
3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock	29
3.5.1 Lý do cần HLC	29

3.5.2 Tạo timestamp khi gửi tin.....	29
3.5.3 Cập nhật HLC khi nhận tin	30
3.5.4 Vai trò của HLC trong hệ thống.....	31
3.6 Triển khai ứng dụng thử nghiệm phục vụ kiểm chứng	31
3.6.1 Ranh giới trách nhiệm trong ứng dụng thử nghiệm	31
3.6.2 Vai trò của ứng dụng desktop	31
3.7 Tính đúng đắn trực giác của phương pháp	31
3.7.1 Tiêu chí hội tụ trạng thái.....	32
CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT.....	34
4.1 Mô hình phân tích và giả định.....	34
4.2 Bất biến 1: Epoch cục bộ chỉ được giữ nguyên hoặc tăng.....	35
4.2.1 Phát biểu bất biến.....	35
4.2.2 Cơ sở từ MLS	35
4.2.3 Cơ chế bảo toàn bất biến.....	35
4.3 Bất biến 2: Với cùng một ActiveView, chỉ có một Token Holder.....	36
4.3.1 Phát biểu bất biến.....	36
4.3.2 Ý nghĩa đối với concurrent commits	37
4.4 Bất biến 3: Phân nhánh phải được nhận biết sau bước đối chiếu lịch sử.....	37
4.4.1 Phân biệt chậm đồng bộ và phân nhánh thực sự.....	37
4.4.2 Tóm tắt phần lịch sử Commit đã đi qua	37
4.5 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh....	39
4.5.1 Vì sao không thể gộp trực tiếp hai nhánh.....	39
4.5.2 Quy tắc chọn nhánh.....	39
4.5.3 Dùng External Proposal như cơ chế hội tụ	40
4.5.4 Xử lý thông điệp phát sinh ở nhánh thua.....	40

4.6 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit.....	41
4.6.1 Phân biệt hai loại thứ tự.....	41
4.6.2 Cơ sở của HLC	41
4.6.3 Giới hạn của HLC	41
4.7 Phân tích chi phí lý thuyết	42
4.7.1 Chi phí gửi application message	42
4.7.2 Chi phí cập nhật thành viên và rekey	42
4.7.3 Chi phí của Single-Writer	42
4.7.4 Chi phí của kiểm tra epoch và đối chiếu trạng thái.....	42
4.7.5 Chi phí của Fork Healing	43
4.7.6 Phân tích cơ chế phát lại thông điệp ứng dụng.....	43
4.8 Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn	44
4.9 So sánh đặc tính với các thuật toán đồng thuận truyền thống.....	44
4.10 Giới hạn lý thuyết của phương pháp.....	45
CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM.....	47
5.1 Mục tiêu và phạm vi đánh giá	47
5.2 Môi trường và phương pháp thí nghiệm	47
5.3 Đánh giá cơ chế Single-Writer và gom đề xuất	48
5.4 Đánh giá hội tụ trong điều kiện mạng hỗn loạn.....	49
5.5 Ảnh hưởng độ sâu phân kỳ đến thời gian phục hồi	50
5.6 Đánh giá khả năng mở rộng của thao tác thành viên.....	53
5.7 Đánh giá chi phí mật mã MLS	54
5.8 Chi phí phụ trợ của lớp điều phối	55
5.9 Kiểm chứng Forward Secrecy bằng thành phần MLS thật.....	56

5.10 Đối chiếu trên ứng dụng desktop	56
5.10.1 Tạo tổ chức và cấp bundle	57
5.10.2 Thiết bị mới vào hệ thống	57
5.10.3 Trao đổi tin nhắn trong nhóm	58
5.11 Nhận xét và giới hạn đánh giá	61
CHƯƠNG 6. KẾT LUẬN	62
6.1 Kết luận chung	62
6.2 Giới hạn của đề án.....	63
6.3 Hướng phát triển.....	64
TÀI LIỆU THAM KHẢO.....	68
PHỤ LỤC.....	70
A. ỨNG DỤNG DEMO VÀ KỊCH BẢN MINH HỌA.....	70
A.1 Kiến trúc phần mềm và công nghệ nền tảng	70
A.2 Các kịch bản sử dụng (Use Cases) cốt lõi	71
A.3 Hình ảnh minh họa các thành phần chức năng	72
A.3.1 Quy trình gia nhập hệ thống và cấp phát Bundle	72
A.3.2 Không gian làm việc và các chức năng cộng tác	75
A.3.3 Chia sẻ tệp mã hóa trong nhóm.....	80
A.3.4 Quản trị hệ thống	81

DANH MỤC HÌNH VẼ

Hình 2.1	Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS . . .	7
Hình 3.1	Kiến trúc logic của giải pháp và ranh giới trách nhiệm giữa lớp điều phối với lớp MLS	17
Hình 3.2	Sơ đồ tổng quan luồng xử lý: bốn cơ chế điều phối bao quanh lõi MLS. Mũi tên liền là luồng xử lý chính, mũi tên đứt là luồng phục hồi định kỳ	19
Hình 5.1	Hiệu quả của cơ chế gom đề xuất khi mức đồng thời tăng . . .	49
Hình 5.2	Hội tụ epoch của các nút trong chaos test	50
Hình 5.3	Thời gian healing (cam) và kích thước group state (xanh) theo độ sâu phân kỳ	51
Hình 5.4	Healing time chuẩn hóa theo kích thước group state — tỷ lệ ms/KB giảm khi D tăng	52
Hình 5.5	Thiết bị thành viên ở trạng thái chờ cấp phát bundle	57
Hình 5.6	Bảng điều khiển quản trị phát hành bundle cho thiết bị mới . .	58
Hình 5.7	Cửa sổ giao tiếp nhóm trên thiết bị Admin	59
Hình 5.8	Cửa sổ giao tiếp nhóm trên thiết bị Alice	59
Hình 5.9	Cửa sổ giao tiếp nhóm trên thiết bị Bob	60
Hình 5.10	Màn hình quản trị thành viên của nhóm	60
Hình A.1	Màn hình khởi tạo định danh cục bộ trong ứng dụng thử nghiệm	72
Hình A.2	Màn hình tạo tổ chức gốc của quản trị viên	73
Hình A.3	Thiết bị thành viên ở trạng thái chờ cấp phát bundle	73
Hình A.4	Bảng điều khiển quản trị phát hành bundle cho thiết bị mới . .	74
Hình A.5	Tổng quan không gian làm việc của ứng dụng desktop thử nghiệm	75
Hình A.6	Giao diện trao đổi tin nhắn trong nhóm trên thiết bị Admin . .	76
Hình A.7	Giao diện trao đổi tin nhắn trong nhóm trên thiết bị Alice . . .	76
Hình A.8	Giao diện trao đổi tin nhắn trong nhóm trên thiết bị Bob . . .	77
Hình A.9	Màn hình hoạt động và thông báo trong ứng dụng thử nghiệm	77
Hình A.10	Không gian kênh giao tiếp theo dạng bài viết và phản hồi . . .	78
Hình A.11	Hộp thoại tạo nhóm, trò chuyện trực tiếp (DM) hoặc kênh . . .	78
Hình A.12	Hộp thoại mời thêm thành viên vào một nhóm đang có sẵn . .	79
Hình A.13	Tệp mã hóa được hiển thị trong dòng tin nhắn của nhóm . . .	80
Hình A.14	Màn hình quản trị thành viên và chính sách mời của nhóm . .	81

DANH MỤC BẢNG BIỂU

Bảng 3.1	Mối liên hệ giữa thách thức, câu hỏi nghiên cứu và cơ chế đề xuất	18
Bảng 4.1	Quy tắc xử lý thông điệp theo epoch	35
Bảng 4.2	Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn	44
Bảng 4.3	Bảng so sánh cơ chế điều phối đề xuất với các thuật toán đồng thuận truyền thống	45
Bảng 5.1	Các tiêu chí đánh giá thực nghiệm	48
Bảng 5.2	So sánh xử lý ngay và cơ chế gom đề xuất	49
Bảng 5.3	Thời gian healing, kích thước group state và tỷ lệ healing/group state theo độ sâu phân kỳ (trung vị của 10 lần chạy) . . .	51
Bảng 5.4	Chi phí thao tác thành viên theo quy mô nhóm	53
Bảng 5.5	So sánh trung vị thời gian xử lý giữa hai hướng mã hóa, với payload 1024 byte và 100 mẫu cho mỗi điểm đo	54
Bảng 5.6	So sánh p95 và p99 ở nhóm 4096 thành viên	54
Bảng 5.7	Chi phí xử lý cục bộ: Điều phối (giả lập MLS) và Điều phối + MLS (MLS thật)	56

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AEAD	Mã hóa có xác thực kèm dữ liệu (Authenticated Encryption with Associated Data)
AP	Khả dụng và chịu đựng phân vùng mạng (Availability and Partition Tolerance)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
BFT	Chịu lỗi Byzantine (Byzantine Fault Tolerance)
CAP	Định lý CAP (Consistency, Availability, Partition Tolerance)
DHT	Bảng băm phân tán (Distributed Hash Table)
DMLS	Bảo mật tầng thông điệp phi tập trung (Decentralized Messaging Layer Security)
DS	Dịch vụ phân phối (Delivery Service)
E2EE	Mã hóa đầu cuối (End-to-End Encryption)
FREEK	Thỏa thuận khóa nhóm liên tục chịu rẽ nhánh (Fork-Resilient Continuous Group Key Agreement)
FS	Bảo mật xuôi (Forward Secrecy)
HLC	Đồng hồ logic hỗn hợp (Hybrid Logical Clock)
IP	Giao thức Internet (Internet Protocol)
LAN	Mạng cục bộ (Local Area Network)
mDNS	Hệ thống phân giải tên miền multicast (multicast Domain Name System)
MLS	Bảo mật tầng thông điệp (Messaging Layer Security)

Thuật ngữ	Ý nghĩa
NAT	Biến đổi địa chỉ mạng (Network Address Translation)
P2P	Mạng ngang hàng (Peer-to-Peer)
PCS	Bảo mật sau thỏa hiệp (Post-Compromise Security)
QUIC	Kết nối Internet UDP nhanh (Quick UDP Internet Connections)
RFC	Tài liệu đặc tả chuẩn / Yêu cầu bình luận (Request for Comments)
SMR	Máy trạng thái nhân bản (State Machine Replication)
TCP	Giao thức điều khiển truyền vận (Transmission Control Protocol)
WAN	Mạng diện rộng (Wide Area Network)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh hoạt động trao đổi thông tin ngày càng phụ thuộc vào hạ tầng số, yêu cầu bảo vệ nội dung liên lạc nội bộ đang trở nên cấp thiết đối với cơ quan, doanh nghiệp và các tổ chức có dữ liệu nhạy cảm. Rò rỉ thông tin không chỉ gây thiệt hại về kinh tế, mà còn ảnh hưởng trực tiếp tới uy tín và an toàn vận hành. Vì vậy, mã hóa đầu cuối đã trở thành một hướng tiếp cận quan trọng, trong đó chỉ các thiết bị đầu cuối hợp lệ mới có khả năng giải mã nội dung thông điệp, còn các thành phần trung gian không được phép truy cập bản rõ [1], [2].

Tuy nhiên, trong nhiều hệ thống nhắn tin bảo mật hiện nay, máy chủ trung tâm vẫn giữ vai trò rất lớn trong quá trình vận hành. Máy chủ thường đảm nhiệm việc định tuyến thông điệp, hỗ trợ thiết bị ngoại tuyến, lưu trữ vật liệu khóa ban đầu và đồng bộ trạng thái giữa các thành viên [2]. Cách tổ chức này giúp hệ thống dễ quản lý hơn, nhưng đồng thời cũng tạo ra sự phụ thuộc đáng kể vào hạ tầng tập trung. Với những môi trường như mạng nội bộ biệt lập, mạng tạm thời không có Internet, hoặc các tổ chức muốn kiểm soát chặt chẽ dữ liệu, sự phụ thuộc đó trở thành một hạn chế thực tế. Những bối cảnh này đòi hỏi một mô hình trong đó dữ liệu được ưu tiên lưu tại thiết bị, các nút có thể trao đổi trực tiếp với nhau, và hệ thống vẫn duy trì được hoạt động ngay cả khi không có máy chủ trung tâm luôn sẵn sàng. Định hướng đó gần với tinh thần ưu tiên dữ liệu cục bộ và mạng ngang hàng [3].

Đối với bài toán liên lạc hai bên, nhiều giao thức đã chứng minh được hiệu quả trong thực tế, tiêu biểu là Double Ratchet. Tuy nhiên, khi mở rộng sang liên lạc nhóm, bài toán không còn dừng ở việc mã hóa từng thông điệp, mà chuyển thành bài toán duy trì một trạng thái mật mã chung giữa nhiều thiết bị có thể gửi, nhận, mất kết nối và quay trở lại ở những thời điểm khác nhau [1], [4]. Nói cách khác, khó khăn cốt lõi của truyền thông nhóm bảo mật không chỉ nằm ở việc giữ bí mật nội dung, mà còn nằm ở việc bảo đảm tất cả thành viên hợp lệ cùng nhìn thấy một trạng thái khóa nhóm nhất quán.

Messaging Layer Security (MLS) đã được chuẩn hóa trong RFC 9420 để giải quyết bài toán thiết lập khóa nhóm bất đồng bộ với các bảo đảm quan trọng như Forward Secrecy và Post-Compromise Security [1]. Nhờ cấu trúc TreeKEM, MLS cho phép cập nhật trạng thái khóa nhóm hiệu quả hơn nhiều so với các cơ chế mã hóa nhóm dựa hoàn toàn trên các kênh liên lạc hai bên. Dù vậy, MLS không phải là một hệ thống nhắn tin hoàn chỉnh. Theo kiến trúc của RFC 9750, giao thức này vẫn cần được đặt trong một môi trường triển khai cụ thể, nơi ứng dụng phải tự giải

quyết các vấn đề như xác thực danh tính, phân phối thông điệp và đặc biệt là xử lý thứ tự của các bản tin làm thay đổi trạng thái nhóm [2].

Chính ở điểm này dẫn đến xuất hiện mâu thuẫn trung tâm của đề tài. Trong mô hình có một Delivery Service nhất quán mạnh, thành phần phân phối này có thể đóng vai trò áp đặt thứ tự tuyến tính cho các Commit và loại bỏ xung đột khi nhiều thành viên cùng cập nhật nhóm tại một epoch. Ngược lại, trong môi trường P2P không có một thực thể trung tâm làm công việc tuần tự hóa, các Commit có thể đến các thiết bị khác nhau theo những thứ tự khác nhau. Hệ quả là các nút có thể áp dụng những chuỗi thay đổi không giống nhau và làm nhóm rơi vào trạng thái phân nhánh mật mã, tức là các thành viên tuy vẫn mang cùng một định danh nhóm nhưng không còn chia sẻ cùng trạng thái khóa để có thể tiếp tục liên lạc [2], [5], [6].

Từ thực tế đó, đồ án lựa chọn nghiên cứu bài toán vận hành MLS trên mạng ngang hàng theo hướng bổ sung một lớp điều phối phi tập trung ở tầng ứng dụng. Mục tiêu của hướng tiếp cận này không phải là sửa đổi lõi mật mã của MLS, mà là xây dựng các cơ chế hỗ trợ để nhóm vẫn có thể duy trì tính nhất quán trạng thái, phát hiện phân nhánh khi cần thiết và hội tụ trở lại sau khi mạng được khôi phục.

1.2 Các giải pháp hiện tại và hạn chế

Các hướng tiếp cận hiện nay đối với truyền thông nhóm bảo mật có thể chia thành hai nhóm lớn. Nhóm thứ nhất mở rộng từ các cơ chế bảo mật hai bên sang môi trường nhóm; nhóm thứ hai sử dụng giao thức thiết lập khóa nhóm chuyên biệt, trong đó MLS là đại diện tiêu biểu [1], [7], [8]. Mỗi hướng đều có giá trị thực tiễn nhất định, nhưng khi đặt trong yêu cầu vận hành P2P hoàn toàn phi tập trung, các giới hạn của chúng bộc lộ rõ.

Với nhóm thứ nhất, những cơ chế như Sender Keys hay Megolm có ưu điểm là triển khai tương đối thực dụng và có chi phí gửi tin thấp khi nhóm đông thành viên [7], [8]. Tuy nhiên, chúng chủ yếu tối ưu cho việc phát tán thông điệp trong nhóm, chứ không trực tiếp giải quyết bài toán duy trì một trạng thái khóa nhóm nhất quán khi thành viên thay đổi thường xuyên hoặc khi mạng bị chia cắt. Trong các trường hợp khóa gửi tin bị lộ hoặc membership biến động liên tục, chi phí rekey và khả năng phục hồi bảo mật của các cơ chế này vẫn còn những hạn chế đáng kể [7], [8].

Với nhóm thứ hai, MLS cung cấp một mô hình chặt chẽ hơn cho quản lý khóa nhóm và đạt được các bảo đảm bảo mật mạnh hơn cho truyền thông quy mô lớn [1]. Dù vậy, MLS chuẩn vẫn giả định hệ thống triển khai có khả năng bảo đảm việc phân phối và tuần tự hóa các bản tin thay đổi trạng thái [2]. Khi đưa MLS sang môi trường P2P, bài toán khó nhất không nằm ở bản thân thao tác mật mã, mà nằm ở

việc thay thế vai trò điều phối vốn thường do Delivery Service đảm nhiệm.

Một số nghiên cứu gần đây, chẳng hạn hướng Decentralized MLS, đã thử giải quyết vấn đề bằng cách mở rộng trực tiếp giao thức MLS để hỗ trợ xử lý Commit ngoài thứ tự trong môi trường phi tập trung [6]. Đây là một hướng nghiên cứu đáng chú ý, nhưng cũng đòi hỏi client duy trì thêm trạng thái mật mã cũ hoặc trạng thái tạm thời, từ đó làm tăng độ phức tạp triển khai và đặt ra thêm đánh đổi đối với Forward Secrecy [5], [6]. Hơn nữa, hướng này hiện vẫn dừng ở mức Internet-Draft, chưa trở thành một chuẩn hoàn chỉnh.

Từ các phân tích trên có thể thấy, rào cản lớn nhất khi đưa MLS lên mạng ngang hàng không xuất phát từ bản thân các thuật toán mật mã. Vấn đề bảo mật dữ liệu nhạy cảm đã được giải quyết trọn vẹn bởi cấu trúc của chuẩn MLS. Khó khăn thực sự nằm ở góc độ hệ phân tán (distributed systems): làm thế nào để duy trì sự nhất quán của trạng thái nhóm khi hệ thống không còn máy chủ trung tâm (Delivery Service) làm nhiệm vụ tuần tự hóa thông điệp. Việc thiếu vắng một cơ chế điều phối đủ thực dụng để giải quyết bài toán đồng thuận mạng này chính là khoảng trống mà đề án hướng tới giải quyết.

1.3 Mục tiêu và định hướng giải pháp

Mục tiêu của đề án là nghiên cứu, thiết kế và triển khai một cơ chế điều phối phi tập trung nhằm hỗ trợ MLS vận hành trên mạng ngang hàng. Về mặt nghiên cứu, đề án tập trung vào ba vấn đề chính: giảm khả năng xuất hiện concurrent commits khi các nút còn cùng ActiveView, bảo vệ tính nhất quán epoch của trạng thái cục bộ trong môi trường bất đồng bộ, và xây dựng cơ chế phục hồi khi phân nhánh trạng thái vẫn xảy ra do phân vùng mạng. Về mặt ứng dụng, đề án hướng tới một hệ thống liên lạc nhóm bảo mật ưu tiên dữ liệu cục bộ, có khả năng vận hành trong các điều kiện mạng không ổn định và giảm phụ thuộc vào hạ tầng tập trung [3]. Ứng dụng desktop được xây dựng trong đề án không chỉ đóng vai trò minh họa giao diện, mà còn là môi trường tích hợp để kiểm chứng trọn vẹn chuỗi thao tác từ khởi tạo định danh, cấp bundle, tham gia tổ chức, tạo nhóm cho tới trao đổi thông điệp.

Trên cơ sở mục tiêu đó, đề án lựa chọn cách tiếp cận giữ nguyên lõi mật mã của MLS/OpenMLS, đồng thời bổ sung một lớp điều phối ở phía ngoài để xử lý các vấn đề mà môi trường P2P đặt ra. Thay vì mở rộng trực tiếp giao thức MLS hoặc đưa vào một cơ chế đồng thuận nặng cho mọi Commit, giải pháp đề xuất tập trung vào bốn nội dung: giới hạn quyền tạo Commit theo từng epoch, kiểm tra tính phù hợp của thông điệp trước khi đưa xuống lớp mật mã, phát hiện và Fork Healing trạng thái khi mạng bị chia cắt, và sử dụng đồng hồ logic lai để ổn định thứ tự hiển

thị thông điệp ứng dụng. Cách tiếp cận này cho phép hệ thống tận dụng các bảo đảm bảo mật của MLS, đồng thời thích nghi tốt hơn với đặc tính bất đồng bộ và phân tán của mạng ngang hàng.

Trong lựa chọn thiết kế này, đồ án chấp nhận một đánh đổi có chủ đích. Hệ thống không theo đuổi nhất quán mạnh tuyệt đối tại mọi thời điểm, bởi điều đó thường đòi hỏi chi phí đồng thuận cao và làm giảm tính sẵn sàng của các phân vùng mạng không đủ điều kiện liên lạc [9]. Thay vào đó, đồ án ưu tiên khả năng hoạt động cục bộ trong điều kiện mạng chia cắt, nhưng vẫn đặt ra cơ chế để các nút có thể quay về cùng một trạng thái mật mã sau khi kết nối được khôi phục. Đây là hướng đi phù hợp hơn với mục tiêu P2P và định hướng ưu tiên dữ liệu cục bộ của hệ thống.

1.4 Đóng góp của đồ án

Trên cơ sở định hướng nêu trên, các đóng góp của đồ án có thể chia thành ba nhóm.

Về thiết kế giải pháp, đồ án xác định bài toán trung tâm của MLS trên mạng ngang hàng dưới góc nhìn nhất quán trạng thái mật mã, qua đó làm rõ rằng khó khăn cốt lõi không nằm ở việc truyền thông điệp mà nằm ở việc duy trì chuỗi trạng thái nhóm hợp lệ khi không còn Delivery Service trung tâm.

Trên cơ sở đó, đồ án đề xuất một lớp điều phối phi tập trung bao quanh MLS, phối hợp bốn cơ chế — Single-Writer theo epoch, kiểm tra nhất quán epoch, phát hiện phân nhánh và Fork Healing, đồng hồ logic lai HLC — để giảm nguy cơ phân nhánh và hỗ trợ hệ thống hội tụ trở lại.

Về triển khai hệ thống, đồ án xây dựng một ứng dụng desktop tích hợp mạng P2P, lưu trữ cục bộ và lõi mật mã MLS/OpenMLS. Các luồng khởi tạo định danh, cấp bundle, quản trị tổ chức và trò chuyện nhóm được triển khai trọn vẹn để kiểm chứng khả năng triển khai của hướng tiếp cận đã đề xuất.

Về đánh giá thực nghiệm, đồ án đo đặc tính hội tụ trạng thái, khả năng phục hồi sau phân vùng mạng và tác động của lớp điều phối đối với quá trình vận hành MLS trong môi trường mạng ngang hàng.

1.5 Bố cục đồ án

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 trình bày cơ sở lý thuyết của đề tài, bao gồm nền tảng MLS, TreeKEM, vai trò của Delivery Service, các hướng tiếp cận liên quan và những đặc tính của môi trường mạng ngang hàng có ảnh hưởng trực tiếp tới bài toán nhất quán trạng thái. Trên nền tảng đó, Chương 3 trình bày chi tiết giao thức điều phối phi tập trung được đề xuất, làm rõ các thành phần, nguyên tắc hoạt động, cách phối hợp giữa lớp điều phối với lõi mật mã MLS,

đồng thời giới thiệu ứng dụng desktop minh họa như môi trường triển khai của các quyết định thiết kế.

Tiếp theo, Chương 4 phân tích cơ sở lý thuyết của giải pháp theo hướng các bất biến quan trọng, điều kiện hội tụ và giới hạn của phương pháp trong môi trường phân tán bất đồng bộ. Chương 5 trình bày quá trình đánh giá thực nghiệm trên ứng dụng thử nghiệm của đồ án, kết hợp giữa dữ liệu đo đạc tự động và các kịch bản thao tác trực tiếp trên ứng dụng để làm rõ khả năng hội tụ, phục hồi sau phân vùng mạng và chi phí vận hành của hệ thống. Cuối cùng, Chương 6 tổng kết các kết quả đạt được, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo của đồ án; phần phụ lục tập hợp bộ ảnh giao diện và các kịch bản minh họa chi tiết của ứng dụng này.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Chương 1 đã trình bày bối cảnh, mục tiêu và định hướng chung của đề tài. Tiếp nối phần mở đầu đó, Chương 2 tập trung làm rõ nền tảng lý thuyết cần thiết cho bài toán vận hành MLS trên mạng ngang hàng. Trọng tâm của chương không phải là liệt kê thật nhiều kiến thức rời rạc, mà là chỉ ra mối liên hệ giữa ba lớp vấn đề: lõi mật mã của MLS, đặc tính bất đồng bộ của mạng P2P và yêu cầu duy trì một trạng thái nhóm nhất quán.

Nội dung chương được tổ chức thành bốn phần. Trước hết, chương phân tích ngữ cảnh bài toán để làm rõ vì sao khi đưa MLS từ mô hình có máy chủ trung tâm sang mô hình ngang hàng, khó khăn lớn nhất không nằm ở việc truyền tin mà nằm ở việc giữ cho các thành viên cùng đi trên một chuỗi trạng thái mật mã hợp lệ. Tiếp theo, chương khảo sát các hướng nghiên cứu liên quan nhằm chỉ ra ưu điểm, giới hạn và khoảng trống còn lại. Sau đó, chương trình bày các kiến thức nền tảng quan trọng nhất của MLS và TreeKEM. Cuối cùng, chương giới thiệu ngắn gọn những đặc tính của mạng ngang hàng, cơ chế phát tán thông điệp và công cụ hỗ trợ sắp xếp thông điệp ứng dụng trong hệ phân tán.

2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán

Trong hệ phân tán nói chung, bài toán nhất quán trạng thái nhằm bảo đảm nhiều nút cùng hiểu và cùng cập nhật dữ liệu theo một quy tắc thống nhất. Đối với hệ thống nhắn tin nhóm được mã hóa đầu cuối, trạng thái cần được giữ nhất quán không chỉ là dữ liệu ứng dụng, mà còn là trạng thái mật mã dùng để sinh khóa, xác thực thành viên và ràng buộc lịch sử hội thoại [1], [2]. Vì vậy, nếu các nút không còn cùng nhìn thấy một trạng thái mật mã chung, khả năng giao tiếp của nhóm sẽ bị ảnh hưởng trực tiếp.

Trong MLS, trạng thái nhóm tiến hóa theo các epoch. Có thể hiểu mỗi epoch là một phiên bản cụ thể của nhóm, trong đó tập thành viên hợp lệ cùng chia sẻ một bộ bí mật tương ứng. Khi nhóm thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa, trạng thái này phải chuyển sang epoch mới. Nếu tất cả thành viên cùng áp dụng các thay đổi theo cùng một thứ tự, nhóm tiếp tục hoạt động bình thường. Ngược lại, nếu các thay đổi được áp dụng theo những thứ tự khác nhau, các thành viên có thể dần tách khỏi nhau về mặt mật mã [1], [5].

Hiện tượng đó được gọi là rẽ nhánh trạng thái mật mã (Cryptographic State Fork). Đây là tình huống trong đó các nút vẫn nghĩ rằng mình thuộc cùng một nhóm, thậm chí có thể cùng nhìn thấy cùng số epoch, nhưng thực tế lại đang nắm

giữ các bí mật nhóm, hàm băm cây (tree hash) hoặc lịch sử bắt tay (transcript hash) khác nhau. Khi đó, thông điệp được tạo trên một nhánh không còn được xử lý hợp lệ trên nhánh còn lại. Khác với xung đột dữ liệu thông thường, trạng thái mật mã đã rẽ nhánh không thể được ghép lại bằng cách trộn hai phiên bản dữ liệu, vì mỗi nhánh đã sinh ra một chuỗi khóa và một lịch sử xác thực riêng [1], [5], [6]. Hiện tượng này được minh họa trong Hình 2.1.

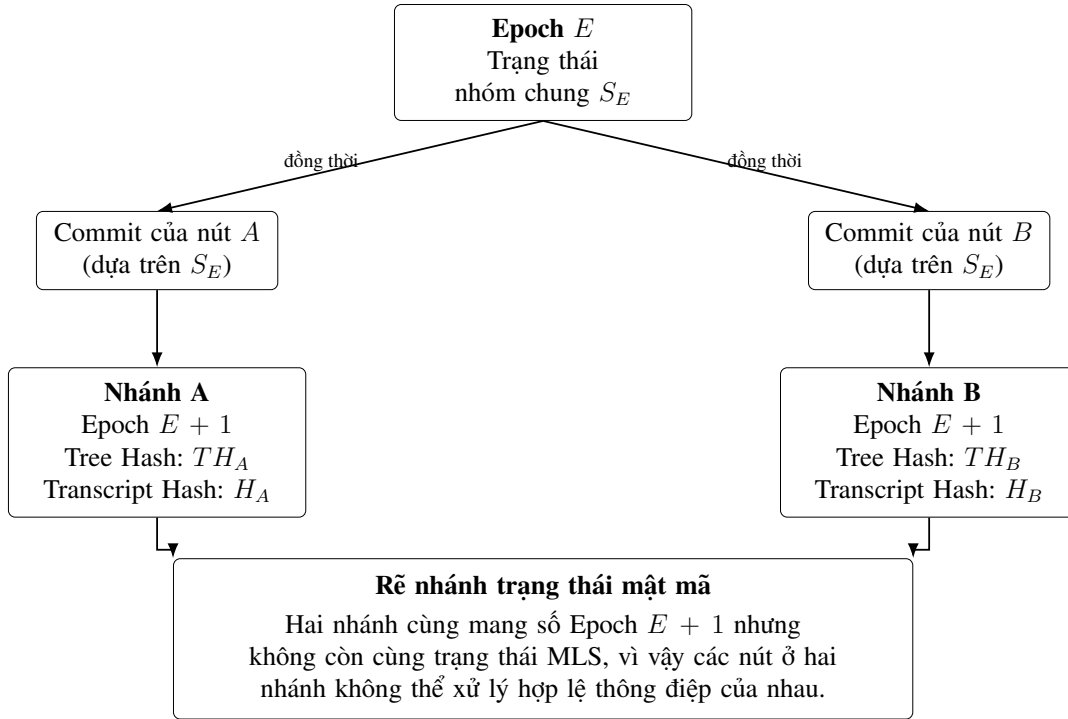


Figure 2.1: Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS

Trong các triển khai thông thường của MLS, việc hạn chế rẽ nhánh được hỗ trợ bởi Delivery Service (DS), có thể hiểu là lớp dịch vụ tiếp nhận và phân phối các thông điệp MLS giữa các thiết bị [2]. Trong môi trường có DS nhất quán mạnh, khi nhiều thành viên cùng tạo Commit ở cùng một epoch, hệ thống có một điểm tự nhiên để quyết định Commit nào được xử lý trước. Nhờ đó, các thay đổi trạng thái dễ được quan sát theo một thứ tự thống nhất hơn.

Khó khăn xuất hiện khi đưa MLS sang mạng ngang hàng. Trong môi trường P2P, thông điệp có thể đến chậm, đến sai thứ tự, bị lặp lại hoặc tạm thời không đến được do thiết bị ngoại tuyến hay do mạng bị chia cắt. Khi đó, không còn một thực thể trung tâm nào tự nhiên đứng ra sắp xếp thứ tự cho các Commit. Nói cách khác, MLS đòi hỏi chuỗi epoch tiến hóa gần như tuyến tính, trong khi mạng ngang hàng bất đồng bộ lại không tự cung cấp thứ tự tuyến tính toàn cục cho các thông điệp thay đổi trạng thái [2], [9]. Đây chính là mâu thuẫn cốt lõi của bài toán mà đề án cần giải quyết.

Dưới góc nhìn của hệ phân tán, quá trình tiến hóa trạng thái của một nhóm MLS mang bản chất của một Máy trạng thái nhân bản (SMR) [10]. Trong mô hình này, mỗi thiết bị của người dùng duy trì một bản sao của trạng thái mật mã (như Ratchet Tree, KeySchedule). Các thao tác *Commit* chính là các phép chuyển trạng thái. Đặc điểm làm cho bài toán đồng thuận trong MLS trở nên khắt khe là tính chất hoàn toàn không giao hoán của thao tác *Commit*. Nếu hai thành viên cùng tạo ra *Commit* tại cùng một epoch, việc áp dụng *Commit* thứ nhất rồi đến *Commit* thứ hai sẽ sinh ra một bộ khóa hoàn toàn khác biệt so với việc áp dụng theo thứ tự ngược lại. Nó không thể được tự động dung hòa bằng các thuật toán gộp dữ liệu bình thường, bởi mỗi bước chuyển trạng thái mật mã đều dùng hàm băm và dẫn xuất ra khóa một chiều dựa trên chính trạng thái liền trước đó. Do tính chất không giao hoán này, hệ thống bắt buộc phải duy trì một thứ tự toàn cục để tránh hiện tượng rẽ nhánh.

Trong phạm vi đồ án, hệ thống được xem như một mạng gồm nhiều thiết bị ngang hàng, mỗi thiết bị lưu cục bộ trạng thái MLS của các nhóm mà nó tham gia. Các tình huống được quan tâm gồm độ trễ mạng không đồng đều, thiết bị ngoại tuyến, sự biến động nút mạng và phân vùng mạng. Mục tiêu của phần nghiên cứu không phải thay đổi các bảo đảm mật mã nền tảng của MLS, mà là tìm cách tổ chức việc điều phối ở tầng ứng dụng để các thiết bị vẫn có thể hội tụ trở lại một trạng thái nhóm hợp lệ sau những bất ổn của mạng.

2.2 Các kết quả nghiên cứu tương tự

Các nghiên cứu liên quan có thể được nhìn từ ba hướng chính: các cơ chế nhắn tin nhóm xuất hiện trước MLS, MLS trong mô hình có Delivery Service và các nỗ lực đưa MLS sang môi trường phi tập trung. Việc phân chia theo hướng này giúp làm rõ vì sao đồ án không chọn lặp lại hoàn toàn một giải pháp có sẵn, mà cần một hướng tiếp cận riêng phù hợp hơn với mục tiêu P2P và định hướng ưu tiên dữ liệu cục bộ.

2.2.1 Các cơ chế mã hóa nhóm trước MLS

Trước khi MLS được chuẩn hóa, nhiều hệ thống nhắn tin nhóm đã mở rộng từ cơ chế bảo mật hai bên sang môi trường nhiều người tham gia. Một nền tảng quan trọng là Double Ratchet, vốn rất hiệu quả cho liên lạc hai bên [4]. Khi đưa sang nhóm, một hướng phổ biến là để mỗi người gửi tự tạo một khóa gửi tin riêng, rồi phân phối khóa đó cho các thành viên còn lại qua các kênh bảo mật hai bên. Cách làm này thường được gọi là Sender Keys [7]. Sau khi khóa gửi tin đã được chia sẻ, người gửi chỉ cần mã hóa một lần cho mỗi thông điệp, còn hạ tầng bên dưới chịu trách nhiệm phát tán bản mã tới cả nhóm. Ưu điểm của hướng này là chi phí gửi tin thấp và triển khai tương đối thực dụng. Tuy nhiên, khi nhóm đông hoặc thay đổi

thành viên thường xuyên, chi phí cập nhật khóa và phục hồi sau thỏa hiệp vẫn tăng đáng kể.

Megolm trong hệ sinh thái Matrix cũng theo tinh thần tối ưu việc gửi tin nhóm [8]. Có thể hiểu đơn giản rằng mỗi người gửi duy trì một phiên gửi ra, tức một trạng thái khóa được dùng liên tiếp để mã hóa nhiều tin nhắn trong cùng một phòng trò chuyện. Bên trong phiên đó, trạng thái khóa được cập nhật dần theo một cơ chế xoay khóa nhóm. Thiết kế này giúp việc gửi tin trong nhóm nhẹ hơn, nhưng điểm đổi lại là khả năng phục hồi sau thỏa hiệp không mạnh bằng những mô hình thay khóa nhóm chặt chẽ hơn. Nếu trạng thái của một phiên bị lộ, phạm vi thông điệp bị ảnh hưởng có thể kéo dài cho đến khi hệ thống tạo phiên mới và phân phối lại khóa.

Nhìn chung, Sender Keys và Megolm đều có giá trị thực tiễn cao đối với bài toán phát tán tin nhắn trong nhóm. Tuy nhiên, chúng không trực tiếp giải quyết bài toán mà đề án quan tâm nhất, đó là duy trì một trạng thái nhóm nhất quán khi nhiều thiết bị có thể cùng thay đổi trạng thái trong môi trường P2P bất đồng bộ. Đây là lý do MLS trở thành nền tảng phù hợp hơn cho đề tài.

2.2.2 MLS với Delivery Service

MLS được chuẩn hóa để cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ với các bảo đảm bảo mật mạnh hơn cho truyền thông nhóm quy mô lớn [1]. Điểm quan trọng của MLS là nó không chỉ tối ưu việc gửi thông điệp, mà còn mô hình hóa rõ quá trình thay đổi thành viên và cập nhật khóa của cả nhóm thông qua các khái niệm như proposal, commit và epoch.

Trong kiến trúc chuẩn của MLS, Delivery Service thường đảm nhiệm việc lưu trữ một số vật liệu khóa công khai ban đầu, phân phối thông điệp MLS và hỗ trợ quá trình truyền thông giữa các client [2]. Nhờ có một đầu mối như vậy, hệ thống triển khai theo mô hình Client-Server có điều kiện thuận lợi hơn để quan sát các Commit theo một thứ tự đủ nhất quán. Vì thế, MLS hoạt động tự nhiên hơn trong môi trường có DS, nhưng khi bỏ đi điểm điều phối này thì bài toán ordering của Commit lập tức trở nên khó hơn nhiều.

2.2.3 Các hướng phi tập trung hóa MLS

Vì quá trình tiến hóa trạng thái của MLS mang bản chất SMR, một hướng tiếp cận trực tiếp là bổ sung một lớp đồng thuận phân tán truyền thống để thống nhất thứ tự Commit. Các cơ chế đồng thuận này thường rơi vào hai nhóm chính: đồng thuận dựa trên số đông (*quorum-based*) hoặc đồng thuận chuỗi khối (*blockchain-based*). Tuy nhiên, cả hai hướng đi này đều gặp phải những rào cản lớn khi áp dụng vào môi trường nhắn tin P2P.

(i) Đối với đồng thuận dựa trên Quorum (diễn hình như BFT [11], Paxos [12] hay Raft [13]), các thuật toán này đòi hỏi một tỷ lệ nhất định các nút mạng (thường là lớn hơn một nửa hoặc hai phần ba) phải trực tuyến và duy trì kết nối liên tục để bầu chọn nhóm trưởng và xác nhận thứ tự thông điệp. Dưới góc độ của định lý CAP [9], các cơ chế này ưu tiên tính Nhất quán (Consistency) và hy sinh tính Khả dụng khi mạng bị chia cắt (Partition). Trong ứng dụng chat P2P, thiết bị của người dùng thường xuyên ngoại tuyến hoặc mạng chập chờn. Việc yêu cầu một quorum trực tuyến sẽ khiến hệ thống thường xuyên bị "đóng băng", người dùng không thể gửi Commit hay thay đổi thành viên nhóm. Thêm vào đó, đối với các thuật toán chịu lỗi Byzantine (BFT), độ phức tạp truyền thông có thể lên tới $\mathcal{O}(N^2)$ cho các vòng xác nhận chéo, tạo ra gánh nặng đáng kể đối với băng thông của thiết bị di động.

(ii) Đối với đồng thuận chuỗi khối (*blockchain-based*) [14], việc sử dụng blockchain để tạo ra một nhật ký tuyến tính chung giải quyết được bài toán thiếu máy chủ trung tâm. Tuy nhiên, kiến trúc này sinh ra độ trễ (latency) tạo khối rất cao, hoàn toàn không đáp ứng được yêu cầu thời gian thực của một ứng dụng trò chuyện. Ngoài ra, việc thiết lập và duy trì một mạng lưới blockchain cũng như cơ chế đồng thuận riêng cho từng nhóm chat độc lập là một sự lãng phí tài nguyên không cần thiết.

Một hướng khác là Decentralized MLS (DMLS) của Kohbrok, trong đó chính bản thân MLS được mở rộng để xử lý Commit đến ngoài thứ tự [6]. Về ý tưởng, client giữ lại thêm một phần vật liệu khóa hoặc trạng thái tạm thời để có thể xử lý những nhánh Commit khác nhau khi mạng phi tập trung không cung cấp ordering rõ ràng. Cách làm này cho thấy một hướng nghiên cứu đáng chú ý, nhưng đồng thời cũng làm tăng độ phức tạp triển khai và đặt ra thêm đánh đổi đối với Forward Secrecy [5], [6]. Hơn nữa, hướng này hiện vẫn ở mức Internet-Draft, chưa trở thành chuẩn hoàn chỉnh.

Từ các hướng trên có thể thấy khoảng trống còn lại nằm ở chỗ sau: cần một cơ chế đủ nhẹ để phù hợp với P2P, nhưng vẫn đủ chặt để giảm nguy cơ rẽ nhánh và hỗ trợ hệ thống hội tụ trở lại sau khi mạng bị chia cắt. Đó cũng là động lực trực tiếp cho hướng tiếp cận mà đề án theo đuổi ở các chương sau.

2.3 Nền tảng MLS và TreeKEM

2.3.1 Các khái niệm cốt lõi: client, group, member và epoch

Trong MLS, client là thực thể nắm giữ khóa mật mã và tham gia vào tiến trình thiết lập trạng thái nhóm. Group là một nhóm logic gồm các client cùng chia sẻ một trạng thái mật mã tại một thời điểm. Member là client đang thực sự nằm trong trạng thái chia sẻ đó và có quyền truy cập các bí mật của nhóm. Epoch, như đã

trình bày ở trên, là một phiên bản cụ thể của trạng thái nhóm [1].

Khi muốn tham gia nhóm, client cần công bố một *KeyPackage* — cấu trúc chứa khóa công khai, chứng chỉ định danh và các bộ mã mà client hỗ trợ [1]. Thành viên trong nhóm dùng *KeyPackage* này để tạo *Proposal Add* đưa client vào nhóm.

Điểm quan trọng cần nhấn mạnh là trạng thái nhóm trong MLS không phải một tập dữ liệu rời rạc có thể ghép lại tùy ý. Nó là một chuỗi epoch nối tiếp nhau, trong đó mỗi epoch mới được sinh ra từ epoch trước đó. Vì vậy, nếu các thành viên không cùng nhìn nhận một chuỗi epoch tương thích, nhóm sẽ mất khả năng duy trì một trạng thái mật mã chung.

2.3.2 Ratchet Tree và TreeKEM

TreeKEM là cơ chế trung tâm giúp MLS cập nhật khóa nhóm hiệu quả. Thay vì để mỗi thành viên phải trao đổi khóa riêng với tất cả các thành viên còn lại, MLS tổ chức các thành viên thành các lá của một cây nhị phân, gọi là Ratchet Tree. Có thể hiểu trực quan rằng mỗi lần nhóm thay đổi, hệ thống chỉ cần cập nhật một số bí mật dọc theo một đường trong cây, thay vì phát lại toàn bộ khóa cho cả nhóm [1].

Khi một thành viên cập nhật khóa, thành viên đó tạo ra một đường dẫn cập nhật từ lá của mình lên gốc cây. Các bí mật mới trên đường đi này được mã hóa cho các nhánh đồng cấp tương ứng, thường được gọi là *copath*, bằng HPKE (Hybrid Public Key Encryption) [15] — cơ chế mã hóa bất đối xứng mà MLS dùng làm nền tảng cho TreeKEM. Nhờ cấu trúc cây, số lượng phép tính và lượng dữ liệu cập nhật chỉ tăng gần theo logarit của số thành viên trong nhóm. Nói ngắn gọn, TreeKEM là cơ chế giúp MLS thay khóa nhóm theo cách có cấu trúc và tiết kiệm hơn so với việc phân phối lại khóa theo kiểu tuyến tính cho toàn bộ thành viên.

Cần lưu ý rằng bí mật ở gốc cây không được dùng trực tiếp để mã hóa thông điệp ứng dụng. Sau mỗi Commit, MLS còn đưa các bí mật mới qua một cơ chế sinh khóa để tạo ra các khóa cụ thể cho từng mục đích như xác thực, mã hóa thông điệp hay xuất vật liệu khóa [1]. Vì vậy, TreeKEM nên được hiểu là cơ chế cập nhật trạng thái khóa nhóm, còn việc sinh các khóa sử dụng trực tiếp cho thông điệp được xử lý ở bước sau.

2.3.3 Proposal, Commit và tiến trình chuyển epoch

MLS phân biệt rõ proposal và commit. Proposal là thông điệp đề xuất một thay đổi cho nhóm, chẳng hạn thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa. Bản thân proposal chưa làm thay đổi epoch. Commit mới là thông điệp thực sự áp dụng một tập proposal và đưa nhóm sang epoch mới [1].

Nhìn dưới góc độ hệ phân tán, có thể xem Commit như thao tác ghi vào trạng

thái mật mã của nhóm. Mỗi Commit đều làm xuất hiện một phiên bản trạng thái mới. Vì thế, nếu tại cùng một epoch mà có nhiều Commit cạnh tranh, nguy cơ rẽ nhánh trạng thái sẽ xuất hiện. Đây là lý do bài toán ordering của Commit luôn giữ vai trò trung tâm khi vận hành MLS trên mạng ngang hàng.

2.3.4 Các dấu hiệu nhận biết trạng thái: tree hash, transcript hash và epoch authenticator

Để kiểm tra các thành viên có còn cùng một trạng thái hay không, MLS sử dụng một số giá trị ràng buộc trạng thái [1]. Trong đó, hàm băm cây (tree hash) phản ánh cấu trúc Ratchet Tree hiện tại. Hàm băm lịch sử bắt tay (transcript hash) tóm tắt lịch sử các bản tin điều khiển của MLS đã được nhóm chấp nhận, chẳng hạn như Proposal và Commit, nhờ đó cho biết các thành viên có thực sự đi qua cùng một chuỗi thay đổi trạng thái hay không. Còn bộ xác thực epoch (epoch authenticator) có thể được dùng để đối chiếu việc các thành viên có thực sự đang chia sẻ cùng trạng thái epoch hay không.

Ý nghĩa của các giá trị này đối với đồ án là rất trực tiếp. Trong môi trường P2P, hai thiết bị có thể cùng báo cùng mã nhóm và cùng số epoch, nhưng điều đó chưa đủ để kết luận rằng chúng còn ở cùng trạng thái MLS. Nếu tree hash hoặc transcript hash khác nhau, về bản chất chúng đã tách thành hai nhánh. Vì vậy, khi nghiên cứu bài toán phát hiện phân nhánh và đồng bộ lại trạng thái, các dấu hiệu nhận biết này đóng vai trò đặc biệt quan trọng.

Các thành phần nêu trên — Ratchet Tree, GroupContext (gồm mã nhóm, số epoch, tree hash, transcript hash và các tiện ích mở rộng), key schedule (chuỗi dẫn xuất khóa giữa các epoch) và secret tree (cây khóa mã hóa cho thông điệp ứng dụng) — hợp lại thành trạng thái đầy đủ của một nhóm MLS tại một epoch, thường được gọi gọn là *group state* (trạng thái nhóm) [1]. Đây là toàn bộ dữ liệu mật mã mà một client cần lưu trữ để có thể mã hóa, giải mã và xử lý các thao tác thay đổi thành viên trong epoch hiện tại. Khi cần truyền hoặc lưu trữ, group state được tuần tự hóa thành một khối dữ liệu có kích thước tỉ lệ với số thành viên và số epoch đã trải qua. Đặc biệt, mỗi thao tác cập nhật khóa tạo ra cặp khóa HPKE mới cho nút lá trong Ratchet Tree, làm cho group state phình to dần qua từng epoch. Khía cạnh này sẽ được thảo luận chi tiết hơn trong Chương 5 khi đánh giá chi phí phục hồi sau phân vùng mạng.

2.3.5 Application messages, delayed messages và External Commit

MLS phân biệt thông điệp bắt tay với thông điệp ứng dụng [1]. Proposal và Commit thuộc nhóm thứ nhất vì chúng làm thay đổi hoặc chuẩn bị thay đổi trạng thái nhóm. Ngược lại, application messages là các tin nhắn ứng dụng được mã hóa

bằng khóa sinh ra từ trạng thái của epoch hiện tại. Cụ thể, MLS dùng cơ chế mã hóa có xác thực và dữ liệu liên kết (Authenticated Encryption with Associated Data – AEAD) [16] để bảo vệ nội dung tin nhắn. AEAD vừa mã hóa bản rõ để chống nghe lén, vừa kèm theo một mã xác thực để người nhận phát hiện nếu tin nhắn bị sửa đổi trong quá trình truyền. Nhờ đó, ngay cả khi tin nhắn đi qua nhiều nút trung gian trong mạng P2P, chỉ những thành viên đang giữ khóa của epoch tương ứng mới có thể giải mã và xác thực nội dung.

Trong mạng thực tế, thông điệp ứng dụng có thể đến muộn hoặc đến sai thứ tự. MLS có cơ chế hỗ trợ một mức độ nhất định cho các thông điệp bị trễ, chẳng hạn thông qua bộ khóa của người gửi và bộ đếm thế hệ. Tuy nhiên, để bảo vệ Forward Secrecy, thiết bị không nên giữ các khóa cũ vô thời hạn. Điều này tạo ra một đánh đổi quen thuộc: nếu xóa khóa cũ quá sớm thì thông điệp đến muộn có thể không giải mã được; nếu giữ khóa cũ quá lâu thì phạm vi thiệt hại khi thiết bị bị thỏa hiệp sẽ tăng [1].

MLS cũng hỗ trợ cơ chế gia nhập từ bên ngoài, thường được gọi là External Commit hoặc External Join [1]. Có thể hiểu đây là cách để một client chưa ở trong trạng thái hiện tại của nhóm gia nhập lại nhóm khi có đủ thông tin công khai cần thiết. Cơ chế này quan trọng đối với đồ án vì khi trạng thái đã rẽ nhánh, một thiết bị có thể cần từ bỏ nhánh cũ và tham gia lại nhánh đang được chọn.

Bên cạnh External Commit, RFC 9420 còn định nghĩa một cơ chế khác cho một đối tượng ngoài nhóm tác động lên trạng thái nhóm, gọi là External Proposal [1]. Hai cơ chế này có đặc tính khác nhau về ai giữ quyền tạo Commit. External Commit là thao tác mà chính node ngoài tự tạo Commit để gia nhập, nên node đó tự chịu trách nhiệm sinh ra epoch mới. External Proposal thì khác: node ngoài chỉ gửi một đề nghị thay đổi, thường là Add, vào nhóm, còn việc đưa đề nghị đó vào Commit thuộc về một thành viên đang ở trong nhóm. Cả hai đều là cơ chế hợp lệ của RFC 9420, nhưng cách phân chia trách nhiệm khác nhau dẫn đến hệ quả khác nhau khi áp dụng vào môi trường P2P.

Một vấn đề khác cần lưu ý là định danh trùng khi gia nhập lại. Khi một node cũ gia nhập lại nhóm, định danh cũ của node đó vẫn còn nằm trong cây MLS dưới dạng một lá không còn hoạt động. Nếu node đó dùng External Commit với cùng định danh, lỗi MLS sẽ từ chối vì phát hiện trùng định danh. Cách giải quyết là kết hợp Remove, loại bỏ lá cũ, và Add, thêm lại với KeyPackage mới, trong cùng một Commit. Khả năng gom nhiều Proposal vào một Commit là một cơ chế được cung cấp sẵn của MLS, cho phép nhiều thay đổi thành viên được xử lý trong cùng một epoch.

2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán

2.4.1 Đặc tính của mạng ngang hàng

Mạng ngang hàng khác với mô hình Client-Server ở chỗ không có một máy chủ trung tâm duy nhất điều phối toàn bộ luồng truyền thông. Các thiết bị có thể tự phát hiện nhau, thiết lập kết nối trực tiếp hoặc gián tiếp và trao đổi dữ liệu qua nhiều đường truyền khác nhau. Cách tổ chức này giúp hệ thống giảm phụ thuộc vào hạ tầng trung tâm, nhưng đồng thời làm cho việc điều phối trở nên khó hơn.

Trong môi trường P2P, các hiện tượng như thiết bị ngoại tuyến, độ trễ không đồng đều, trùng lặp thông điệp, biến động nút mạng hoặc phân vùng mạng là điều bình thường. Với ứng dụng thông thường, chúng chủ yếu gây chậm hoặc mất dữ liệu tạm thời. Với MLS, các hiện tượng đó còn có thể kéo theo hệ quả nghiêm trọng hơn: một số thiết bị đã chuyển sang epoch mới trong khi thiết bị khác vẫn ở epoch cũ, hoặc hai phân vùng mạng cùng tạo ra những Commit khác nhau. Điều này cho thấy lớp mạng P2P chỉ giải quyết việc truyền thông, chứ không tự giải quyết bài toán nhất quán trạng thái mật mã.

2.4.2 go-libp2p và GossipSub

Trong ứng dụng thử nghiệm của đồ án, tầng mạng được xây dựng trên go-libp2p [17]. Đây là thư viện P2P mô-đun hóa, cho phép mỗi thiết bị có một định danh mạng riêng, thiết lập kết nối qua nhiều cơ chế truyền dẫn và trao đổi dữ liệu trên nhiều luồng logic. Những khả năng này phù hợp với yêu cầu của một ứng dụng nhắn tin không phụ thuộc hoàn toàn vào máy chủ trung tâm.

Một thành phần quan trọng của libp2p là GossipSub [18], tức cơ chế phát tán thông điệp theo mô hình *publish/subscribe*. Có thể hiểu đơn giản rằng khi một thiết bị gửi thông điệp vào một chủ đề chung, thông điệp đó sẽ được lan truyền dần qua các thiết bị lân cận trong mạng. Cơ chế này phù hợp để phát tán proposal, commit hoặc các thông báo trạng thái trong môi trường P2P.

Tuy nhiên, GossipSub chỉ là cơ chế phát tán thông điệp, không phải cơ chế đồng thuận. Nó không bảo đảm mọi thiết bị nhận thông điệp theo cùng một thứ tự, không tự chọn Commit thắng cuộc khi có xung đột, và cũng không bảo đảm một thông điệp chỉ được nhận đúng một lần. Vì vậy, chỉ đưa MLS lên trên GossipSub là chưa đủ để bảo vệ chuỗi trạng thái của nhóm.

2.4.3 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng

Trong cùng một epoch, nhiều thành viên có thể gửi thông điệp ứng dụng song song. Những thông điệp này không nhất thiết cần một thứ tự mật mã toàn cục như Commit, nhưng giao diện chat vẫn cần một thứ tự hiển thị đủ ổn định giữa các thiết

bị. Nếu chỉ dựa vào đồng hồ vật lý, hệ thống dễ gặp sai lệch do lệch giờ hoặc do môi trường mạng không ổn định.

Vì vậy, hệ phân tán thường sử dụng các dạng đồng hồ logic để biểu diễn quan hệ thứ tự giữa các sự kiện. Lamport Clock cho một thứ tự logic đơn giản [19]. Hybrid Logical Clock (HLC) kết hợp thời gian vật lý và bộ đếm logic để tạo ra dấu thời gian gần với thời gian thực hơn, nhưng vẫn ổn định hơn khi đồng hồ giữa các thiết bị không hoàn toàn khớp nhau [20]. Trong đề án, loại dấu thời gian này chỉ có ý nghĩa đối với việc sắp xếp hiển thị các thông điệp ứng dụng; nó không thay thế epoch, chữ ký hay các cơ chế bảo vệ trạng thái của MLS.

2.4.4 Khoảng trống cần giải quyết

Từ các phân tích trên có thể thấy bài toán của đề tài nằm ở giao điểm giữa mật mã học nhóm và hệ phân tán. MLS cung cấp lõi mật mã mạnh nhưng giả định hệ thống triển khai phải giải quyết việc phân phối và ordering của thông điệp. Libp2p và GossipSub cung cấp hạ tầng truyền thông ngang hàng nhưng không cung cấp thứ tự tuyến tính cho Commit. HLC hỗ trợ sắp xếp thông điệp ứng dụng nhưng không bảo vệ trạng thái mật mã của nhóm [2], [17], [20].

Khoảng trống nghiên cứu vì vậy không nằm ở chỗ thiếu một giao thức mã hóa nhóm, mà nằm ở chỗ thiếu một cơ chế điều phối đủ nhẹ để phù hợp với P2P, nhưng vẫn đủ chặt để giảm nguy cơ rẽ nhánh và hỗ trợ hội tụ trở lại sau khi mạng bất ổn. Khoảng trống đó là cơ sở trực tiếp cho giải pháp sẽ được trình bày ở Chương 3.

Chương 2 đã làm rõ ngữ cảnh lý thuyết của bài toán vận hành MLS trên mạng ngang hàng. Trước hết, chương chỉ ra rằng khó khăn trung tâm của đề tài không chỉ là truyền tin trong môi trường P2P, mà là duy trì một trạng thái mật mã chung cho toàn nhóm khi không còn điểm tuần tự hóa trung tâm. Tiếp theo, chương khảo sát các hướng tiếp cận liên quan, từ Sender Keys, Megolm đến MLS trong mô hình có Delivery Service và các nỗ lực phi tập trung hóa MLS, qua đó làm nổi bật khoảng trống nghiên cứu mà đề án hướng tới.

Bên cạnh đó, chương cũng trình bày các kiến thức nền tảng cần thiết để hiểu phần phương pháp ở các chương sau, bao gồm epoch, proposal, commit, TreeKEM, các dấu hiệu nhận biết trạng thái và cơ chế gia nhập lại nhóm. Cuối cùng, chương phân tích ngắn gọn các đặc tính của mạng ngang hàng, vai trò của libp2p, GossipSub và đồng hồ logic trong việc hỗ trợ truyền thông và sắp xếp thông điệp ứng dụng. Những nội dung này tạo tiền đề trực tiếp cho Chương 3, nơi giải pháp điều phối phi tập trung của đề án sẽ được trình bày chi tiết.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

Tổng quan chương.

Chương 2 đã trình bày nền tảng của MLS, TreeKEM, mạng ngang hàng và nguyên nhân dẫn tới phân nhánh trạng thái mật mã khi nhiều Commit cùng xuất hiện trong một Epoch. Trên cơ sở đó, Chương 3 tập trung vào lớp điều phối phi tập trung mà đề án đề xuất để MLS có thể vận hành trong môi trường P2P mà không cần một Delivery Service trung tâm sắp thứ tự Commit.

Chương này trình bày phương pháp đề xuất ở mức giao thức, bao gồm: bài toán và yêu cầu thiết kế, cơ chế Single-Writer theo epoch, cơ chế kiểm tra nhất quán epoch, cơ chế phát hiện và Fork Healing, và cách dùng đồng hồ logic lai để sắp xếp thông điệp ứng dụng. Cuối chương có một mục ngắn nói về cách ứng dụng thử nghiệm triển khai các nguyên tắc này; phần kiểm chứng và ảnh minh họa nằm ở Chương 5 và phụ lục.

3.1 Tổng quan bài toán và hướng tiếp cận

3.1.1 Bài toán cần giải quyết

Trong mô hình MLS thông thường, Delivery Service không chỉ chuyển tiếp thông điệp mà còn giúp các thành viên nhìn thấy cùng một thứ tự Commit [1], [2]. Khi chuyển sang môi trường P2P và bỏ điểm tuần tự hóa trung tâm này, hệ thống phải đối mặt với một rủi ro trực tiếp: hai hoặc nhiều thành viên có thể cùng phát hành Commit cho cùng một Epoch. Khi đó, nhóm không còn tiến hóa theo một chuỗi trạng thái duy nhất, mà có thể tách thành nhiều nhánh trạng thái mật mã khác nhau.

Giả sử nhóm đang ở Epoch E . Nếu thành viên A tạo Commit C_A và thành viên B đồng thời tạo Commit C_B , một phần nút có thể áp dụng C_A trước, trong khi phần còn lại áp dụng C_B trước. Kết quả là nhóm có thể sinh ra hai trạng thái $S_A(E + 1)$ và $S_B(E + 1)$ khác nhau dù cùng mang nhãn Epoch $E + 1$. Khi đó, các nút ở hai nhánh không còn cùng tree hash, cùng lịch sử Commit hay cùng khóa ứng dụng, nên cũng không còn xử lý thông điệp của nhau một cách hợp lệ [5], [6].

Vì vậy, phương pháp đề xuất trong đề án phải đồng thời đáp ứng bốn yêu cầu: (i) giảm mạnh khả năng xuất hiện concurrent commits khi mạng còn liên thông; (ii) buộc mỗi nút chỉ xử lý thông điệp trên trạng thái MLS tương thích với epoch cục bộ của nó; (iii) phát hiện được sự phân kỳ trạng thái khi phân vùng mạng xảy ra và hỗ trợ hội tụ trở lại một nhánh hợp lệ; (iv) giữ được một thứ tự hiển thị ổn định cho các thông điệp ứng dụng trong cùng một Epoch mà không nhằm lẫn vai

trò đó với thứ tự Commit.

3.1.2 Kiến trúc của giải pháp

Về kiến trúc, phương pháp chia thành hai tầng. Tầng điều phối quyết định khi nào một nút được quyền tạo Commit, cách phân loại thông điệp theo epoch, cách đối chiếu trạng thái nhánh và cách khôi phục sau phân vùng mạng. Tầng MLS lo việc xử lý Proposal, Commit, Welcome, External Proposal, External Commit, mã hóa và giải mã thông điệp theo đúng quy tắc của MLS [1].

Hình 3.1 cho thấy cách phân chia trách nhiệm này: lớp điều phối không lập trình lại MLS, còn lõi MLS không tự quyết định thứ tự điều phối trong môi trường phân tán. Nhờ cách tách này, đề án giữ nguyên các bảo đảm mật mã của MLS và chỉ bổ sung lớp điều phối cho bối cảnh P2P.

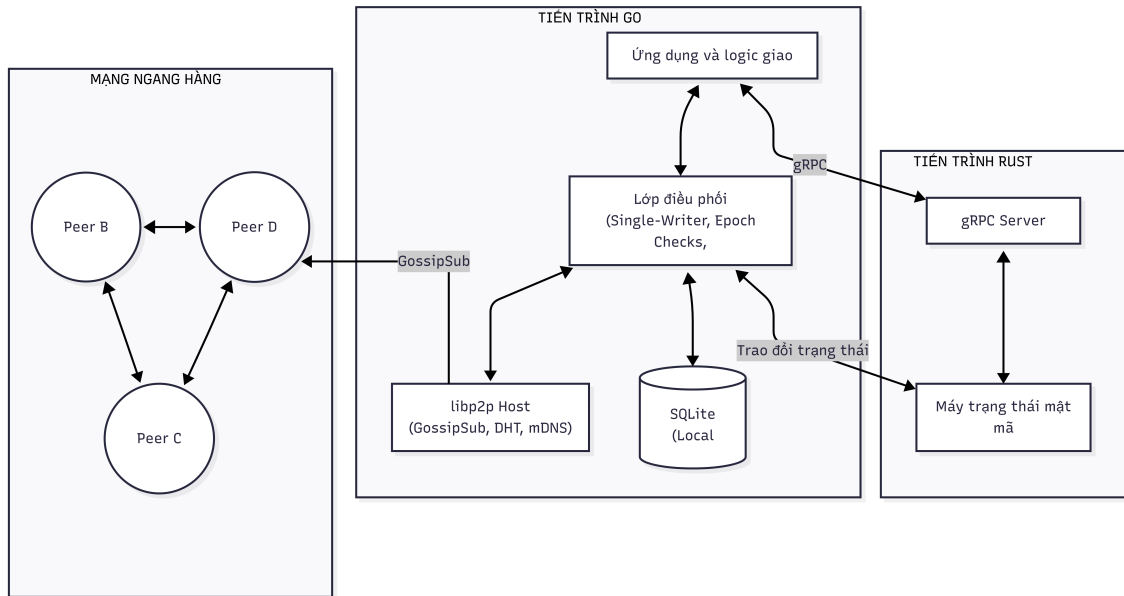


Figure 3.1: Kiến trúc logic của giải pháp và ranh giới trách nhiệm giữa lớp điều phối với lớp MLS

3.1.3 Các cơ chế chính của phương pháp

Phương pháp đề xuất gồm bốn cơ chế liên kết với nhau: (1) Single-Writer theo epoch để giới hạn quyền phát hành Commit; (2) kiểm tra nhất quán epoch để ngăn nút xử lý thông điệp trên trạng thái MLS không tương thích; (3) phát hiện và Fork Healing để đưa các nút hội tụ lại sau phân vùng mạng; (4) đồng hồ logic lai HLC để sắp xếp thông điệp ứng dụng sau khi đã qua lớp kiểm tra của MLS.

Bốn cơ chế này không thay thế MLS mà chỉ bao quanh nó, bù vào vai trò mà Delivery Service thường đảm nhiệm trong mô hình tập trung. Đóng góp của đề án là cách tổ chức lớp điều phối chứ không phải sửa đổi thuật toán mật mã cốt lõi.

3.1.4 Mối liên hệ giữa thách thức, câu hỏi nghiên cứu và cơ chế

Để làm rõ mạch lập luận từ bài toán đến giải pháp, Bảng 3.1 tóm tắt mối liên hệ giữa các thách thức chính, câu hỏi nghiên cứu tương ứng và cơ chế mà đề án đề xuất. Bốn cơ chế này không hoạt động độc lập mà phối hợp theo một luồng xử lý thống nhất: thông điệp đến từ mạng ngang hàng trước hết được kiểm tra tính tương thích epoch, sau đó chỉ những Commit do Token Holder phát hành mới được chấp nhận, khi phát hiện phân nhánh qua đối chiếu trạng thái định kỳ thì Fork Healing đưa các nút về lại một nhánh chung, và cuối cùng HLC sắp xếp thứ tự hiển thị cho các tin nhắn ứng dụng đã giải mã hợp lệ. Sơ đồ tổng quan ở Hình 3.2 minh họa cách bốn cơ chế bao quanh lõi MLS và phối hợp với nhau, bổ sung cho Hình 3.1 vốn chỉ cho thấy ranh giới kiến trúc hai tầng.

Table 3.1: Mối liên hệ giữa thách thức, câu hỏi nghiên cứu và cơ chế đề xuất

Thách thức	Câu hỏi nghiên cứu	Cơ chế đề xuất
Nhiều thành viên cùng tạo Commit trong một epoch khi không có Delivery Service	Làm thế nào để giảm xung đột Commit khi mạng còn liên thông?	Single-Writer theo epoch: bầu Token Holder xác định trước
Thông điệp MLS đến trễ hoặc sai thứ tự, có thể thuộc epoch cũ	Làm thế nào để đảm bảo trạng thái cục bộ phù hợp trước khi xử lý thông điệp?	Kiểm tra nhất quán epoch
Phân vùng mạng dẫn đến phân nhánh trạng thái mật mã	Làm thế nào để phát hiện phân nhánh và đưa hệ thống trở lại một nhánh hợp lệ?	Phát hiện phân nhánh bằng history hash và Fork Healing bằng External Proposal
Thứ tự nhận tin nhắn ứng dụng không đồng nhất giữa các nút	Làm thế nào để giữ thứ tự hiển thị tin nhắn ổn định?	Đồng hồ logic lai (HLC)

3.1.5 Phạm vi bảo mật và Mô hình mối đe dọa (Threat Model)

Vì đề án sử dụng mạng P2P thay cho máy chủ trung tâm, rủi ro bảo mật thực chất lại mở rộng hơn do mọi nút trong mạng đều có thể tham gia định tuyến tin nhắn và tiếp cận siêu dữ liệu (metadata). Để làm rõ ranh giới giữa an toàn mật mã và đồng thuận phân tán, đề án xác định mô hình mối đe dọa gồm ba đối tượng chính:

- **Kẻ nghe lén (Passive Eavesdropper):** Bất kỳ nút P2P trung gian nào cố tình theo dõi luồng dữ liệu truyền qua. Rủi ro này được ủy thác hoàn toàn cho lớp mật mã: hệ thống chống lại bằng cơ chế mã hóa có xác thực (AEAD) của MLS [16], đảm bảo trung gian tuyệt đối không thể truy cập bản rõ.
- **Thành viên bị chiếm quyền (Compromised Member):** Một thành viên hợp lệ trong nhóm bị kẻ tấn công đánh cắp thiết bị hoặc lộ vật liệu khóa. Hệ thống kế thừa và chống lại rủi ro này bằng các thuộc tính Forward Secrecy (FS) và

Post-Compromise Security (PCS) mặc định của cơ chế TreeKEM trong MLS [1].

- **Kẻ phá hoại mạng (Active Attacker / Malicious Node):** Nút mạng cố tình giữ lại tin nhắn, phát tán thông điệp sai thứ tự, hoặc cố ý tạo phân nhánh nhằm chia rẽ sự đồng thuận của nhóm. Đây chính là trọng tâm mà lớp điều phối phi tập trung của đề án trực tiếp giải quyết thông qua cơ chế bầu Token Holder (Single-Writer) và quá trình hàn gắn phân nhánh (Fork Healing).

Nói cách khác, đề án giả định các cấu trúc mật mã của MLS là an toàn. Bài toán bảo mật được giải quyết bằng việc tích hợp chuẩn này; phần việc của đề án là giữ cho hệ thống không bị sụp đổ hay phân nhánh khi mạng ngang hàng hoạt động bất đồng bộ.

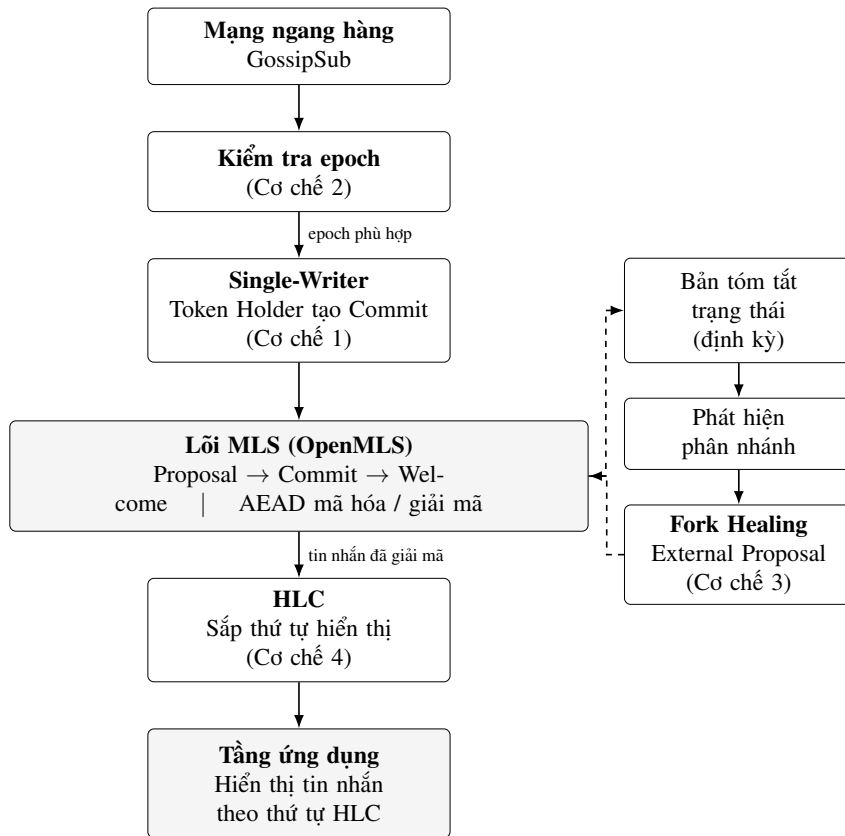


Figure 3.2: Sơ đồ tổng quan luồng xử lý: bốn cơ chế điều phối bao quanh lõi MLS. Mũi tên liền là luồng xử lý chính, mũi tên đứt là luồng phục hồi định kỳ

3.2 Giao thức Single-Writer theo epoch

3.2.1 Mục tiêu thiết kế

Tiến hóa trạng thái nhóm về bản chất là một bài toán Máy trạng thái nhân bản (SMR) [10], nên thiết kế giao thức bị chi phối bởi các đánh đổi của định lý CAP [9]. Nếu cố đạt tính Nhất quán mạnh như các thuật toán đồng thuận truyền thống, hệ thống nhắn tin P2P sẽ mất tính Khả dụng mỗi khi mạng bị chia cắt.

Đồ án chọn hướng ưu tiên tính Khả dụng và chịu đựng phân vùng mạng (AP), kết hợp với mô hình Nhất quán cuối cùng [21]. Nguyên nhân trực tiếp của phân nhánh là nhiều Commit hợp lệ cùng xuất hiện trong một epoch, nên đồ án kiểm soát bằng cách chỉ cho một thành viên duy nhất phát hành Commit. Nếu mạng bị chia cắt và thông tin về Token bị phân mảnh, sinh ra nhiều nhánh Commit độc lập, hệ thống cho phép các nhóm con tiếp tục hoạt động, sau đó dùng cơ chế hàn gắn để đưa nhóm về một trạng thái chung khi mạng ổn định lại.

Ý tưởng trung tâm của cơ chế này là tách vai trò Proposal và Commit. Thành viên hợp lệ nào cũng có thể phát biểu mong muốn thay đổi nhóm dưới dạng Proposal, nhưng chỉ một thành viên duy nhất trong tập ứng viên hợp lệ mới được quyền gom các Proposal đó để tạo Commit. Thành viên này được gọi là Token Holder của Epoch đang xét.

3.2.2 Tập ứng viên hợp lệ

Token Holder không được chọn từ toàn bộ các nút mà hệ thống nhìn thấy trên mạng, mà chỉ được chọn từ tập ứng viên hợp lệ của nhóm. Tập này được xác định bởi ba loại điều kiện: tư cách thành viên trong trạng thái MLS, tình trạng còn hoạt động trên mạng và chính sách quyền của ứng dụng. Ở mức khái niệm, tập ứng viên tại Epoch E được mô tả như sau:

$$Eligible(E) = \mathcal{M}(E) \cap \mathcal{A}(E) \cap \mathcal{C}(E) \setminus \mathcal{S}(E) \quad (3.1)$$

Trong đó, $\mathcal{M}(E)$ là tập thành viên mà trạng thái MLS còn xem là hợp lệ ở Epoch E ; $\mathcal{A}(E)$ là tập thành viên đang được quan sát là còn hoạt động; $\mathcal{C}(E)$ là tập thành viên mà chính sách ứng dụng cho phép tham gia tạo Commit; còn $\mathcal{S}(E)$ là tập thành viên tạm thời bị loại khỏi vai trò điều phối vì lỗi, ngoại tuyến quá lâu hoặc bị chính sách ứng dụng đình chỉ.

Cách mô hình hóa này giúp phân biệt rõ hai chuyện thường bị trộn lẫn. Một thành viên có thể vẫn còn trong nhóm ở tầng MLS nhưng đang ngoại tuyến, nên không thích hợp giữ vai trò Token Holder. Ngược lại, một nút đang trực tuyến nhưng không thuộc trạng thái nhóm cũng không có quyền tham gia quyết định Commit. Do đó, chỉ phần giao của các điều kiện trên mới có ý nghĩa đối với bài toán điều phối.

3.2.3 Hàm bầu chọn Token Holder

Sau khi xác định được tập ứng viên hợp lệ, mỗi nút tự tính Token Holder bằng cùng một quy tắc xác định trước cục bộ. Thành viên có giá trị băm nhỏ nhất khi ghép định danh nhóm, số epoch và định danh nút sẽ được chọn:

$$TH(E) = \operatorname{argmin}_{n \in \text{Eligible}(E)} H(\text{group_id} \parallel E \parallel n) \quad (3.2)$$

Ở đây, H là hàm băm mật mã như SHA-256. Việc đưa group_id vào đầu vào giúp một nút không mặc nhiên giữ ưu thế ở mọi nhóm; việc đưa E vào đầu vào giúp quyền giữ Token thay đổi theo từng epoch; còn định danh nút bảo đảm mọi nút đang có cùng ActiveView sẽ đi tới cùng kết quả. Thuật toán bầu chọn được mô tả trong Thuật toán 1.

Algorithm 1: Thuật toán bầu chọn Token Holder (*ComputeTokenHolder*)

Đầu vào: groupID : định danh nhóm; epoch : Epoch hiện tại; members : tập thành viên MLS; activeView : tập thành viên đang hoạt động; authorized : tập thành viên có quyền tạo Commit; suspended : tập thành viên bị đình chỉ tạm thời.

Đầu ra : holderID : định danh Token Holder được chọn.

```

 $\text{eligible} \leftarrow \text{members} \cap \text{activeView} \cap \text{authorized} \setminus \text{suspended};$ 
if  $\text{eligible}$  là tập rỗng then
    | return Lỗi NoEligibleCommitter;
end
 $\text{bestID} \leftarrow \text{null};$ 
 $\text{bestHash} \leftarrow \text{MAX\_HASH};$ 
for mỗi  $\text{peerID}$  trong  $\text{eligible}$  do
    |  $\text{data} \leftarrow \text{Encode}(\text{groupID}, \text{epoch}, \text{peerID});$ 
    |  $\text{candidateHash} \leftarrow \text{SHA256}(\text{data});$ 
    | if  $\text{candidateHash} < \text{bestHash}$  then
    | |  $\text{bestHash} \leftarrow \text{candidateHash};$ 
    | |  $\text{bestID} \leftarrow \text{peerID};$ 
    | end
end
return  $\text{bestID};$ 

```

Ưu điểm chính của cách chọn này là không cần thêm một vòng bỏ phiếu mạng cho mỗi Commit. Nếu các nút có cùng trạng thái nhóm và cùng ActiveView, chúng sẽ tự tính ra cùng một Token Holder. Chi phí của bước bầu chọn vì thế chủ yếu là chi phí cục bộ, tăng tuyến tính theo số lượng ứng viên hợp lệ.

3.2.4 Tạo Proposal và Commit

Trong cơ chế đề xuất, Proposal là yêu cầu thay đổi nhóm, còn Commit là thao tác thực sự làm nhóm chuyển sang Epoch mới [1]. Tất cả thành viên hợp lệ đều có thể tạo Proposal. Tuy nhiên, chỉ Token Holder của Epoch hiện tại mới có quyền gom các Proposal hợp lệ để phát hành Commit.

Quy tắc này cho phép hệ thống chấp nhận nhiều ý định thay đổi cùng lúc nhưng vẫn duy trì một nút duy nhất cho mỗi epoch được quyền Commit. Khi mạng còn

liên thông, các nút sẽ có cùng Token Holder và vì thế giảm mạnh khả năng sinh concurrent commits. Quy trình tạo Commit của Token Holder được mô tả trong Thuật toán 2.

Algorithm 2: Quy trình tạo Commit của Token Holder
(*CommitByTokenHolder*)

Đầu vào: *localState*: trạng thái MLS cục bộ tại Epoch E ; *proposalQueue*: hàng đợi Proposal hợp lệ; *holderID*: Token Holder tại Epoch E ; *selfID*: định danh nút cục bộ.

Đầu ra : Bản tin Commit được phát tán nếu nút cục bộ là Token Holder.

if $selfID \neq holderID$ **then**

 | Kết thúc;

end

Chọn tập Proposal hợp lệ P từ *proposalQueue*;

if P rỗng và không có nhu cầu cập nhật cục bộ **then**

 | Kết thúc;

end

Tạo Commit từ *localState* và P bằng lớp MLS;

if Commit hợp lệ **then**

 | Phát tán Commit cho toàn nhóm;

end

else

 | Loại bỏ các Proposal không còn hợp lệ và ghi nhận lỗi;

end

Trong nguyên lý này, MLS chịu trách nhiệm kiểm tra tính hợp lệ mật mã của Commit, còn lớp điều phối chịu trách nhiệm quyết định ai được quyền phát hành Commit ở Epoch đó. Việc tách ranh giới như vậy giúp giữ rõ trách nhiệm của từng tầng.

3.2.5 Chuyển quyền và failover

Sau mỗi Commit hợp lệ, nhóm chuyển sang Epoch $E + 1$ và mọi nút tự tính lại Token Holder từ trạng thái mới. Vì quy tắc chọn phụ thuộc vào epoch, quyền phát hành Commit có thể luân chuyển giữa các thành viên thay vì gán cố định vào một nút.

Trong mạng P2P, Token Holder có thể ngoại tuyến hoặc không kịp phát hành Commit. Để tránh bế tắc, các nút duy trì một danh sách các thành viên được ghi nhận là đang hoạt động trong nhóm, gọi là ActiveView. Nếu Token Holder hiện tại không còn được quan sát là đang hoạt động hoặc vượt quá thời gian chờ hợp lý, các nút sẽ loại nó khỏi tập ứng viên của Epoch đó và tính lại Token Holder.

Failover này không phải là cơ chế đồng thuận Byzantine đầy đủ. Khi mạng bị

chia cắt, các phân vùng có thể có *ActiveView* khác nhau và tính ra các Token Holder khác nhau. Vai trò của Single-Writer là giảm nguy cơ concurrent commits khi các nút còn chung *ActiveView*, chứ không loại bỏ hoàn toàn khả năng fork trong mọi điều kiện mạng.

3.3 Kiểm tra nhất quán Epoch

3.3.1 Mục tiêu thiết kế

MLS yêu cầu mỗi Commit và thông điệp liên quan phải được xử lý trên đúng trạng thái nhóm tương ứng [1]. Nếu nút áp dụng bản tin lên trạng thái không tương thích, kết quả có thể là lỗi giải mã, lỗi xác thực, hoặc làm hỏng tiến trình tiến hóa trạng thái cục bộ. Vì vậy, đề án đặt một lớp kiểm tra ngoài MLS để phân loại thông điệp trước khi chuyển xuống tầng mật mã.

Trong mạng P2P bất đồng bộ, thông điệp có thể đến chậm, đến sớm, bị lặp hoặc sai thứ tự. Lớp điều phối cần xác định bản tin thuộc epoch hiện tại, quá khứ hay tương lai trước khi xử lý. Sau khi phân loại, nút mới quyết định chuyển bản tin xuống MLS, từ chối, hay đệm lại để đồng bộ.

3.3.2 Cấu trúc bản tin điều phối

Để phân loại thông điệp, đề án dùng một cấu trúc bản tin điều phối như trong Mã nguồn 3.1. Đây chỉ là mô hình khái niệm để giải thích vai trò từng trường thông tin; ứng dụng thử nghiệm có thể mã hóa khác đi miễn là giữ nguyên ý nghĩa logic.

Listing 3.1: Mô hình khái niệm của lớp bao điều phối

```

1  type CoordinationEnvelope struct {
2      GroupID      []byte
3      SenderID     PeerID
4      Type         MessageType
5      Epoch        uint64
6      Timestamp    HLCTimestamp
7      Payload      []byte
8  }
```

Trong đó, *groupID* cho biết bản tin thuộc nhóm nào, *senderID* là nút gửi, *type* cho biết đây là Proposal, Commit hay application message, còn *epoch* cho biết trạng thái MLS mà bản tin được tạo ra. Trường *timestamp* chỉ dùng sắp xếp thông điệp ứng dụng, không dùng để quyết định Commit nào hợp lệ.

3.3.3 Phân loại thông điệp theo Epoch

Khi nhận một bản tin, nút cục bộ không xử lý ngay mà trước hết so sánh *env.epoch* với epoch cục bộ của nhóm tương ứng. Có ba tình huống chính. Nếu

hai giá trị bằng nhau, bản tin thuộc về trạng thái hiện tại và có thể được xử lý theo luồng bình thường. Nếu *env.epoch* nhỏ hơn epoch cục bộ, bản tin thuộc về quá khứ và không được phép kéo trạng thái nhóm lùi lại. Nếu *env.epoch* lớn hơn epoch cục bộ, nút cục bộ được xem là đang đi sau và phải thực hiện đồng bộ hóa thay vì tăng epoch một cách nhảy cóc.

Thuật toán 3 mô tả bước phân loại cơ bản này.

Algorithm 3: Phân loại bản tin theo epoch (*ProcessIncomingEnvelope*)

Đầu vào: *env*: một bản tin ở tầng điều phối; *localEpoch*: epoch cục bộ của nhóm.

Đầu ra : Hành động xử lý tương ứng.

```

if env.epoch < localEpoch then
    | Từ chối áp dụng bản tin lên trạng thái hiện tại;
else
    | if env.epoch > localEpoch then
    |     | Đưa bản tin vào bộ đệm thông điệp tương lai;
    |     | Kích hoạt quá trình đồng bộ trạng thái;
    | else
    |     | Chuyển bản tin sang luồng xử lý tương ứng với loại của nó;
    | end
end
  
```

Cách phân loại này giữ cho epoch cục bộ của mỗi nút là một đại lượng đơn điệu không giảm. Nút không tự ý tăng epoch chỉ vì nhìn thấy bản tin từ tương lai, mà chỉ tăng epoch sau khi đã xử lý thành công các bước chuyển trạng thái cần thiết.

3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai

Một thông điệp đến từ tương lai không tự nó là bằng chứng của phân nhánh. Trong nhiều trường hợp, điều đó chỉ cho thấy nút cục bộ đã bỏ lỡ một hoặc nhiều Commit trung gian. Vì vậy, phản ứng đúng của lớp điều phối không phải là kết luận ngay rằng hệ thống đã fork, mà là trước hết kiểm tra xem lịch sử Commit của nút cục bộ có còn là tiền tố hợp lệ của lịch sử trên nhánh xa hay không. Cơ chế kiểm tra này dựa trên một công thức lịch sử Commit, sẽ được trình bày chi tiết ở mục 3.4.

Nếu kiểm tra tiền tố cho thấy lịch sử cục bộ vẫn nằm trên cùng một chuỗi với nhánh còn lại kia, nút chỉ cần áp dụng tuần tự các Commit còn thiếu để bắt kịp. Nếu lịch sử đã phân kỳ, nút mới chuyển sang quy trình Fork Healing. Cả hai trường hợp đều dựa trên cùng một nguyên tắc: một nút không được nhảy cóc sang trạng thái mới nếu chưa có một con đường hợp lệ để đi từ trạng thái hiện tại của nó sang trạng thái đó.

3.3.5 Xử lý thao tác bị bỏ lỡ

Trong mạng bất đồng bộ, một Proposal do người dùng tạo ra có thể không được đưa vào Commit của Epoch mà nó hướng tới, chẳng hạn vì Token Holder đã chốt một Commit khác trước đó. Đồ án xử lý tình huống này theo nguyên tắc đề xuất lại có điều kiện. Cụ thể, nếu sau khi nhóm chuyển sang Epoch mới mà thao tác của người dùng vẫn còn hợp lệ về mặt ngữ nghĩa, nút cục bộ có thể tạo lại Proposal trên trạng thái mới rồi phát tán lại. Nếu thao tác không còn hợp lệ nữa, ứng dụng phải báo rõ cho người dùng thay vì tiếp tục giữ một ý định đã lỗi thời.

Nguyên tắc này giúp tránh việc đánh đồng giữa một Proposal bị bỏ lỡ và một thao tác phải bị hủy bỏ vĩnh viễn. Đồng thời, nó cũng giữ cho quyền quyết định Commit vẫn nằm ở Token Holder của từng epoch, thay vì cho phép các thao tác cũ tự ý chen vào một trạng thái nhóm đã thay đổi.

3.4 Cơ chế Fork Healing trạng thái

3.4.1 Bối cảnh phân vùng mạng

Single-Writer hoạt động tốt nhất khi các nút còn có cùng ActiveView. Tuy nhiên, trong mạng P2P, phân vùng mạng có thể chia nhóm thành nhiều phân vùng không liên lạc được với nhau. Khi đó, mỗi phân vùng chỉ quan sát được một phần thành viên đang hoạt động và có thể tính ra các Token Holder khác nhau. Nếu các phân vùng tiếp tục tiến hóa độc lập, hệ thống có thể xuất hiện nhiều nhánh trạng thái MLS khác nhau.

Đồ án xem phân vùng mạng là trường hợp phải xử lý ngay từ thiết kế chứ không phải ngoại lệ hiếm. Vấn đề không phải là ngăn hoàn toàn phân nhánh trong môi trường bất đồng bộ – điều đó không khả thi – mà là nhận ra sự phân kỳ và đưa hệ thống hội tụ lại khi các phân vùng nổi thông.

3.4.2 Phát hiện nghi vấn phân nhánh và đối chiếu trạng thái

Để nhận biết tình trạng phân kỳ, mỗi nút định kỳ công bố một bản tóm tắt trạng thái nhánh đang giữ. Ở mức khái niệm, bản tóm tắt này có thể được mô tả như Mã nguồn 3.2.

Listing 3.2: Bản tóm tắt trạng thái nhánh dùng cho đối chiếu

```

1 type GroupStateAnnouncement struct {
2     GroupID          []byte
3     Epoch            uint64
4     TreeHash         []byte
5     HistoryHash      []byte    // R(E) = H(R(E-1) ||
        ↪ CommitHash(E))
6     ActiveMemberCount uint32

```

Ý nghĩa của thông điệp này không phải là thay thế toàn bộ trạng thái MLS, mà là cung cấp một cách đủ ngắn gọn để lớp điều phối có thể so sánh các nhánh với nhau. Trong đó, epoch cho biết mức tiến hóa của nhánh, tree hash phản ánh trạng thái cây MLS, còn history hash là giá trị tóm tắt toàn bộ chuỗi Commit mà nút đã chấp nhận từ epoch 1 đến epoch hiện tại.

Để tính history hash, mỗi nút duy trì một giá trị theo công thức:

$$R(0) = r_0, \quad R(e) = H(R(e-1) \parallel C(e))$$

trong đó r_0 là hằng số khởi tạo, $C(e)$ là giá trị băm của Commit tại epoch e , còn H là hàm băm mật mã. Giá trị $R(E)$ tại epoch hiện tại phản ánh toàn bộ lịch sử Commit từ epoch 1 đến epoch E . Nếu hai nút có cùng $R(e)$ tại cùng một mốc e , thì trong phạm vi giả định rằng các Commit được chuẩn hóa và băm theo cùng một quy tắc, có thể xem như chúng đã đi qua cùng một chuỗi các thao tác lịch sử để cùng đến mốc đó.

Dựa vào cơ chế này, việc phân biệt giữa chậm đồng bộ và phân nhánh chỉ cần đưa về một phép kiểm tra tiền tố. Xét hai nút A và B với $E_A \leq E_B$. Nếu $R_B(E_A) = R_A(E_A)$, tức là giá trị tại epoch mới nhất của chuỗi ngắn hơn trùng khớp với đoạn tương ứng trong chuỗi dài hơn, thì A chỉ đang đi sau trên cùng một nhánh, không có phân nhánh. Ngược lại, nếu $R_B(E_A) \neq R_A(E_A)$, thì hai nút đã đi qua những thao tác lịch sử khác nhau và phân nhánh thực sự đã xảy ra.

Trong trường hợp hai nút đang ở cùng epoch, việc so sánh trực tiếp hai giá trị $R(E)$ là đủ để kết luận. Trong trường hợp một nút có epoch thấp hơn, nút đó cần thêm một lượt hỏi–đáp để lấy giá trị R của nút kia tại epoch của mình rồi mới đối chiếu.

3.4.3 Hàm trọng số nhánh

Khi hệ thống đã xác định rằng đang tồn tại nhiều nhánh khác nhau của cùng một nhóm, nó cần một quy tắc xác định trước để chọn nhánh tiếp tục. Đồ án dùng một hàm trọng số nhánh với thứ tự ưu tiên từ trái sang phải như sau:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (3.3)$$

Trong đó, $C_{active}(B)$ là số thành viên đang hoạt động quan sát được trên nhánh B , $E(B)$ là epoch hiện tại của nhánh, còn $H_{commit}(B)$ là giá trị băm của Commit

gần nhất gắn với nhánh đó. Các nhánh được so sánh theo thứ tự từ điển. Nhánh có nhiều thành viên hoạt động hơn được ưu tiên trước; nếu bằng nhau, nhánh có epoch cao hơn được ưu tiên; nếu vẫn bằng nhau, giá trị băm Commit dùng để phân biệt khi hai nhánh vẫn bằng nhau.

Thuật toán 4 mô tả quy tắc so sánh này.

Algorithm 4: So sánh trọng số nhánh (*CompareBranchWeight*)

Đầu vào: B_a, B_b : hai nhánh cần so sánh.

Đầu ra : Nhánh được chọn làm nhánh tiếp tục.

if $C_{active}(B_a) \neq C_{active}(B_b)$ **then**

 Chọn nhánh có C_{active} lớn hơn;

 Kết thúc;

end

if $E(B_a) \neq E(B_b)$ **then**

 Chọn nhánh có epoch lớn hơn;

 Kết thúc;

end

 Chọn nhánh có H_{commit} lớn hơn theo thứ tự từ điển;

Quy tắc này không nói rằng nhánh được chọn là nhánh “đúng” theo nghĩa tuyệt đối. Nó chỉ bảo đảm rằng nếu các nút cùng quan sát một tập nhánh và áp dụng cùng quy tắc xác định trước, chúng sẽ đi đến cùng một quyết định.

3.4.4 Quy trình Fork Healing

Sau khi chọn được nhánh thắng, hệ thống không cố hợp nhất trực tiếp hai trạng thái MLS đã phân kỳ. Trộn khóa, trộn transcript hay ghép thủ công hai trạng thái như vậy không thể duy trì trạng thái hợp lệ của MLS. Thay vào đó, nút ở nhánh thua lấy thông tin của nhánh thắng rồi gia nhập lại bằng External Proposal [1].

Khi phát hiện mình ở nhánh thua, nút đầu tiên hủy trạng thái MLS cũ đang giữ trong bộ nhớ và tạo một KeyPackage mới. Sau đó, nút bọc KeyPackage này vào một External Proposal với thao tác Add và phát qua GossipSub. Token Holder của epoch hiện tại nhận được proposal này, kiểm tra xem định danh cũ của nút còn nằm trong cây MLS nhóm hay không. Nếu còn, Token Holder tạo thêm một Remove Proposal cho nút cũ đó. Sau đó Token Holder sẽ tạo một Commit duy nhất chứa tất cả proposals đã gom, sinh thông điệp Welcome và chuyển tiếp cho nút ở nhánh thua. Nút nhánh thua sau đó sẽ có thể khôi phục trạng thái nhóm mới, rồi tiến hành tự phát lại các tin nhắn do chính nó tạo ra trong thời gian phân vùng.

Quy trình tổng quát được mô tả trong Thuật toán 5.

Nếu nhiều nút cùng bị rơi vào nhánh thua và mỗi nút tự tạo Commit để gia

Algorithm 5: Quy trình Fork Healing (*ForkHealing*)

Đầu vào: *Branches*: tập các nhánh đang được quan sát sau khi đã đổi chiều trạng thái.

Đầu ra : Một trạng thái MLS hợp lệ chung để nhóm tiếp tục vận hành.

winningBranch $\leftarrow$ *SelectBranch*(*Branches*);

for mỗi nút đang ở nhánh thua **do**

 Hủy trạng thái MLS cũ;

 Tạo KeyPackage mới;

 Gửi External Proposal (Add) qua GossipSub;

end

Token Holder::

Nhận External Proposal;

if định danh cũ còn trong cây **then**

 Tạo Remove Proposal cho lá cũ;

end

Tạo Add Proposal và buffer vào hàng đợi;

Khi hết hạn gom: tạo Commit chứa tất cả proposals;

Sinh Welcome và chuyển tiếp cho nút nhánh thua;

Nút nhánh thua::

ProcessWelcome $\rightarrow$ khôi phục trạng thái;

Tự phát lại tin nhắn do chính mình tạo (Autonomous Replay);

nhập, hệ thống sẽ đối mặt với hiện tượng N Commits cạnh tranh trong cùng epoch, phần lớn bị từ chối và phải thử lại. Việc dùng External Proposal kết hợp với Token Holder gom tất cả đề nghị vào một Commit duy nhất giúp tránh tình huống này. Dù có K nút cùng cần heal, hệ thống chỉ tạo đúng một Commit.

Sau khi quá trình Fork Healing hoàn tất và các nút nhánh thua đã thực hiện gia nhập lại thành công, hệ thống phải đối mặt với một vấn đề mang tính nền tảng của mật mã mã hóa đầu cuối là Backward Secrecy. Do đặc điểm của cơ chế gia nhập lại qua External Proposal, các nút từ một nhánh, khi gia nhập vào nhánh còn lại sẽ không sở hữu khóa bí mật của các epoch đã qua trong thời gian phân mảnh mạng của nhánh đó. Điều này dẫn tới hệ quả là chúng không thể giải mã các thông điệp trong lịch sử đã được nhánh kia trao đổi.

Để lấp đầy khoảng trống dữ liệu này mà không phá vỡ mô hình mã hóa đầu cuối, đề án sử dụng một cơ chế khôi phục thông điệp là khi nhận diện được mốc hợp nhất nhánh, mỗi thiết bị ở cả hai nhánh đều tiến hành quét lại toàn bộ tin nhắn được lưu cục bộ của mình. Hệ thống yêu cầu mỗi nút tự bọc lại các thông điệp do chính nó tạo ra trong thời gian đứt mạng bằng khóa nhóm của epoch mới nhất, sau đó phát lại vào mạng. Quy tắc này đảm bảo dữ liệu được phục hồi mà không vi phạm ranh giới thẩm quyền của MLS, bởi vì thao tác mã hóa được thực hiện bởi

chính tác giả gốc của thông điệp, không cho phép bất kỳ nút nào tự ý đại diện cho thông điệp của người khác.

Tuy nhiên, nếu hàng nghìn thông điệp được phát lại thành từng gói tin đơn lẻ ngay sau khi hợp nhất, mạng sẽ đối mặt với hiện tượng bão mạng gây nghẽn nghiêm trọng. Mặc dù về mặt vật lý, tổng dung lượng dữ liệu của các tin nhắn là không thể thay đổi, nhưng thay vì bắt mạng lưới gánh chịu hàng nghìn lần chi phí đóng gói thông điệp và ép CPU chạy thuật toán mã hóa MLS hàng nghìn lần, hệ thống sẽ gom hàng loạt tin nhắn này lại và gộp chung vào các gói tin lớn. Do mạng P2P có giới hạn kích thước gói tin, số lượng tin nhắn sẽ được chia thành những khối dữ liệu để đẩy đi. Cách làm này không làm giảm tổng dung lượng, nhưng sẽ giúp triệt tiêu hoàn toàn lượng metadata định tuyến khổng lồ không cần thiết.

3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock

3.5.1 Lý do cần HLC

Epoch trong MLS chỉ quyết định thứ tự thay đổi trạng thái mật mã. Nó không tạo ra một thứ tự toàn phần cho các thông điệp ứng dụng được gửi gần đồng thời trong cùng một Epoch. Trong môi trường P2P, hai thiết bị có thể nhận các thông điệp đó theo thứ tự khác nhau do độ trễ mạng, dù cả hai cuối cùng đều giải mã được đầy đủ.

Nếu giao diện chỉ hiển thị theo thời điểm nhận cục bộ, hai người dùng có thể nhìn thấy lịch sử hội thoại theo hai thứ tự khác nhau. Để giảm tình huống đó, đề án dùng đồng hồ logic lai HLC. HLC kết hợp thành phần thời gian gần với đồng hồ vật lý với một bộ đếm logic, từ đó tạo ra một khóa sắp thứ tự ổn định hơn cho các thông điệp ứng dụng [20].

$$HLC = (L, C, N) \quad (3.4)$$

Trong đó, L là thành phần thời gian logic bám theo thời gian vật lý, C là bộ đếm logic dùng khi nhiều sự kiện có cùng L , còn N là định danh nút dùng để phân biệt khi hai timestamp vẫn bằng nhau.

3.5.2 Tạo timestamp khi gửi tin

Khi gửi một thông điệp ứng dụng, nút cục bộ lấy thời gian vật lý hiện tại rồi so sánh với giá trị L đang lưu. Nếu thời gian vật lý đã tiến lên, L được cập nhật theo thời gian vật lý đó và bộ đếm C trở về 0. Nếu thời gian vật lý chưa tăng, hệ thống chỉ tăng bộ đếm logic. Thuật toán này được mô tả trong Thuật toán 6.

Algorithm 6: Sinh thời gian logic lai (HLC_Now)**Đầu vào:** hlc : trạng thái HLC cục bộ; $nodeID$: định danh nút cục bộ.**Đầu ra :** Timestamp HLC mới ($L, C, nodeID$). $pt \leftarrow$ thời gian vật lý hiện tại;**if** $pt > hlc.L$ **then** $hlc.L \leftarrow pt$; $hlc.C \leftarrow 0$;**end****else** $hlc.C \leftarrow hlc.C + 1$;**end****return** ($hlc.L, hlc.C, nodeID$);**3.5.3 Cập nhật HLC khi nhận tin**

Khi nhận một thông điệp ứng dụng mang timestamp HLC, nút cục bộ cập nhật đồng hồ của mình theo giá trị lớn nhất giữa đồng hồ cục bộ, timestamp nhận được và thời gian vật lý hiện tại. Cách cập nhật này bảo toàn thứ tự nhân quả cơ bản mà không buộc các thiết bị phải đồng bộ đồng hồ tuyệt đối. Thuật toán tương ứng được mô tả trong Thuật toán 7.

Algorithm 7: Cập nhật đồng hồ logic lai (HLC_Update)**Đầu vào:** hlc : trạng thái HLC cục bộ; msg : timestamp HLC nhận được
($msg.L, msg.C, msg.NodeID$).**Đầu ra :** Trạng thái HLC cục bộ sau khi cập nhật. $pt \leftarrow$ thời gian vật lý hiện tại; $newL \leftarrow \max(hlc.L, msg.L, pt)$;**if** $newL == hlc.L$ **and** $newL == msg.L$ **then** $hlc.C \leftarrow \max(hlc.C, msg.C) + 1$;**end****else if** $newL == hlc.L$ **then** $hlc.C \leftarrow hlc.C + 1$;**end****else if** $newL == msg.L$ **then** $hlc.C \leftarrow msg.C + 1$;**end****else** $hlc.C \leftarrow 0$;**end** $hlc.L \leftarrow newL$;**return** hlc ;

3.5.4 Vai trò của HLC trong hệ thống

HLC chỉ được dùng để sắp thứ tự hiển thị các thông điệp ứng dụng đã vượt qua bước kiểm tra epoch và đã được MLS xử lý hợp lệ. Nó không quyết định Commit nào đúng, không thay thế lớp điều phối ở tầng trạng thái và không được phép ảnh hưởng tới key schedule của MLS. Sự tách vai trò này là điều kiện cần để hệ thống vừa giữ đúng mô hình bảo mật của MLS, vừa có một thứ tự hiển thị ổn định hơn ở tầng ứng dụng.

3.6 Triển khai ứng dụng thử nghiệm phục vụ kiểm chứng

3.6.1 Ranh giới trách nhiệm trong ứng dụng thử nghiệm

Ở mức triển khai ứng dụng thử nghiệm, các nguyên tắc trên được ánh xạ thành một kiến trúc hai tầng gồm một tiến trình chủ chịu trách nhiệm điều phối và một thành phần MLS chịu trách nhiệm xử lý mật mã. Trạng thái bền vững của nhóm được lưu ở phía host; khi cần xử lý Proposal, Commit, External Proposal, External Commit hoặc application message, host lấy trạng thái hiện tại, chuyển nó cho tầng MLS xử lý rồi ghi lại trạng thái mới sau khi thao tác hoàn tất.

Cách triển khai này giữ đúng ranh giới đã nêu: quyết định điều phối nằm ở tầng ngoài, MLS chỉ xử lý các phép biến đổi trạng thái hợp lệ. Việc ứng dụng dùng công nghệ gì ở tầng giao diện không làm thay đổi bản chất của đóng góp nghiên cứu.

3.6.2 Vai trò của ứng dụng desktop

Trong ứng dụng thử nghiệm của đồ án, ứng dụng desktop không được xem là phần đóng góp thuật toán mới. Vai trò của nó là triển khai các luồng thao tác quan trọng nhất để kiểm chứng phương pháp: tham gia hệ thống bằng bundle, tạo và quản trị nhóm, và trao đổi thông điệp trên nhiều nút.

Vì lý do đó, các màn hình cụ thể, ảnh chụp ứng dụng và kịch bản demo không được trình bày chi tiết ngay trong Chương 3. Thay vào đó, Chương 5 sẽ dùng ứng dụng thử nghiệm như một môi trường kiểm chứng đầu cuối, còn phụ lục sẽ tập hợp đầy đủ các hình minh họa cần thiết cho việc đối chiếu.

3.7 Tính đúng đắn trực giác của phương pháp

Khi các nút còn chung ActiveView, Single-Writer giảm khả năng concurrent commits vì mỗi epoch chỉ có một Token Holder được quyền Commit. Kết hợp với kiểm tra epoch, cơ chế này buộc mọi bước tiến hóa trạng thái cục bộ phải đi qua chuỗi Commit hợp lệ, không cho phép nút tự ý xử lý bản tin trên trạng thái không tương thích.

Khi phân vùng mạng xảy ra, phương pháp không giả định hệ thống giữ được một trạng thái duy nhất mọi lúc. Nó chấp nhận phân kỳ tạm thời, nhưng yêu cầu

lớp điều phối nhận ra khác biệt giữa các nhánh, thử bắt kịp khi cần, rồi chọn một nhánh để hội tụ bằng cơ chế hợp lệ của MLS. Hệ thống không cố hợp nhất trực tiếp hai trạng thái đã rẽ nhánh.

HLC giữ thứ tự hiển thị ổn định cho thông điệp ứng dụng sau khi đã xử lý hợp lệ. Phương pháp đề xuất quan tâm cả đến nhất quán trạng thái mật mã lẫn tính quan sát được ở tầng ứng dụng, nhưng vẫn tách rõ hai loại ordering này.

3.7.1 Tiêu chí hội tụ trạng thái

Trước khi kết thúc chương, cần phát biểu rõ tiêu chí mà đề án dùng để xem hệ thống đã hội tụ. Thuật ngữ "hội tụ" xuất hiện trong nhiều phần của báo cáo, từ mục tiêu nghiên cứu đến đánh giá thực nghiệm, nên việc thống nhất một định nghĩa sẽ giúp các lập luận ở các chương sau có cơ sở rõ ràng.

Định nghĩa 3.1. Một nhóm MLS gồm tập nút hợp lệ $\mathcal{N}$ được xem là đã hội tụ tại epoch E khi và chỉ khi mọi nút $i \in \mathcal{N}$ thỏa mãn đồng thời ba điều kiện: (i) cùng số epoch, tức $E_i = E$; (ii) cùng mã băm cây, tức $\text{TreeHash}_i = \text{TreeHash}$; (iii) cùng mã băm lịch sử Commit, tức $R_i(E) = R(E)$.

Ba điều kiện trên tương ứng với việc các nút không chỉ cùng mang nhãn epoch mà còn cùng trạng thái cây MLS và cùng chuỗi lịch sử thay đổi nhóm. Nếu chỉ thỏa mãn điều kiện (i) mà không thỏa mãn (ii) và (iii), các nút vẫn ở trạng thái phân nhánh và chưa được xem là hội tụ. Trong thực tế, khi một nút gia nhập lại nhánh thắng qua External Proposal, trạng thái mới của nút đó được xây dựng lại từ nhánh đã chọn, nên cả ba điều kiện đều được thỏa mãn cùng lúc.

Cần phân biệt hội tụ trạng thái mật mã với hội tụ ActiveView. Hội tụ ActiveView xảy ra khi các nút cùng quan sát một tập thành viên đang hoạt động, nhưng điều này không đảm bảo hội tụ trạng thái mật mã. Hai nút có thể cùng ActiveView nhưng vẫn ở hai nhánh khác nhau trong cùng một epoch. Hội tụ ActiveView là điều kiện thuận lợi để Single-Writer giảm việc Commit đồng thời, nhưng tiêu chí cuối cùng để kết luận hệ thống đã phục hồi sau phân vùng mạng vẫn là hội tụ trạng thái mật mã theo Định nghĩa 3.1.

Kết chương. Chương 3 đã trình bày lớp điều phối phi tập trung mà đề án đề xuất cho MLS trên mạng ngang hàng. Phần cốt lõi của phương pháp gồm bốn cơ chế: Single-Writer theo epoch để hạn chế concurrent commits, kiểm tra nhất quán epoch để bảo vệ trạng thái cục bộ, phát hiện và Fork Healing để hỗ trợ hội tụ sau phân vùng mạng, và HLC để sắp thứ tự thông điệp ứng dụng.

Các cơ chế trên tạo thành phần nghiên cứu trung tâm của đề án. Ứng dụng thử nghiệm chỉ đóng vai trò triển khai và kiểm chứng các nguyên tắc đó trong một hệ

thông chạy được. Trên cơ sở này, Chương 4 sẽ tiếp tục phân tích các bất biến và giới hạn lý thuyết của phương pháp trước khi Chương 5 chuyển sang phần kiểm chứng bằng ứng dụng thử nghiệm và thực nghiệm.

CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT

Tổng quan chương.

Chương 3 đã trình bày lớp điều phối phi tập trung được đề xuất để đưa MLS vào môi trường mạng ngang hàng. Tuy nhiên, việc nêu cơ chế thôi chưa đủ để khẳng định hướng tiếp cận là hợp lý. Vì vậy, chương này tiếp tục phân tích cơ sở lý thuyết của các cơ chế đã nêu, làm rõ chúng bảo toàn được những bất biến nào, trong phạm vi giả định nào và giới hạn của chúng nằm ở đâu.

Trong phần còn lại, chương sẽ lần lượt phân tích tính đơn điệu của epoch cục bộ, tính duy nhất của Token Holder khi các nút có cùng ActiveView, khả năng nhận biết phân nhánh trạng thái, lý do không gộp trực tiếp hai trạng thái MLS đã tách nhánh, vai trò giới hạn của HLC và chi phí lý thuyết của lớp điều phối. Toàn bộ lập luận trong chương chỉ bám vào mô hình giao thức đã nêu ở Chương 3, không dựa vào chi tiết giao diện hay kỹ thuật triển khai của ứng dụng thử nghiệm.

4.1 Mô hình phân tích và giả định

Xét một nhóm MLS có định danh *groupID* và trạng thái hiện hành ở epoch *E*. Mỗi nút giữ một trạng thái cục bộ, trong đó phần lõi là trạng thái MLS dùng để xử lý Proposal, Commit và application message; phần còn lại là metadata điều phối phục vụ bầu chọn Token Holder, đối chiếu trạng thái và phát hiện phân nhánh. Trong phạm vi chương này, cần phân biệt rõ hai lớp thông tin đó. Trạng thái MLS đầy đủ quyết định tính hợp lệ về mật mã, còn metadata điều phối chỉ đóng vai trò quan sát và điều hướng quá trình hội tụ.

Mạng ngang hàng được xem là môi trường bất đồng bộ. Thông điệp có thể đến trễ, đến sai thứ tự, bị lặp hoặc tạm thời không đến được. Khi mạng còn liên thông, các nút cuối cùng có thể trao đổi lại thông tin với nhau, nhưng không giả định rằng mọi nút quan sát thông điệp theo cùng một thứ tự. Khi xảy ra phân vùng mạng, hai tập nút khác nhau có thể hình thành hai ActiveView khác nhau về những thành viên còn đang hoạt động.

Ký hiệu $ActiveView_i(E)$ là tập thành viên mà nút *i* quan sát là còn hoạt động tại epoch *E*. Trong điều kiện ổn định, các nút có xu hướng quan sát cùng một *ActiveView*. Khi mạng bị chia cắt, hai phân vùng có thể giữ hai *ActiveView* khác nhau. Chính khác biệt này làm cho cùng một quy tắc bầu chọn xác định trước vẫn có thể cho ra hai Token Holder khác nhau ở hai phân vùng khác nhau.

Các lập luận dưới đây được xây dựng trong phạm vi giả định của đề án: các nút nhìn chung tuân thủ giao thức, các thao tác MLS cốt lõi được OpenMLS xử lý

đúng, và mục tiêu chính là phân tích hành vi của lớp điều phối khi thiếu Delivery Service tập trung. Chương này không tìm cách bao phủ toàn bộ không gian tấn công của một hệ phân tán đối kháng mạnh (strong adversary) – bài toán đó thuộc về các giao thức đồng thuận chịu lỗi Byzantine [11], nằm ngoài phạm vi đồ án.

4.2 Bất biến 1: Epoch cục bộ chỉ được giữ nguyên hoặc tăng

4.2.1 Phát biểu bất biến

Tại mỗi nút, epoch cục bộ chỉ được giữ nguyên hoặc tăng lên sau khi nút xử lý thành công một Commit hợp lệ. Nút không được áp dụng một thông điệp thuộc epoch cũ lên trạng thái hiện tại, vì điều đó có thể kéo trạng thái nhóm quay về một mốc cũ và làm cho bản mã bị xử lý trên một trạng thái MLS không còn tương thích.

Bất biến này có thể viết ngắn gọn như sau:

$$E_i(t+1) \geq E_i(t)$$

trong đó $E_i(t)$ là epoch cục bộ của nút i tại thời điểm logic t .

4.2.2 Cơ sở từ MLS

Trong MLS, mỗi Commit hợp lệ đưa nhóm sang một epoch mới và làm thay đổi trạng thái cây, lịch sử bắt tay cùng lịch trình khóa của nhóm [1]. Vì vậy, một thông điệp được tạo ở epoch cũ không thể được xem là tương thích một cách mặc nhiên với trạng thái hiện tại. Nếu nút cho phép áp dụng tùy tiện các thông điệp cũ, chuỗi tiến hóa của trạng thái nhóm sẽ bị phá vỡ.

4.2.3 Cơ chế bảo toàn bất biến

Chương 3 đã đề xuất cơ chế kiểm tra epoch ở lớp điều phối trước khi chuyển thông điệp xuống MLS. Quy tắc xử lý có thể tóm tắt như Bảng 4.1.

Table 4.1: Quy tắc xử lý thông điệp theo epoch

Quan hệ epoch	Hành động của lớp điều phối
$E_{msg} = E_{local}$	Thông điệp được chuyển tiếp cho lớp MLS xử lý theo luồng tiến hóa bình thường.
$E_{msg} < E_{local}$	Thông điệp không được áp dụng lên trạng thái hiện tại, nhằm tránh kéo trạng thái cục bộ lùi về một mốc cũ.
$E_{msg} > E_{local}$	Thông điệp chưa được xử lý ngay; nút tạm giữ lại và kích hoạt bước đồng bộ trạng thái.

Mệnh đề 4.1. Nếu một nút chỉ tăng epoch sau khi xử lý thành công Commit hợp lệ và không áp dụng thông điệp thuộc epoch cũ lên trạng thái hiện tại, thì epoch cục

bộ của nút là đơn điệu không giảm.

Lập luận. Theo Bảng 4.1, mọi thông điệp cũ đều bị chặn ở lớp điều phối, nên trạng thái MLS hiện tại không bị đề ngược bởi dữ liệu quá khứ. Đối với thông điệp đi trước trạng thái cục bộ, nút không áp dụng ngay mà phải đồng bộ lại chuỗi trạng thái. Bởi trạng thái mới chỉ xuất hiện sau khi xử lý Commit hợp lệ, chuỗi giá trị epoch tại một nút không thể tự giảm xuống.

Bất biến này không đồng nghĩa với việc mọi nút luôn giữ cùng một epoch ở mọi thời điểm. Trong mạng ngang hàng, có nút có thể tạm thời chậm hơn do mất kết nối hoặc nhận thông điệp muộn. Điều bất biến bảo toàn là mỗi nút không tự kéo trạng thái của chính mình lùi về phía sau.

4.3 Bất biến 2: Với cùng một ActiveView, chỉ có một Token Holder

4.3.1 Phát biểu bất biến

Ở epoch E , nếu hai nút có cùng $groupID$, cùng số epoch và cùng tập ứng viên hợp lệ, chúng phải tính ra cùng một Token Holder. Token Holder là nút duy nhất được quyền tạo Commit trong view đó.

Tập ứng viên hợp lệ được xác định như sau:

$$\begin{aligned} Eligible(E) = & GroupMembers(E) \cap ActiveView(E) \\ & \cap AuthorizedCommitters(E) \setminus RemovedOrSuspended(E) \end{aligned} \quad (4.1)$$

Trong đó, $GroupMembers(E)$ là tập thành viên còn có mặt trong trạng thái MLS tại epoch E ; $ActiveView(E)$ là tập thành viên còn được quan sát là hoạt động; $AuthorizedCommitters(E)$ là tập thành viên được chính sách ứng dụng cho phép tạo Commit; và $RemovedOrSuspended(E)$ là tập thành viên bị loại tạm thời khỏi vòng bầu chọn.

Token Holder được tính bằng quy tắc xác định trước:

$$TokenHolder(E) = \operatorname{argmin}_{n \in Eligible(E)} H(groupID \parallel E \parallel n) \quad (4.2)$$

trong đó H là hàm băm mật mã, chẳng hạn SHA-256.

Mệnh đề 4.2. Nếu hai nút có cùng $groupID$, cùng epoch E và cùng tập $Eligible(E)$, thì chúng chọn cùng một Token Holder.

Lập luận. Khi đầu vào của Công thức (4.2) là như nhau, tập giá trị băm sinh ra trên hai nút cũng như nhau. Do phép chọn argmin luôn cho ra cùng một kết quả,

kết quả cuối cùng bắt buộc trùng nhau. Nói cách khác, hệ thống không cần thêm một vòng bỏ phiếu mạng chỉ để quyết định ai là người tạo Commit trong cùng một ActiveView.

4.3.2 Ý nghĩa đối với concurrent commits

Trong MLS, Commit là thao tác làm thay đổi trạng thái nhóm và đưa nhóm sang epoch mới [1]. Nếu mọi thành viên đều có thể tự Commit tại cùng một epoch, hệ thống sẽ dễ sinh nhiều nhánh cạnh tranh. Cơ chế Single-Writer giải quyết đúng điểm này bằng cách giới hạn quyền tạo Commit vào một nút duy nhất trong từng view.

Điều cần nhấn mạnh là bất biến trên chỉ đúng khi các nút đang có cùng Active-View. Nếu hai phân vùng giữ hai *ActiveView* khác nhau, chúng có thể hình thành hai tập $Eligible(E)$ khác nhau và từ đó chọn ra hai Token Holder khác nhau. Vì vậy, Single-Writer không phải là lời hứa rằng hệ thống không bao giờ phân nhánh; vai trò đúng của nó là giảm mạnh khả năng sinh concurrent commits khi mạng còn đủ liên thông để các nút chia sẻ cùng một ActiveView.

4.4 Bất biến 3: Phân nhánh phải được nhận biết sau bước đối chiếu lịch sử

4.4.1 Phân biệt chậm đồng bộ và phân nhánh thực sự

Trong mạng ngang hàng, khác biệt quan sát được giữa hai nút chưa đủ để kết luận rằng nhóm đã phân nhánh. Nếu một nút có epoch thấp hơn nút khác, khả năng tự nhiên hơn là nó chỉ đang chậm đồng bộ. Ngay cả khi hai nút đều công bố những fingerprint trạng thái khác nhau, kết luận có fork vẫn chỉ nên được đưa ra sau khi đã kiểm tra xem một bên có còn là phần kéo dài hợp lệ của bên kia hay không. Vì vậy, bài toán ở đây không đơn thuần là so sánh hai trạng thái hiện tại, mà là đối chiếu lịch sử Commit mà mỗi nút đã chấp nhận.

4.4.2 Tóm tắt phần lịch sử Commit đã đi qua

Ký hiệu $C_i(e)$ là giá trị băm của Commit mà nút i đã chấp nhận tại epoch e . Từ dãy các Commit này, mỗi nút tính một giá trị tóm tắt cho toàn bộ phần lịch sử mà nó đã đi qua:

$$\begin{aligned} R_i(0) &= r_0 \\ R_i(e) &= H(R_i(e-1) \parallel C_i(e)) \end{aligned}$$

trong đó r_0 là hằng khởi tạo chung, còn H là hàm băm mật mã. Giá trị $R_i(e)$ không chỉ phản ánh Commit gần nhất, mà còn gói lại toàn bộ chuỗi Commit từ

epoch 1 đến epoch e . Nếu hai nút có cùng $R(e)$ tại cùng một mốc e , thì trong phạm vi giả định rằng các Commit được chuẩn hóa và được bấm theo cùng một quy tắc, có thể xem như chúng đã đi qua cùng một đoạn lịch sử đến mốc đó.

Để phục vụ bước đối chiếu, mỗi nút công bố định kỳ một bản tóm tắt trạng thái ngắn gọn:

$$StateSummary = (groupID, E, treeHash, historyHash)$$

Trong đó, $treeHash$ phản ánh trạng thái cây MLS hiện tại, còn $historyHash$ chính là $R(E)$. Bản tóm tắt này không thay thế trạng thái MLS đầy đủ, nhưng đủ để làm tín hiệu ban đầu cho bước đối chiếu. Nếu hai nút đang ở cùng epoch, việc so sánh trực tiếp hai giá trị $R(E)$ là đủ để biết chúng có cùng lịch sử Commit đến epoch đó hay không. Nếu một nút A đang ở epoch E_A thấp hơn nút B ở epoch E_B , nút A chưa thể kết luận gì chỉ từ $R_B(E_B)$. Khi đó, A cần thêm một lượt hỏi–đáp để lấy đúng giá trị $R_B(E_A)$ rồi mới đối chiếu với $R_A(E_A)$.

Mệnh đề 4.3. Trong phạm vi giả định của đề án, giả sử hai nút A và B thuộc cùng một nhóm và $E_A \leq E_B$. Sau khi A nhận được $StateSummary$ của B và, nếu cần, lấy thêm giá trị $R_B(E_A)$ qua một lượt đối chiếu bổ sung, nếu

$$R_B(E_A) \neq R_A(E_A)$$

thì lớp điều phối có cơ sở để kết luận rằng A và B không còn đứng trên cùng một chuỗi Commit. Ngược lại, nếu

$$R_B(E_A) = R_A(E_A)$$

thì khác biệt quan sát được vẫn còn có thể giải thích bằng việc A đang chậm đồng bộ và lịch sử của A chỉ là một tiền tố của lịch sử ở B .

Lập luận. Do $R_i(e)$ được xây dựng bằng cách bấm lũy tiến trên toàn bộ dãy Commit từ 1 đến e , nên điều kiện $R_B(E_A) = R_A(E_A)$ có nghĩa là hai nút đã chấp nhận cùng một đoạn lịch sử đến epoch E_A . Nếu B còn đi xa hơn sau mốc đó, thì sự khác biệt hiện tại chỉ phản ánh việc B đã xử lý thêm các Commit mới mà A chưa kịp nhận. Ngược lại, nếu $R_B(E_A) \neq R_A(E_A)$, thì phải tồn tại ít nhất một epoch không vượt quá E_A mà hai bên đã chấp nhận hai Commit khác nhau, hoặc đã đi

qua hai đoạn lịch sử khác nhau. Khi đó, khác biệt quan sát được không còn là chậm đồng bộ đơn thuần mà đã trở thành khác biệt về lịch sử tiến hóa của trạng thái nhóm.

Trong lập luận này, *treeHash* vẫn giữ vai trò một fingerprint ngắn gọn của trạng thái MLS hiện hành và vẫn hữu ích trong bước đối chiếu trạng thái hoặc giai đoạn phục hồi sau phân nhánh. Tuy nhiên, tín hiệu quyết định để phân biệt giữa “đi sau trên cùng một nhánh” và “đã tách sang nhánh khác” là phép kiểm tra tính trùng khớp của tiền tố lịch sử Commit tại epoch nhỏ hơn, chứ không phải chỉ là so sánh Commit gần nhất ở đỉnh hiện tại.

4.5 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh

4.5.1 Vì sao không thể gộp trực tiếp hai nhánh

Sau khi phân nhánh, mỗi nhánh MLS có lịch sử Commit, trạng thái cây, lịch sử bắt tay và lịch trình khóa riêng. Nếu lớp điều phối tự ý trộn hai trạng thái đó để tạo ra một trạng thái mới, trạng thái mới sẽ không còn là kết quả của một chuỗi thao tác MLS hợp lệ [1]. Khi ấy, hệ thống không còn có thể dựa vào các bảo đảm của MLS/OpenMLS để khẳng định trạng thái mới là hợp lệ về mật mã.

Vì lý do đó, phương pháp của đồ án không tìm cách hợp nhất trực tiếp hai nhánh đã tách. Thay vào đó, nó chọn một nhánh tiếp tục và đưa các nút ở nhánh còn lại gia nhập lại nhóm theo con đường MLS hợp lệ. Cách xử lý này giữ cho quá trình hội tụ vẫn nằm trong mô hình vận hành chuẩn của MLS.

4.5.2 Quy tắc chọn nhánh

Giả sử sau khi mạng nối lại, hệ thống quan sát được tập các nhánh

$$\mathcal{B} = \{B_1, B_2, \dots, B_k\}$$

Mỗi nhánh B được gắn với một trọng số:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (4.3)$$

Trong đó, $C_{active}(B)$ là số thành viên còn hoạt động của nhánh, $E(B)$ là epoch hiện tại của nhánh, còn $H_{commit}(B)$ là giá trị băm của Commit gần nhất dùng để phân biệt khi hai nhánh vẫn bằng nhau. Hai nhánh được so sánh theo thứ tự từ điển trên ba thành phần này.

Quy tắc trên không nhằm khẳng định nhánh thắng là nhánh "đúng tuyệt đối" trong mọi trạng thái của mạng. Vai trò của nó khi thông tin đã đủ là tạo ra một quy tắc hội tụ nhất quán: nhiều nút quan sát cùng một tập nhánh sẽ có xu hướng chọn cùng một nhánh tiếp tục.

4.5.3 Dùng External Proposal như cơ chế hội tụ

Sau khi một nhánh được chọn, các nút ở nhánh thua không mang trạng thái cũ đi ghép với nhánh thắng. Chúng gửi External Proposal để xin gia nhập lại, còn việc tạo Commit thuộc về Token Holder. Khi đó, các khóa cũ của nhánh thua phải bị loại bỏ, còn trạng thái mới được xây dựng lại từ nhánh đã chọn.

Mệnh đề 4.4. Nếu các nút quan sát cùng một tập nhánh và cùng áp dụng quy tắc chọn nhánh ở Công thức (4.3), thì chúng sẽ có cùng cơ sở để chọn nhánh tiếp tục; việc đưa các nút của nhánh thua gia nhập lại qua External Proposal được Token Holder gom vào Commit giúp hệ thống đạt hội tụ trạng thái mật mã theo Định nghĩa 3.1, tuân thủ theo tiêu chuẩn MLS mà không cần phải tự chế ra một cơ chế mật mã mới.

Lập luận. Thành phần $C_{active}(B)$ ưu tiên nhánh còn duy trì được nhiều thành viên hoạt động hơn, $E(B)$ ưu tiên nhánh tiến xa hơn khi số thành viên còn hoạt động bằng nhau, còn $H_{commit}(B)$ dùng để phân biệt khi hai tiêu chí trước vẫn trùng nhau. Sau khi đã chọn được nhánh tiếp tục, quá trình gia nhập lại không dựa trên việc gộp hai trạng thái cũ, mà dựa trên chính các cơ chế thay đổi thành viên mà MLS cho phép. Nhờ vậy, đường hội tụ vẫn nằm trong không gian trạng thái hợp lệ của giao thức.

4.5.4 Xử lý thông điệp phát sinh ở nhánh thua

Trong thời gian phân vùng mạng, người dùng ở mỗi nhánh vẫn có thể gửi các tin nhắn ứng dụng. Khi hệ thống đạt hội tụ trạng thái mật mã trở lại, các thông điệp sinh ra ở nhánh thua đặt ra một câu hỏi riêng: nội dung đó có thể còn giá trị với người dùng, nhưng bản mã gốc của nó lại thuộc về một trạng thái MLS không còn tiếp tục được sử dụng.

Phương pháp của đề án không phát lại nguyên trạng các bản mã này. Nếu chính sách ứng dụng cho phép phục hồi nội dung, mỗi nút chỉ được tự mã hóa lại các thông điệp do chính nó tạo ra, rồi gửi chúng như các application message mới dưới trạng thái hợp lệ của nhánh thắng. Cách làm này giữ ba ranh giới quan trọng. Thứ nhất, hệ thống không kéo dài vòng đời sử dụng của các khóa thuộc nhánh thua. Thứ hai, một nút không được tự ý phát lại nội dung thay cho nút khác. Thứ ba, thông điệp sau khi quay lại hệ thống sẽ là thông điệp mới của trạng thái hiện hành, chứ không phải là mảnh ghép thô của trạng thái cũ.

4.6 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit

4.6.1 Phân biệt hai loại thứ tự

Trong hệ thống cần tách bạch hai loại thứ tự. Loại thứ nhất là thứ tự Commit, quyết định chuỗi tiến hóa của trạng thái MLS. Loại thứ hai là thứ tự hiển thị application message sau khi chúng đã được chấp nhận và giải mã hợp lệ. Hai loại thứ tự này phục vụ hai mục tiêu khác nhau nên không thể dùng lẫn cho nhau.

MLS đã có cơ chế riêng để ràng buộc application message với đúng nhóm, đúng epoch và đúng ngưỡng cảnh mật mã [1]. Tuy nhiên, ở mức ứng dụng, các nút vẫn cần một quy tắc ổn định để hiển thị cùng một tập thông điệp theo thứ tự nhất quán hơn. HLC được đưa vào đúng cho mục đích đó.

4.6.2 Cơ sở của HLC

Timestamp HLC của một thông điệp được biểu diễn dưới dạng

$$HLC = (L, C, NodeID) \quad (4.4)$$

trong đó L là thành phần thời gian logic gần với thời gian vật lý, C là bộ đếm logic, còn $NodeID$ dùng để phân biệt khi hai timestamp vẫn bằng nhau. Theo Kulkarni và Demirbas, nếu sự kiện e xảy ra trước sự kiện f theo quan hệ happened-before, timestamp HLC của e sẽ nhỏ hơn timestamp của f [20]. Đồ án khai thác tính chất này ở tầng hiển thị thông điệp ứng dụng.

Mệnh đề 4.5. HLC giúp tạo thứ tự hiển thị xác định trước cho cùng một tập application message đã được giải mã hợp lệ, nhưng không thay thế cơ chế ordering của Commit.

Lập luận. Nếu message A là nguyên nhân dẫn đến message B , chẳng hạn người dùng gửi phản hồi sau khi đã đọc A , thì timestamp HLC của B sẽ lớn hơn timestamp của A theo thứ tự từ điển. Với các message đồng thời, hệ thống dùng bộ ba $(L, C, NodeID)$ để phân biệt khi hai timestamp vẫn bằng nhau. Nhờ đó, khi hai nút đã có cùng tập thông điệp hợp lệ, chúng có thể sắp xếp tập đó theo cùng một quy tắc mà không đưa HLC tham gia vào việc quyết định ai được Commit.

4.6.3 Giới hạn của HLC

HLC không được dùng để xác định Commit nào hợp lệ, cũng không được dùng để sửa phân nhánh trạng thái MLS. Một Commit chỉ hợp lệ khi phù hợp với trạng thái epoch hiện tại và vượt qua các bước kiểm tra của MLS. Việc tách ranh giới này giúp hệ thống tránh nhầm một đồng hồ ứng dụng với cơ chế điều phối tiến hóa

trạng thái mật mã.

4.7 Phân tích chi phí lý thuyết

Phần này chỉ xét chi phí gắn với bản thân phương pháp đề xuất, không đi sâu vào chi phí cụ thể của ứng dụng thử nghiệm. Các số đo triển khai và chi phí triển khai sẽ được chuyển sang Chương 5.

4.7.1 Chi phí gửi application message

Trong mô hình mã hóa từng cặp, người gửi thường phải tạo hoặc bọc khóa riêng cho từng người nhận. Với nhóm có N thành viên, chi phí này có xu hướng tăng theo số người nhận.

Với MLS, application message được bảo vệ bằng khóa dẫn xuất từ trạng thái nhóm tại epoch hiện tại. Người gửi không cần tạo một bản mã độc lập cho từng thành viên. Vì vậy, xét riêng phần mã hóa nội dung, chi phí không tăng theo cùng cách như mô hình mã hóa từng cặp [1]. Dĩ nhiên, chi phí đầu cuối của một hệ thống thực tế còn phụ thuộc vào lớp mạng và cơ chế phát tán thông điệp, nhưng đó là vấn đề của triển khai chứ không phải của nguyên lý MLS.

4.7.2 Chi phí cập nhật thành viên và rekey

Các thao tác thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa đều đi qua Proposal và Commit. Ở mức TreeKEM, việc tổ chức khóa theo cấu trúc cây nhị phân giúp tối ưu hóa đáng kể chi phí tính toán và truyền thông so với việc mã hóa riêng rẽ cho từng cặp thành viên [1]. Cụ thể, khi quy mô nhóm N tăng lên, chi phí thay đổi thành viên chỉ tăng theo độ phức tạp logarit $\mathcal{O}(\log N)$, thay vì tăng tuyến tính $\mathcal{O}(N)$ như các mô hình phân phối khóa truyền thống.

4.7.3 Chi phí của Single-Writer

Single-Writer không yêu cầu một vòng đồng thuận mạng riêng để bầu Token Holder trong điều kiện các nút có cùng ActiveView. Mỗi nút chỉ cần duyệt qua tập $\text{Eligible}(E)$ và tính hàm băm cho từng ứng viên. Nếu số ứng viên hợp lệ là N , chi phí tính toán cục bộ của bước chọn Token Holder là $\mathcal{O}(N)$ theo cách triển khai trực tiếp.

Điểm quan trọng nằm ở chỗ đây là chi phí cục bộ, không phải chi phí nhiều vòng mạng. Trong bối cảnh mạng ngang hàng, tránh phải chạy một cơ chế đồng thuận nặng cho mỗi Commit là lợi thế thực sự của phương pháp.

4.7.4 Chi phí của kiểm tra epoch và đối chiếu trạng thái

Kiểm tra epoch chỉ yêu cầu so sánh một số trường metadata như *groupID*, loại thông điệp và epoch trước khi quyết định hướng xử lý. Nếu chỉ xét riêng lớp điều phối, chi phí này có thể xem là $\mathcal{O}(1)$ trên mỗi thông điệp.

Đối chiếu trạng thái để nhận biết phân nhánh đòi hỏi các nút trao đổi một bản tóm tắt ngắn chứa số epoch, $treeHash$ và giá trị tóm tắt lịch sử $R(E)$. Kích thước của bản tóm tắt này vẫn là hằng số theo kích thước nhóm, nên chi phí của một lần công bố trạng thái đơn lẻ vẫn ở mức $\mathcal{O}(1)$ theo số trường cần gửi. Khi xuất hiện nghi ngờ rằng một nút đang đi trước nhưng có thể đã tách nhánh, hai bên cần thêm một lượt hỏi–đáp để đối chiếu giá trị $R(e)$ tại epoch nhỏ hơn. Chi phí phụ thêm này không nằm trên mọi thông điệp, mà chỉ phát sinh khi lớp điều phối cần phân biệt giữa chậm đồng bộ và phân nhánh thực sự.

4.7.5 Chi phí của Fork Healing

Fork Healing chỉ phát sinh khi đã có phân nhánh trạng thái. Chi phí của nó gồm ba phần: nhận diện khác biệt, chọn nhánh tiếp tục và đưa các nút ở nhánh thua gia nhập lại. Phần chọn nhánh có chi phí phụ thuộc vào số nhánh đang quan sát được, thường nhỏ hơn nhiều so với số thành viên của nhóm. Phần tốn kém hơn nằm ở bước gia nhập lại, vì bước này yêu cầu hệ thống quay trở lại một luồng thao tác MLS hợp lệ thay vì chỉ sửa metadata ở ngoài.

Khi dùng External Proposal, chi phí tạo Commit cho fork healing không tăng theo số nút cần heal. Token Holder gom tất cả External Proposals vào một Commit duy nhất, nên dù có K nút cùng cần gia nhập lại, hệ thống chỉ tạo đúng một Commit. Điều này khác với việc mỗi nút tự tạo Commit riêng, khi đó K Commits cạnh tranh sẽ dẫn đến $K-1$ lần bị từ chối và phải thử lại, tạo ra chi phí $\mathcal{O}(K^2)$.

Về bản chất, đây là một chi phí bất thường chứ không phải chi phí xuất hiện trên mọi thông điệp. Đề án chấp nhận chi phí này để đổi lấy khả năng tiếp tục hoạt động cục bộ trong lúc mạng bị chia cắt, rồi hội tụ trở lại khi kết nối được khôi phục.

4.7.6 Phân tích cơ chế phát lại thông điệp ứng dụng

Quy trình tự mã hóa và phát lại thông điệp đóng vai trò quan trọng trong việc khôi phục tính liên tục của dữ liệu sau khi hợp nhất nhánh. Về mặt bảo mật, cách làm này không vi phạm thuộc tính Forward Secrecy hay Backward Secrecy của giao thức MLS. Khi một thành viên phát lại thông điệp lịch sử của chính họ, bản chất là thiết bị của tác giả đang tự giải mã bản rõ của mình ở cục bộ và mã hóa lại bằng khóa nhóm của epoch mới nhất. Do đó, các nút vừa mới gia nhập hoàn toàn có thể giải mã được bản mã mới này mà không cần tiếp cận vào vật liệu khóa của các epoch cũ. Việc không cho phép một nút đại diện mã hóa lại thông điệp của nút khác cũng đảm bảo tính xác thực và tránh mạo danh người khác.

Về khía cạnh hiệu năng mạng lưới, đề án sử dụng kỹ thuật gom tin nhấn để hạn chế tối đa bão mạng. Tổng dung lượng dữ liệu cần truyền đi là không đổi. Tuy nhiên, việc gom nhiều tin nhắn vào chung các gói tin giúp giảm đáng kể số lần

CPU phải mã hóa MLS và giảm lượng lớn metadata định vị đính kèm trong các gói tin mạng.

4.8 Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn

Bảng 4.2 tóm tắt các bất biến chính đã được phân tích trong chương này và cơ chế tương ứng của phương pháp.

Table 4.2: Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn

Bất biến / Thuộc tính	Cơ chế bảo toàn	Ý nghĩa
Epoch cục bộ chỉ giữ nguyên hoặc tăng	Kiểm tra epoch	Ngăn nút áp dụng thông điệp lên trạng thái MLS cũ hoặc không tương thích.
Một Token Holder trong cùng ActiveView	Single-Writer theo epoch	Giảm concurrent commits khi các nút có cùng ActiveView.
Dấu hiệu phân nhánh có thể quan sát được	Bản tóm tắt trạng thái, <i>treeHash</i> , giá trị tóm tắt lịch sử Commit	Cho phép phân biệt giữa chậm đồng bộ và phân nhánh thực sự sau bước đối chiếu.
Không gộp trực tiếp trạng thái MLS đã tách nhánh	Chọn nhánh và gia nhập lại theo luồng MLS hợp lệ	Tránh tự tạo ra trạng thái mật mã ngoài mô hình MLS.
Thứ tự hiển thị không quyết định Commit	HLC	Tách application ordering khỏi crypto ordering.

4.9 So sánh đặc tính với các thuật toán đồng thuận truyền thống

Mục này trình bày sự đối sánh về mặt lý thuyết giữa giao thức điều phối đề xuất (Single-Writer Token kết hợp hàn gắn phân nhánh) với các cơ chế đồng thuận truyền thống (như Raft [13] hay BFT [11]) và Blockchain [14], nhằm làm rõ tính phù hợp của giải pháp đối với đặc thù bất đồng bộ của mạng ngang hàng.

Thứ nhất, về độ phức tạp khi truyền thông. Các thuật toán đồng thuận dựa trên Quorum như BFT [11] yêu cầu các nút mạng trao đổi thông báo chéo để đạt được thỏa thuận, dẫn đến độ phức tạp có thể lên tới $\mathcal{O}(N^2)$ ở một số thời điểm, trong đó N là số lượng nút tham gia vào vòng đồng thuận. Ngay cả với các biến thể tối ưu hơn, số lượng thông điệp trao đổi vẫn phụ thuộc vào quy mô mạng. Đối với giao thức đề xuất, nhờ việc quyền phát hành Commit được gán duy nhất cho Token Holder của epoch hiện tại, thông điệp Commit chỉ cần được phát sóng một lần qua mạng. Tại thời điểm phát hành, độ phức tạp của bước chốt trạng thái đối với Token Holder được giữ ở mức $\mathcal{O}(1)$, giúp tiết kiệm băng thông đáng kể.

Thứ hai, về khả năng hoạt động khi mạng bị chia cắt. Trong kịch bản mạng bị chia cắt thành các phân vùng không còn liên thông, các thuật toán đồng thuận như Raft [13] yêu cầu một phân vùng phải chứa phần lớn số nút ($> N/2$) để có thể tiếp tục xác nhận trạng thái mới. Phân vùng nhỏ hơn sẽ hoàn toàn mất khả năng tiến hóa trạng thái (mất Availability). Ngược lại, thiết kế của đề án đi theo hướng ưu

tiên tính Khả dụng trong định lý CAP. Nếu Token Holder nằm ở một phân vùng, phân vùng đó vẫn có thể vận hành và cập nhật trạng thái MLS một cách độc lập. Những nút ở phân vùng bị đứt kết nối sẽ tạm thời không đồng bộ, nhưng khi mạng được khôi phục, chúng sẽ sử dụng quy tắc chọn nhánh và cơ chế External Proposal để đạt hội tụ trạng thái mật mã trở lại, qua đó đạt được tính Nhất quán cuối cùng [21]. Sự đánh đổi này đặc biệt phù hợp cho các ứng dụng trò chuyện phi tập trung, nơi yêu cầu không làm gián đoạn trải nghiệm người dùng khi rút mạng được đặt lên hàng đầu.

Bảng 4.3 tổng hợp lại các phân tích trên để làm nổi bật sự khác biệt và ưu điểm của giải pháp đề xuất so với các hướng tiếp cận truyền thống.

Table 4.3: Bảng so sánh cơ chế điều phối đề xuất với các thuật toán đồng thuận truyền thống

Tiêu chí đánh giá	Đồng thuận Quorum (Raft/BFT)	Blockchain	Giải pháp đề xuất (Single-Writer Token)
Mô hình nhất quán	Nhất quán mạnh	Nhất quán cuối cùng (sau khi khối được xác nhận)	Nhất quán cuối cùng
Sức chịu đựng phân vùng mạng	Kém (hệ thống đóng băng nếu không thể đạt $\text{Quorum} > N/2$)	Tốt	Rất tốt (các phân vùng nhỏ vẫn hoạt động độc lập và hàn gắn sau)
Độ phức tạp truyền thông tạo Commit	Nặng nề (từ $\mathcal{O}(N)$ đến $\mathcal{O}(N^2)$ do phải trao đổi chéo)	Nặng nề (phát sóng khối)	Rất nhẹ ($\mathcal{O}(1)$ - Token Holder phát sóng một lần)
Độ trễ xác nhận (Latency)	Phụ thuộc vào RTT mạng (cần chờ xác nhận từ đa số)	Rất cao (phụ thuộc thời gian sinh khối)	Ngay lập tức và cục bộ (không cần biểu quyết)
Sự phù hợp cho P2P Chat	Kém (dễ bị treo mạng, tốn kém tài nguyên)	Không phù hợp (cấu trúc quá cồng kềnh)	Rất cao (linh hoạt, tiết kiệm băng thông)

4.10 Giới hạn lý thuyết của phương pháp

Phương pháp đề xuất không nhằm thay thế hoàn toàn các giao thức đồng thuận mạnh trong mọi bài toán phân tán. Nó được xây dựng cho bối cảnh mạng ngang hàng nơi các nút cần tiếp tục hoạt động cục bộ trong khi vẫn cố gắng giữ trạng thái nhóm không phân kỳ quá xa [3], [9]. Vì vậy, điểm mạnh của phương pháp nằm ở khả năng giảm khả năng phân nhánh trong điều kiện bình thường và tạo ra một đường quay về thống nhất khi phân nhánh đã xảy ra.

Single-Writer phát huy hiệu quả rõ nhất khi các nút còn chia sẻ cùng một ActiveView. Khi ActiveView giữa các phân vùng đã lệch nhau, chính các cơ chế đổi chiều trạng thái và Fork Healing mới trở thành phần quyết định khả năng hội tụ. Tương tự, quy tắc chọn nhánh không bảo đảm rằng mọi nút sẽ chọn cùng một kết quả ngay tức thì trong mọi thời điểm; để điều đó xảy ra, các nút phải có đủ thông

tin về các nhánh đang tồn tại.

Cuối cùng, HLC chỉ giải quyết thứ tự hiển thị thông điệp ứng dụng. Nó không tham gia vào việc xác nhận Commit, không thay thế kiểm tra epoch và cũng không sửa được các phân nhánh trạng thái. Việc giữ ranh giới này là điều cần thiết để tránh trộn lẫn hai tầng trách nhiệm vốn khác nhau.

Kết chương. Chương này đã phân tích cơ sở lý thuyết của lớp điều phối phi tập trung được đề xuất cho MLS trên mạng ngang hàng. Các lập luận cho thấy, trong phạm vi giả định của đồ án, phương pháp có thể bảo toàn được các bất biến quan trọng như việc epoch cục bộ chỉ giữ nguyên hoặc tăng, tính duy nhất của Token Holder trong cùng ActiveView, khả năng nhận biết phân nhánh sau bước đối chiếu trạng thái, nguyên tắc không gộp trực tiếp hai nhánh MLS đã tách và vai trò giới hạn của HLC ở tầng ứng dụng.

Những phân tích này cũng làm rõ giới hạn của phương pháp: Single-Writer không loại bỏ hoàn toàn mọi khả năng phân nhánh, còn việc hội tụ sau phân vùng mạng vẫn phụ thuộc vào khả năng trao đổi lại thông tin giữa các nút. Trên nền tảng đó, Chương 5 sẽ chuyển sang phần kiểm chứng thực nghiệm, nhằm đối chiếu các lập luận lý thuyết của chương này với dữ liệu đo đạc và các kịch bản chạy trên ứng dụng thử nghiệm.

CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM

Tổng quan chương.

Sau phần phân tích lý thuyết ở Chương 4, đồ án cần một bước kiểm chứng để trả lời câu hỏi liệu các cơ chế đã đề xuất có vận hành được trên hệ thống thực nghiệm hay không. Vì vậy, chương này trình bày phần đánh giá thực nghiệm của đồ án, trong đó ứng dụng desktop thử nghiệm được dùng như một môi trường kiểm chứng tích hợp, còn các bài kiểm thử tự động và dữ liệu benchmark được dùng để đối chiếu các thuộc tính đã phân tích ở chương trước.

Chương này tập trung vào ba nhóm nội dung. Nhóm thứ nhất đánh giá hiệu quả của cơ chế Single-Writer và gom đề xuất trong việc giảm xung đột Commit, sau đó kiểm chứng khả năng hội tụ và phục hồi sau phân vùng mạng. Nhóm thứ hai xem xét chi phí mật mã, chi phí điều phối và thuộc tính bảo mật còn được giữ lại khi hệ thống chạy với thành phần MLS thật. Nhóm thứ ba kiểm chứng các kịch bản thao tác trên ứng dụng thử nghiệm để làm rõ rằng các giả định của giao thức đã đi vào luồng sử dụng thật.

5.1 Mục tiêu và phạm vi đánh giá

Phần đánh giá của đồ án không chỉ nhằm đo hiệu năng thuần túy, mà trước hết nhằm kiểm chứng các luận điểm đã được xây dựng từ Chương 3 và Chương 4. Câu hỏi cần trả lời là: trong điều kiện mạng ngang hàng có thể mất kết nối, trễ gói và chia cắt, các nút có còn hội tụ về cùng trạng thái MLS hay không; cơ chế Single-Writer có còn giảm được xung đột Commit hay không; và việc tách lớp điều phối khỏi lõi MLS có giữ được các thuộc tính bảo mật chính hay không.

Từ đó, các mục tiêu đánh giá được chia thành ba hướng. Hướng thứ nhất xem xét hiệu quả vận hành của cơ chế Single-Writer và cơ chế gom đề xuất, tức là xem hệ thống có giảm được các lần tạo Commit cạnh tranh hay không. Hướng thứ hai kiểm tra tính đúng đắn của tiến trình điều phối, trong đó các đại lượng quan trọng là epoch cuối cùng, mã băm cây cuối cùng, số Commit hợp lệ trong mỗi epoch và khả năng phát hiện phân nhánh. Hướng thứ ba kiểm tra phần mật mã và an toàn, đặc biệt là thuộc tính Forward Secrecy sau khi một thành viên bị loại khỏi nhóm.

5.2 Môi trường và phương pháp thí nghiệm

Các thí nghiệm được thực hiện trực tiếp trên mã nguồn của hệ thống. Phần điều phối được viết bằng Go; phần MLS được triển khai bằng Rust/OpenMLS [22] và được Go khởi chạy dưới dạng tiến trình đi kèm qua gRPC [23] trên localhost. Với những kiểm thử cần cô lập logic điều phối, hệ thống sử dụng các thành phần giả

Table 5.1: Các tiêu chí đánh giá thực nghiệm

Tiêu chí	Ý nghĩa đánh giá
Hội tụ epoch	Điều kiện (i) trong Định nghĩa 3.1: các nút hợp lệ phải kết thúc ở cùng một epoch sau khi mạng ổn định trở lại.
Hội tụ mã băm cây	Điều kiện (ii) trong Định nghĩa 3.1: các nút hợp lệ phải có cùng mã băm cây MLS.
Single-Writer trong cùng epoch	Không xuất hiện hai Commit hợp lệ cạnh tranh trong cùng một epoch khi các nút có cùng ActiveView.
Hiệu quả gom đề xuất	Nhiều Proposal gần nhau có thể được gom vào một Commit thay vì sinh nhiều epoch liên tiếp.
Thời gian phục hồi phân vùng	Khoảng thời gian từ khi mạng được nối lại đến khi các nút đạt hội tụ trạng thái mật mã.
Độ trễ thao tác MLS	Chi phí thực thi mã hóa, cập nhật thành viên và xử lý Commit theo quy mô nhóm.
Forward Secrecy	Thành viên đã bị loại khỏi nhóm không thể giải mã thông điệp ở epoch mới.

lập như FakeNetwork, FakeClock và MockMLSEngine. Với những kiểm thử cần xác nhận thuộc tính mật mã, hệ thống dùng thành phần MLS thật để tránh rút ra kết luận từ dữ liệu mô phỏng thuần túy.

Dữ liệu đo đạc và các biểu đồ phục vụ chương này được lưu lại trong thư mục `evaluation`. Trong số đó, phần đối chiếu quan trọng nhất là so sánh giữa mô hình mã hóa từng cặp truyền thống và hướng xử lý MLS hiện tại của hệ thống. Cách so sánh này giúp làm rõ chi phí xử lý thay đổi như thế nào khi số lượng thành viên tăng lên.

Phạm vi đánh giá có hai giới hạn cần nêu trước. Thứ nhất, các chaos test chủ yếu dùng mạng giả lập để kiểm tra hành vi giao thức, nên chưa phản ánh đầy đủ độ trễ, mật gói và tính phức tạp của mạng libp2p thực tế. Thứ hai, các benchmark MLS phản ánh chi phí xử lý của lõi MLS cùng phần trao đổi trạng thái giữa các thành phần, chứ chưa phải là toàn bộ độ trễ mà người dùng cảm nhận ở mức ứng dụng desktop.

5.3 Đánh giá cơ chế Single-Writer và gom đề xuất

Một rủi ro lớn khi đưa MLS lên mạng ngang hàng là nhiều nút cùng tạo Commit cho cùng một epoch. Nếu điều đó xảy ra, cây MLS sẽ bị tách nhánh và các thành viên không còn cùng một trạng thái mật mã. Cơ chế Single-Writer được đưa ra để chặn đúng tình huống này: chỉ Token Holder của epoch hiện tại được tạo Commit, còn các nút khác chỉ phát tán Proposal.

Để đánh giá hiệu quả của cơ chế gom đề xuất, thí nghiệm so sánh hai chiến lược. Chiến lược thứ nhất xử lý ngay khi nhận Proposal, nên các Proposal đến muộn trong cùng một epoch dễ phải thử lại sau khi epoch đã đổi. Chiến lược thứ hai chờ một khoảng gom 1 giây để Token Holder đưa nhiều Proposal vào cùng một Commit.

Table 5.2: So sánh xử lý ngay và cơ chế gom đề xuất

Mức đồng thời	Xử lý ngay			Gom theo lô		
	Proposal	Commit	Tỷ lệ thành công	Proposal	Commit	Tỷ lệ thành công
1	1	1	1,00	1	1	1,00
2	3	2	0,67	2	1	1,00
3	6	3	0,50	3	1	1,00
4	10	4	0,40	4	1	1,00
5	15	5	0,33	5	1	1,00

ĐÁNH GIÁ HIỆU NĂNG PHÂN PHỐI ĐỀ XUẤT MLS (CONCURRENCY SWEEP EVALUATION)
So sánh Cơ chế Commit Ngay lập tức (Baseline) vs Gom Batch 1 Giây (Optimized)

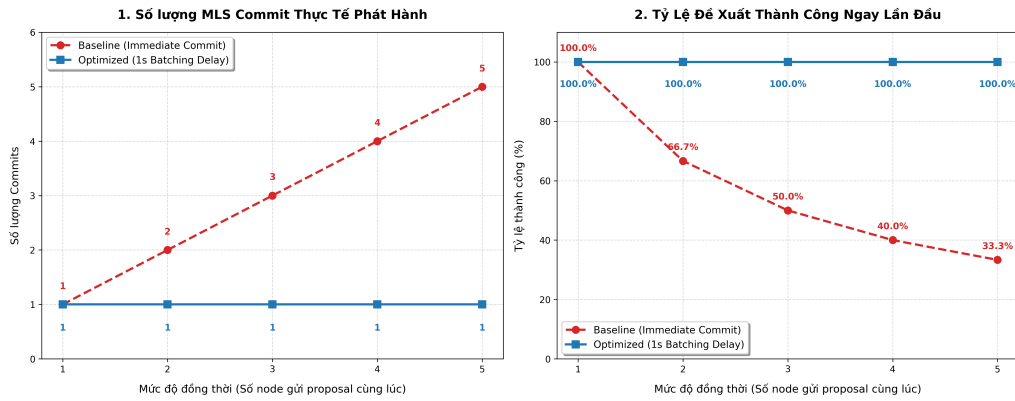


Figure 5.1: Hiệu quả của cơ chế gom đề xuất khi mức đồng thời tăng

Bảng 5.2 và Hình 5.1 cho thấy khi mức đồng thời tăng, chiến lược xử lý ngay phải tạo thêm Proposal do các nút thường rơi vào tình trạng gửi muộn so với epoch hiện hành. Trong khi đó, chiến lược gom theo lô giữ số Proposal đúng bằng số thao tác thực và vẫn chỉ cần một Commit. Kết quả này củng cố lập luận ở Chương 4 rằng vai trò của cơ chế Single-Writer không chỉ nằm ở tính đúng đắn, mà còn ở hiệu quả vận hành: hệ thống cố giảm xung đột Commit từ đầu thay vì xử lý hậu quả sau đó.

5.4 Đánh giá hội tụ trong điều kiện mạng hỗn loạn

Bài kiểm thử quan trọng nhất cho lớp điều phối là `TestIntegration_Chaos_Convergence`. Thí nghiệm tạo một cụm gồm năm nút cùng tham gia một nhóm MLS. Trong quá trình chạy, tiến trình thử nghiệm liên tục chia mạng thành hai phân vùng, giữ trạng thái phân vùng trong khoảng 600 ms rồi nối mạng lại. Đồng thời, các nút vẫn tiếp tục gửi thông điệp và phát sinh thao tác thay đổi thành viên.

Kết quả được ghi vào `evaluation/data/chaos_metrics.csv`, gồm

thời điểm đo, định danh nút, epoch hiện tại và mã băm cây rút gọn. Hình 5.2 minh họa quá trình các nút có thể tạm thời phân kỳ trong lúc mạng bị chia cắt, sau đó hội tụ về cùng epoch và cùng mã băm cây khi mạng được nối lại.

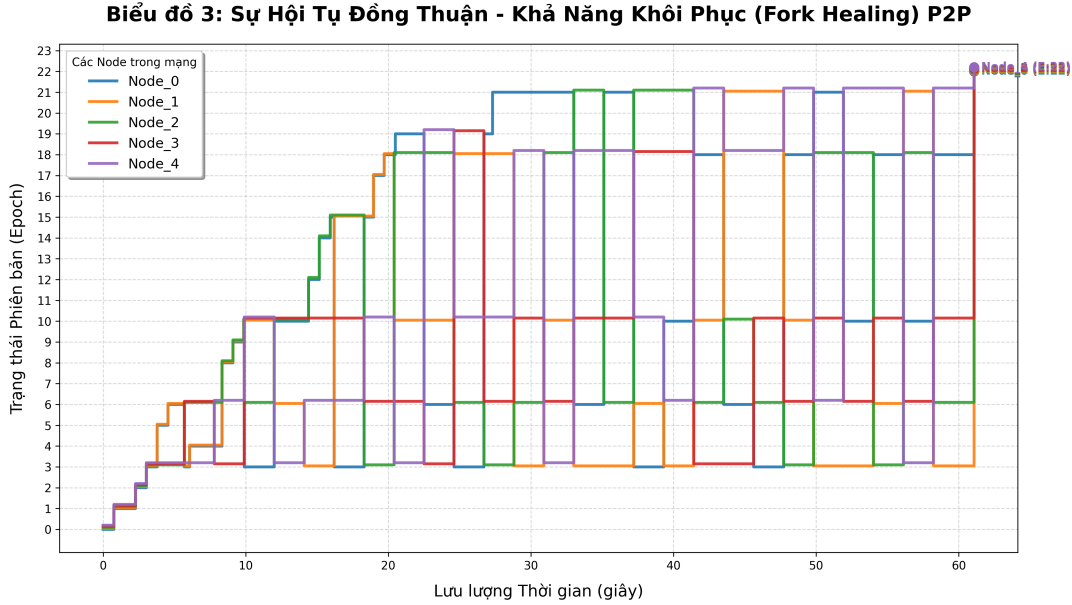


Figure 5.2: Hội tụ epoch của các nút trong chaos test

Kết quả cuối cùng cho thấy các nút hợp lệ hội tụ về cùng epoch và cùng mã băm cây. Điều này phù hợp với lập luận ở Chương 4: khác biệt trạng thái được nhận diện trước, sau đó các nút đối chiếu nhánh, chọn nhánh tiếp tục và đưa nhánh còn lại quay về một trạng thái thống nhất. Quan trọng hơn, quá trình này diễn ra mà không cần một Delivery Service trung tâm áp đặt thứ tự từ bên ngoài.

5.5 Ảnh hưởng độ sâu phân kỳ đến thời gian phục hồi

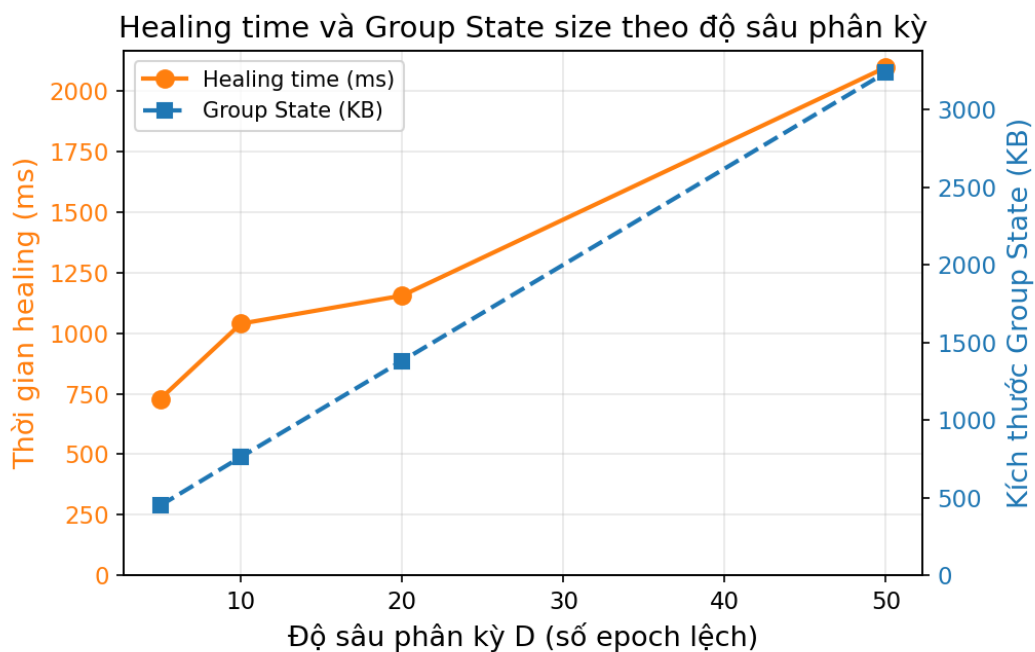
Khi hai nhánh MLS đã phân kỳ nhiều epoch, quá trình healing phải xử lý một khoảng cách trạng thái lớn hơn. Thí nghiệm này đo thời gian healing khi độ sâu phân kỳ, tức số epoch mà hai nhánh đã lệch nhau, tăng từ 5 đến 50. Cụm thử nghiệm gồm 32 nút, được chia thành hai phân vùng: nhánh thắng có 30 nút và nhánh thua có 2 nút. Trong thời gian phân vùng, nhánh thắng thực hiện tăng D epoch thật bằng các thao tác Commit, tạo ra groupState MLS lớn dần theo D . Nhánh thua tăng 2 epoch. Sau khi mạng được nối lại và kích hoạt fork detection, đoạn mã kiểm thử đo thời gian healing của nút thua bằng khoảng thời gian từ lúc hàm `runHeal()` bắt đầu cho đến khi hoàn thành, bao gồm các bước: snapshot orphan, tạo KeyPackage, phát tán ProposalJoin, chờ Welcome, hoán đổi trạng thái, phát lại thông điệp và dọn dẹp. Mỗi giá trị D được chạy 10 lần và lấy kết quả trung vị để giảm nhiễu.

Dữ liệu được trình bày ở Bảng 5.3 và minh họa ở Hình 5.3.

Bảng 5.3 cho thấy thời gian healing tăng từ 728 ms ở $D = 5$ lên 2098 ms ở

Table 5.3: Thời gian healing, kích thước group state và tỷ lệ healing/group state theo độ sâu phân kỳ (trung vị của 10 lần chạy)

D (epoch lệch)	Healing (ms)	Group State (KB)	ms/KB
5	728	449	1,62
10	1039	762	1,36
20	1155	1381	0,84
50	2098	3240	0,65

**Figure 5.3:** Thời gian healing (cam) và kích thước group state (xanh) theo độ sâu phân kỳ

$D = 50$. Hình 5.3 cho thấy đường healing time (cam) và đường kích thước group state (xanh) tăng cùng nhau, gợi ý rằng hai đại lượng này có liên hệ với nhau.

Tuy nhiên, sự tăng trưởng này không có nghĩa là giao thức healing phải replay D epoch. Quá trình healing luôn thực hiện đúng một chu kỳ: nút thua tạo KeyPackage mới, phát tán ProposalJoin, Token Holder trên nhánh thắng nhận ProposalJoin và chuyển thành Remove + Add, tạo một Commit duy nhất, gửi Welcome lại cho nút thua, nút thua xử lý Welcome và hoán đổi trạng thái. Số bước giao thức, số thông điệp trao đổi và số thao tác MLS không thay đổi theo D . Vậy câu hỏi đặt ra là: tại sao thời gian vẫn tăng?

Nguyên nhân nằm ở kích thước nhóm được triển khai trong thư viện OpenMLS. Mỗi thao tác ProposalUpdate tạo ra một cặp khóa HPKE mới cho nút lá trong ratchet tree, và OpenMLS [22] tuần tự hóa toàn bộ cây khi xuất group state. Sau 50 epoch, group state đã lớn tới 3240 KB so với 449 KB ở 5 epoch. Mọi lệnh gọi gRPC giữa lớp điều phối (Go) và lõi MLS (Rust) — bao gồm HasMember, CreateProposal, CreateCommit và ProcessWelcome — đều phải gửi nguyên group state qua mạng nội bộ. Khi group state lớn hơn, thời gian gRPC dài hơn, kéo theo thời gian chờ Welcome cũng dài hơn.

Để tách biệt hai thành phần này, đồ án tính tỷ lệ healing time trên kích thước group state (ms/KB) và trình bày ở Hình 5.4.

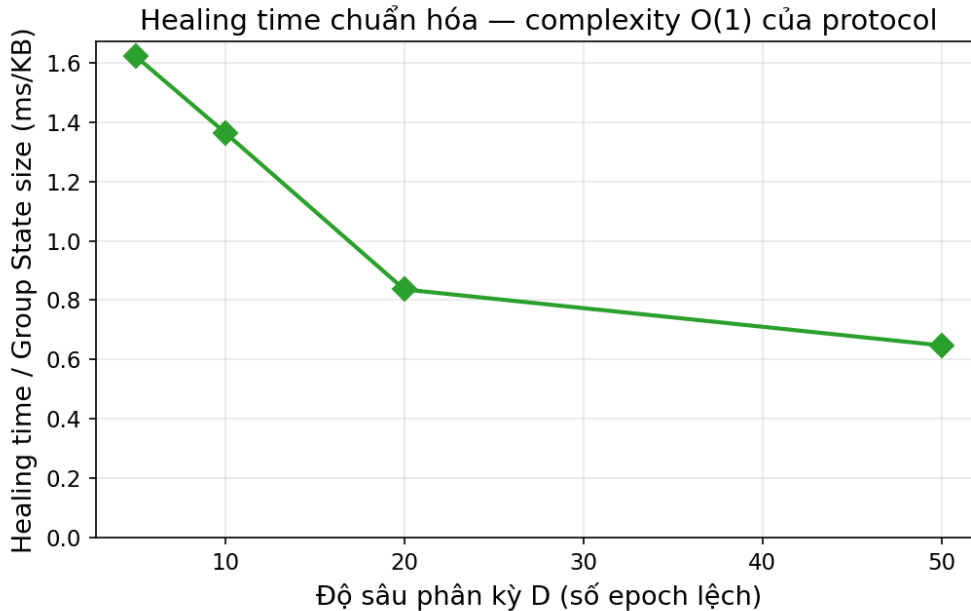


Figure 5.4: Healing time chuẩn hóa theo kích thước group state — tỷ lệ ms/KB giảm khi D tăng

Hình 5.4 cho thấy tỷ lệ ms/KB giảm từ 1,62 ở $D = 5$ xuống 0,65 ở $D = 50$. Nghĩa là khi group state lớn gấp 7 lần ($D = 50$ so với $D = 5$), thời gian healing chỉ

tăng khoảng 2,9 lần, chứ không tăng theo cùng tỷ lệ. Điều này cho phép tách thành hai thành phần:

Thứ nhất, **chi phí cố định của giao thức**: phần bầu Token Holder, phát tán ProposalJoin qua GossipSub, chờ Commit và gói tin Welcome, nút thua hoán đổi trạng thái MLS của mình sang nhóm mới và phát lại các tin nhắn trong nhánh cũ. Phần này không phụ thuộc vào D hay kích thước group state, nên là hằng số — tương ứng với độ phức tạp $O(1)$ của giao thức.

Thứ hai, **chi phí xử lý MLS**: phần serialize/deserialize group state trong các lệnh gọi gRPC đến lỗi Rust. Phần này tỉ lệ với kích thước group state, chứ không tỉ lệ với D . Khi group state lớn, phần này chiếm tỷ trọng cao hơn, làm cho tổng thời gian tăng. Nguyên nhân của chi phí này nằm ở hai yếu tố cộng lại: một là thư viện OpenMLS [22] tuần tự hóa toàn bộ ratchet tree khi xuất group state, làm cho trạng thái nhóm phình to qua từng epoch; hai là kiến trúc stateless của lỗi Rust — lỗi MLS không giữ trạng thái nhóm trong bộ nhớ mà nhận nguyên group state từ phía Go qua mỗi lệnh gọi gRPC, tính toán rồi trả lại trạng thái mới. Nếu lỗi MLS giữ trạng thái trong RAM như một thư viện liên kết trực tiếp, mỗi lần gọi chỉ cần gửi delta thay vì toàn bộ group state, chi phí này sẽ giảm đáng kể. Đây là hạn chế của kiến trúc triển khai hiện tại, không phải chi phí của giao thức điều phối.

Như vậy, giao thức fork healing đạt độ phức tạp $O(1)$ về số bước. Thời gian tăng theo kích thước group state là hệ quả của cách triển khai hiện tại — bao gồm cả đặc tính tuần tự hóa của OpenMLS và kiến trúc stateless của lỗi Rust — chứ không phải do bản thân giao thức phải xử lý theo D .

5.6 Đánh giá khả năng mở rộng của thao tác thành viên

Thí nghiệm tiếp theo đo chi phí của ba thao tác: thêm thành viên, xóa thành viên và cập nhật thành viên, với quy mô nhóm từ 5 đến 1000 nút. Dữ liệu chi tiết được trình bày ở Bảng 5.4.

Table 5.4: Chi phí thao tác thành viên theo quy mô nhóm

Quy mô nhóm	Thêm thành viên (ms)	Xóa thành viên (ms)	Cập nhật (ms)
5	0,00	0,52	0,00
10	0,53	1,05	0,52
50	6,19	7,26	6,59
100	24,53	23,53	24,67
250	130,47	129,70	128,83
500	527,36	531,18	517,97
750	1265,50	1169,21	1188,41
1000	2237,29	2194,79	2146,94

Kết quả cho thấy chi phí tăng rõ khi số lượng thành viên tăng. Điều này phù hợp với phân tích lý thuyết ở Chương 4: thao tác thay đổi thành viên luôn đắt hơn gửi application message thông thường, vì nó kéo theo Proposal, Commit và cập nhật trạng thái nhóm. Tuy vậy, kết quả này cần được hiểu là chi phí của pipeline hiện tại của ứng dụng thử nghiệm, không phải là giới hạn tuyệt đối của MLS về mặt lý thuyết.

5.7 Đánh giá chi phí mật mã MLS

Phần này dùng số liệu từ benchmark `crypto-engine/src/bin/mls_benchmark.rs`. Chương trình này tạo sẵn các trạng thái nhóm với kích thước từ 16 đến 4096 thành viên, rồi lặp lại mỗi phép đo 100 lần với payload 1024 byte. Thời gian của từng lần chạy được lấy bằng `Instant::now()`, sau đó sắp xếp lại để rút ra ba mốc là trung vị, p95 và p99. Trong chương này, bảng số liệu chỉ giữ giá trị trung vị để cách đọc gọn hơn.

Benchmark trên có hai đường đo chính. Đường mã hóa từng cặp, tương ứng với nhãn `pairwise_baseline` trong dữ liệu, mô phỏng cách gửi kiểu từng cặp, nghĩa là phía gửi phải thực hiện thao tác bọc khóa và mã hóa riêng cho từng người nhận. Đường MLS hiện tại, tương ứng với nhãn `current_full_blob_mls_encrypt`, đi theo đúng cách làm hiện nay của hệ thống: hàm `encrypt_message` nhận toàn bộ `group_state`, nạp trạng thái đó vào OpenMLS [22], tạo MLS message, tuần tự hóa bản mã rồi xuất lại trạng thái nhóm mới. Vì vậy, con số của đường MLS hiện tại không chỉ phản ánh phần mã hóa thuần túy, mà còn bao gồm cả chi phí nạp và xuất trạng thái.

Table 5.5: So sánh trung vị thời gian xử lý giữa hai hướng mã hóa, với payload 1024 byte và 100 mẫu cho mỗi điểm đo

Số nút	Mã hóa từng cặp (ms)	MLS (ms)
16	3,62	0,55
256	59,62	5,41
1024	238,75	19,86
4096	955,18	78,95

Table 5.6: So sánh p95 và p99 ở nhóm 4096 thành viên

Mốc đo	Từng cặp (ms)	MLS (ms)
p95	1023,66	95,04
p99	1081,72	110,85

Các con số trong Bảng 5.5 cho thấy hai xu hướng khá rõ. Thứ nhất, đường mã hóa từng cặp tăng gần tuyến tính theo số thành viên: từ 3,62 ms ở nhóm 16 thành viên lên 59,62 ms ở nhóm 256 thành viên, 238,75 ms ở nhóm 1024 thành viên và 955,18 ms ở nhóm 4096 thành viên. Thứ hai, đường MLS hiện tại cũng tăng theo quy mô nhóm, nhưng chậm hơn nhiều: các mốc tương ứng lần lượt là 0,55 ms, 5,41 ms, 19,86 ms và 78,95 ms. Điều này phù hợp với ưu thế cơ bản của MLS: phía gửi không phải mã hóa lại riêng cho từng người nhận [1].

Bảng 5.6 cho thấy kết luận trên không chỉ đúng ở mức trung vị. Ngay tại nhóm 4096 thành viên, p95 của đường từng cặp là 1023,66 ms và p99 là 1081,72 ms, trong khi đường MLS hiện tại có p95 là 95,04 ms và p99 là 110,85 ms. Điều đó cho thấy ngay cả ở các lần chạy chậm hơn bình thường, chênh lệch giữa hai hướng vẫn còn rất lớn.

Tuy nhiên, benchmark này cũng cho thấy điểm nghẽn của cách triển khai hiện tại. Trong mã nguồn, hàm `encrypt_message` không chỉ tạo bản mã mà còn phải nạp lại toàn bộ trạng thái nhóm và xuất lại trạng thái mới sau mỗi lần gửi. Vì vậy, dù tránh được chi phí mã hóa từng cặp, đường này vẫn tăng theo kích thước `group_state`. Đây là lý do làm cho thời gian đo ở nhóm lớn chưa thấp như một cách triển khai mà trạng thái được giữ sẵn trong bộ nhớ.

5.8 Chi phí phụ trợ của lớp điều phối

Ngoài phần mật mã, hệ thống còn tốn thêm thời gian cho lớp điều phối. Để đo riêng phần này, hệ thống thực hiện hai loại đo đạc. Loại thứ nhất thay lõi MLS bằng thành phần giả lập để cô lập riêng chi phí điều phối, ghi nhận dưới nhãn "Điều phối". Loại thứ hai dùng thành phần MLS thật qua gRPC, ghi nhận dưới nhãn "Điều phối + MLS". Hiệu giữa hai đường cho ra chi phí mật mã MLS. Dữ liệu được trình bày ở Bảng 5.7.

Bảng 5.7 cho thấy hai xu hướng rõ rệt. Thứ nhất, đường điều phối tăng theo quy mô nhóm nhưng ở mức thấp: từ 1,58 ms ở nhóm 16 thành viên lên 2204,85 ms ở nhóm 1000 thành viên. Thứ hai, đường tổng (điều phối + MLS) chỉ có dữ liệu đến nhóm 64 thành viên do giới hạn thời gian chạy của thành phần MLS thật, nhưng đã cho thấy chênh lệch rất lớn so với đường điều phối: 39,73 ms ở nhóm 16 và 650,70 ms ở nhóm 64. Phần lớn chênh lệch này nằm ở chi phí MLS, tương ứng 38,15 ms và 639,40 ms.

Tỷ trọng điều phối, tức phần chi phí không nằm ở MLS, giảm dần khi quy mô nhóm tăng: từ 3,97% ở nhóm 16 xuống 1,74% ở nhóm 64. Điều này cho thấy phần điều phối chỉ chiếm một tỷ lệ nhỏ của tổng chi phí cục bộ, và tỷ lệ này càng giảm khi nhóm càng đông. Phần tốn thời gian nhất vẫn nằm ở việc xử lý mật mã MLS.

Table 5.7: Chi phí xử lý cục bộ: Điều phối (giả lập MLS) và Điều phối + MLS (MLS thật)

Số thành viên	Điều phối (ms)	Tổng (ms)	MLS (ms)	Tỷ trọng (%)
16	1,58	39,73	38,15	3,97
32	3,08	115,43	112,35	2,67
64	11,30	650,70	639,40	1,74
128	35,55	—	—	—
256	134,86	—	—	—
512	539,69	—	—	—
1000	2204,85	—	—	—

Lớp điều phối có bổ sung thêm công việc như bầu Token Holder, kiểm tra epoch và giữ metadata cục bộ, nhưng phần bổ sung đó không phải là chỗ gây tốn thời gian chính. Nút thắt lớn hơn vẫn nằm ở việc xử lý và trao đổi trạng thái nhóm có kích thước ngày càng lớn.

5.9 Kiểm chứng Forward Secrecy bằng thành phần MLS thật

Một kiểm chứng bảo mật quan trọng là bài kiểm thử `TestBusinessP1_E2E_RealSidecar_ForwardSecrecy`. Khác với các kiểm thử điều phối dùng thành phần giả lập, bài kiểm thử này sử dụng trực tiếp thành phần MLS thật. Kịch bản gồm các bước: Alice tạo nhóm, Bob tham gia nhóm, Alice loại Bob khỏi nhóm, sau đó Alice gửi một thông điệp mới ở epoch sau khi Bob đã bị loại.

Thông điệp này vẫn được đưa vào pipeline xử lý phía Bob như một envelope nhận được từ mạng. Nếu hệ thống vận hành sai, Bob có thể giải mã và lưu plaintext vào cơ sở dữ liệu cục bộ. Kết quả kiểm thử cho thấy Bob không giải mã được thông điệp đó và không có plaintext nào được lưu lại. Điều này cho thấy việc đặt lớp điều phối bên ngoài MLS không làm suy giảm thuộc tính Forward Secrecy trong kịch bản được kiểm thử.

Kết quả này có ý nghĩa rõ ràng đối với mô hình bảo mật của đề án. Một nút từng là thành viên hợp lệ của nhóm nhưng đã bị loại khỏi cây MLS sẽ không còn quyền đọc các thông điệp tương lai. Lớp điều phối chỉ quyết định tiến trình trạng thái và thứ tự xử lý; quyền giải mã cuối cùng vẫn do trạng thái mật mã MLS kiểm soát.

5.10 Đối chiếu trên ứng dụng desktop

Bên cạnh các số đo lấy từ kiểm thử tự động, đề án còn đối chiếu thêm trên ứng dụng desktop của đề án. Phần này không nhằm giới thiệu giao diện cho đầy đủ, mà

để kiểm tra xem các đề xuất của đề tài đã nêu ở Chương 3 có thể thành một luồng sử dụng thực tế hay không. Vì vậy, ở mỗi kịch bản, đề án chỉ nói hai ý chính: người dùng làm gì trên giao diện, và thao tác đó cho thấy điều gì ở phía giao thức.

Các hình minh họa đầy đủ được đưa xuống Phụ lục A. Trong phần thân chương này, đề án chỉ chọn một số hình thực sự cần cho việc giải thích các kịch bản kiểm chứng, để mạch trình bày vẫn tập trung vào phần thực nghiệm.

5.10.1 Tạo tổ chức và cấp bundle

Kịch bản đầu tiên diễn ra trên hai máy. Ở máy quản trị, người dùng tạo tổ chức và mở phần quản trị để cấp bundle cho một thiết bị mới. Ở máy thành viên, người dùng tạo định danh cục bộ rồi dừng ở màn hình chờ bundle, nơi ứng dụng hiển thị PeerID và khóa công khai MLS của thiết bị. Nhìn ở mức thao tác, đây chỉ là một quy trình đăng ký. Nhưng nhìn ở mức giao thức, nó cho thấy một thiết bị không thể tự ý vào mạng của tổ chức; nó phải có bundle hợp lệ do phía quản trị cấp.

Các Hình 5.5 và Hình 5.6 trình bày hai đầu của quy trình này. Màn hình chờ bundle cho thấy thiết bị thành viên phải tự tạo định danh trước khi xin tham gia, còn màn hình quản trị cho thấy quyền cho tham gia nằm ở phía quản trị viên, chứ không do một máy chủ trung tâm tự động cấp phát. Chi tiết này bám đúng với giả định an toàn mà đề án đặt ra từ đầu.

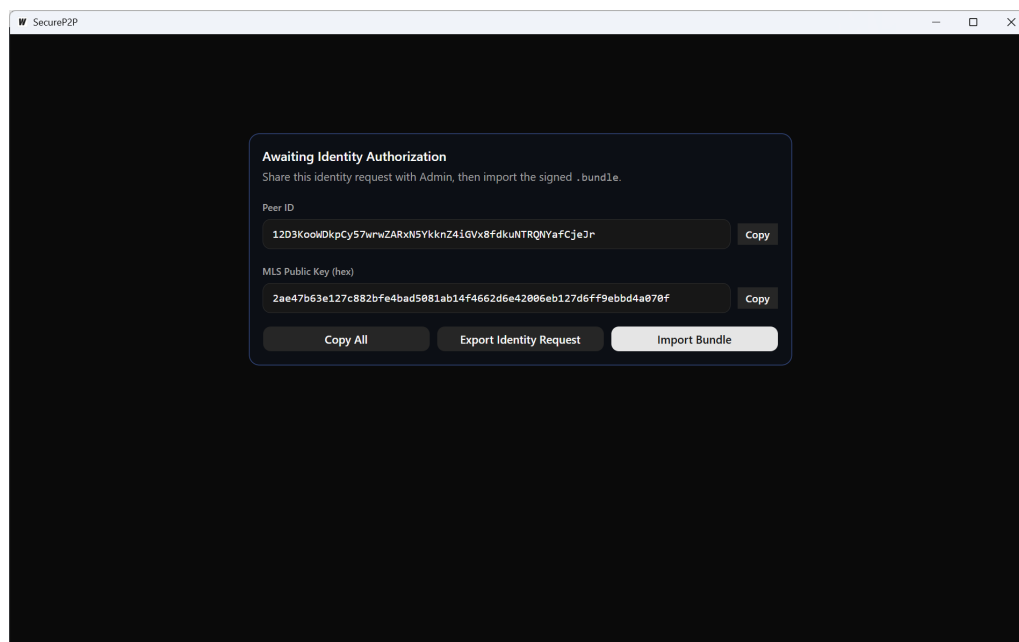


Figure 5.5: Thiết bị thành viên ở trạng thái chờ cấp phát bundle

5.10.2 Thiết bị mới vào hệ thống

Sau khi nhận được bundle, người dùng nhập bundle vào ứng dụng. Nếu bundle hợp lệ, ứng dụng chuyển từ màn hình chờ sang màn hình làm việc chính. Việc

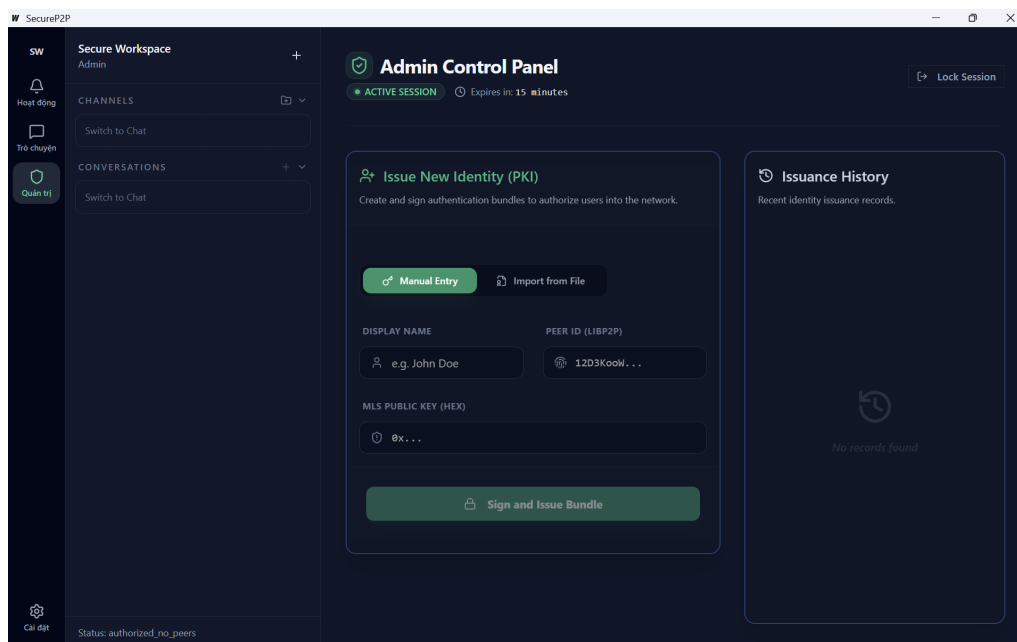


Figure 5.6: Bảng điều khiển quản trị phát hành bundle cho thiết bị mới

chuyển màn hình này nhìn qua thì đơn giản, nhưng về bản chất nó cho thấy thiết bị chỉ thực sự vào hệ thống sau khi bundle được kiểm tra xong, dữ liệu tổ chức được nạp vào máy cục bộ và các quan hệ nhóm cần thiết được khởi tạo.

Kịch bản này cũng cho thấy giao diện chỉ là lớp bên ngoài. Nó không tự quyết định thiết bị đã tham gia hay chưa; nó chỉ hiển thị kết quả sau khi phần điều phối và phần lưu trữ cục bộ cập nhật xong trạng thái mới.

5.10.3 Trao đổi tin nhắn trong nhóm

Kịch bản tiếp theo mở đồng thời các cửa sổ ứng dụng của Admin, Alice và Bob. Sau khi nhóm được tạo và thêm đủ thành viên, các bên cùng gửi tin nhắn vào một cuộc trò chuyện chung. Đây là phần dễ nhìn nhất của hệ thống, nhưng điều đáng chú ý không chỉ là việc tin nhắn hiện ra trên màn hình. Trước đó, các thay đổi về thành viên phải đi qua Proposal và Commit để mọi phía cùng ổn định về một trạng thái nhóm. Chỉ sau bước đó, các tin nhắn mới được mã hóa bằng MLS và hiển thị theo cùng một logic sắp xếp.

Các Hình 5.7, Hình 5.8 và Hình 5.9 minh họa các cửa sổ này để người đọc nhìn được trạng thái đồng bộ của cùng một nhóm ở nhiều phía khác nhau.

Đi cùng với nhắn tin là phần quản trị thành viên. Khi người dùng mở phần chi tiết phòng để xem danh sách thành viên, đổi chính sách mời hoặc thêm người mới, giao diện thực chất đang kích hoạt các thao tác thay đổi thành viên ở phía sau.

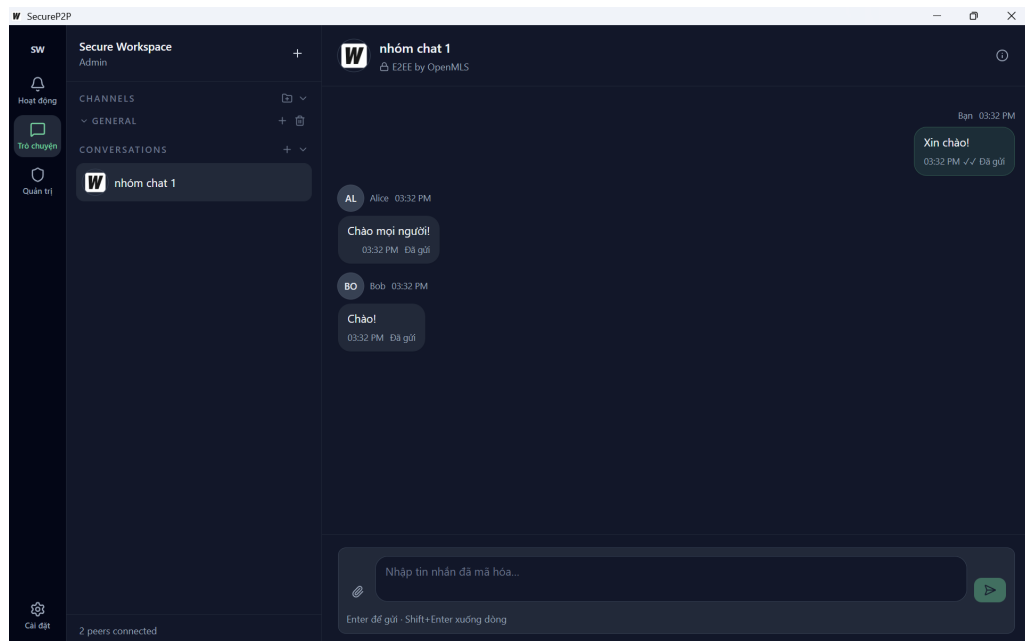


Figure 5.7: Cửa sổ giao tiếp nhóm trên thiết bị Admin

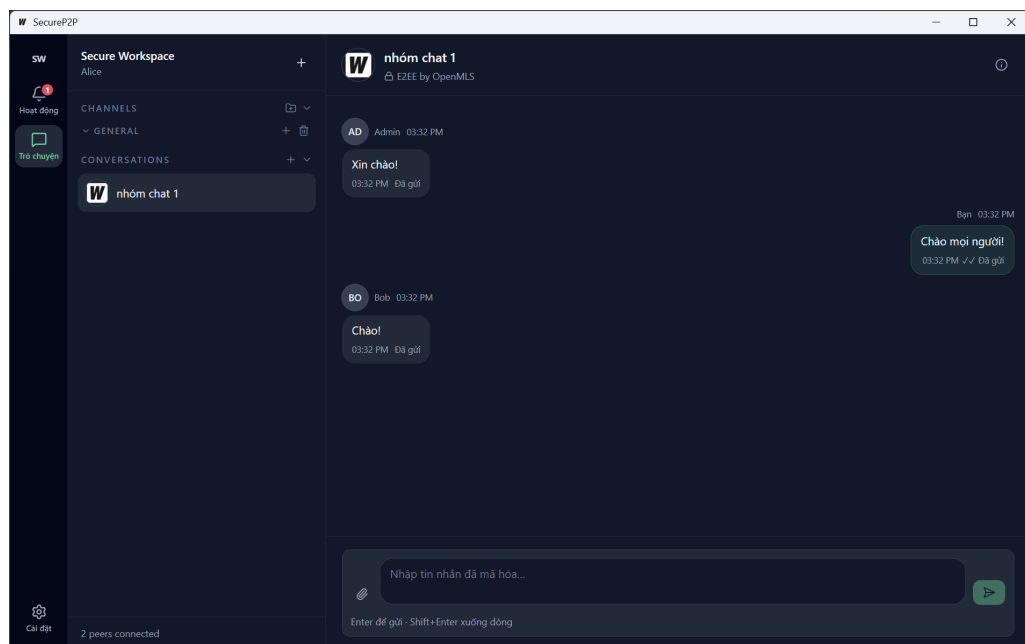


Figure 5.8: Cửa sổ giao tiếp nhóm trên thiết bị Alice

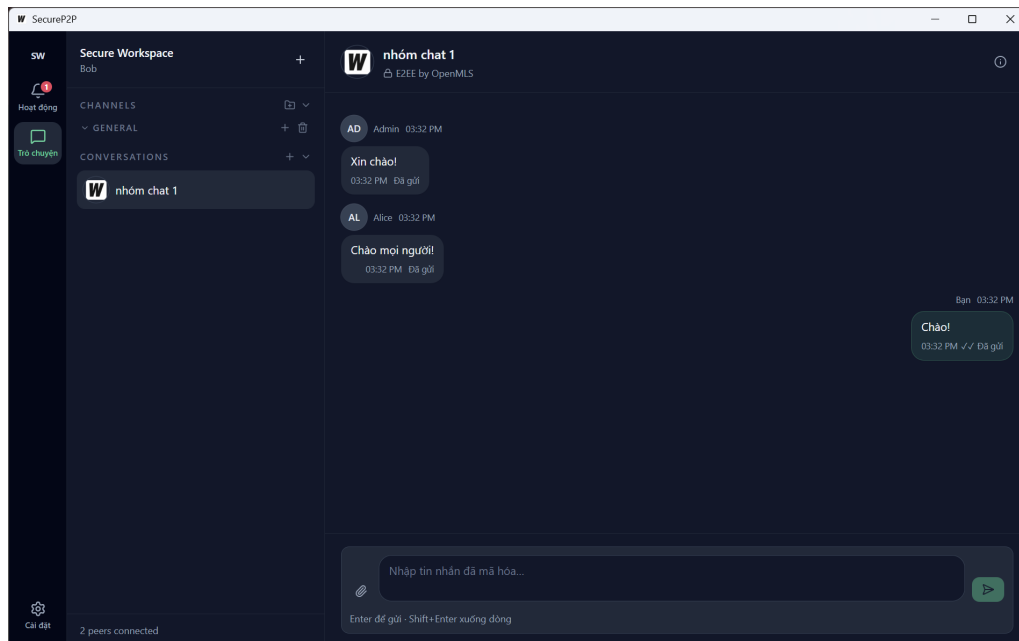


Figure 5.9: Cửa sổ giao tiếp nhóm trên thiết bị Bob

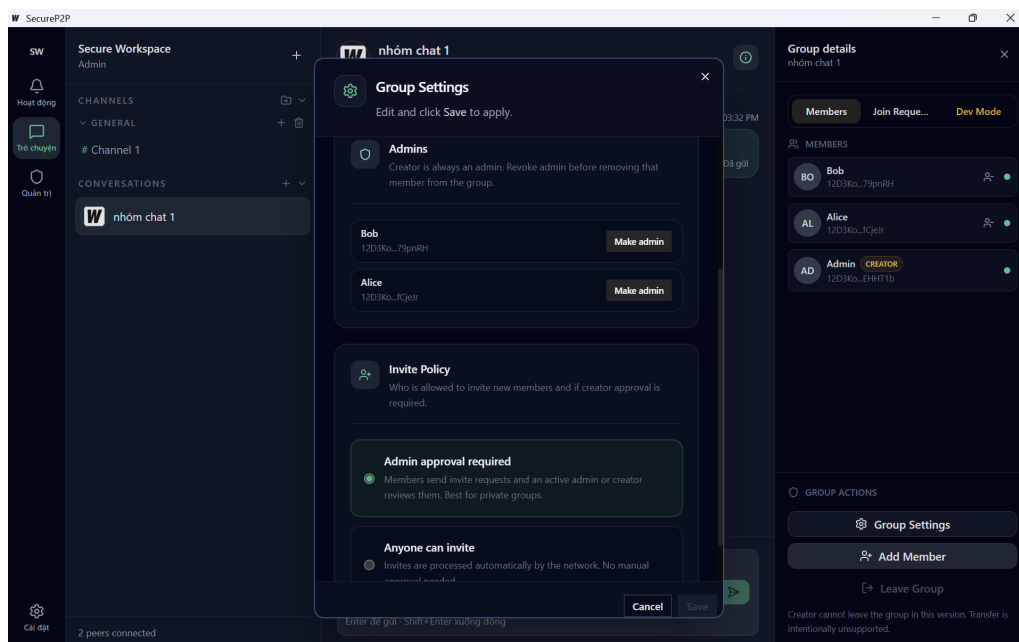


Figure 5.10: Màn hình quản trị thành viên của nhóm

5.11 Nhận xét và giới hạn đánh giá

Các kết quả thực nghiệm cho thấy hướng tiếp cận của đề án đạt được ba điểm quan trọng. Thứ nhất, cơ chế Single-Writer và cơ chế gom đề xuất giúp giảm xung đột Commit trong các kịch bản đã đo. Thứ hai, tiến trình đối chiếu trạng thái và Fork Healing dựa trên External Proposal cho phép các nút quay về cùng epoch và cùng mã băm cây sau phân vùng mạng. Thứ ba, khi kết hợp với thành phần MLS thật, hệ thống vẫn giữ được thuộc tính Forward Secrecy trong phạm vi kiểm thử của đề án.

Tuy nhiên, phần đánh giá vẫn còn giới hạn. Các chaos test hiện nay chủ yếu dùng mạng giả lập nên chưa phản ánh đầy đủ độ trễ, mất gói và hành vi NAT của môi trường thực. Các benchmark MLS cho thấy xu hướng chi phí rõ ràng, nhưng chưa phải là đánh giá đầu cuối của toàn bộ ứng dụng desktop trên nhiều máy độc lập. Thí nghiệm đo độ sâu phân kỳ cho thấy thời gian healing tăng theo kích thước group state khi độ sâu phân kỳ lớn, nhưng số bước giao thức vẫn không đổi — luôn là một ProposalJoin, một Commit và một Welcome. Phần thời gian tăng chủ yếu nằm ở chi phí serialize/deserialize group state trong các lệnh gọi gRPC đến lõi MLS, chứ không phải do giao thức phải replay từng epoch lệch. Do đó, mô hình trao đổi toàn trạng thái giữa các thành phần còn gây chi phí lớn khi nhóm rất đông thành viên; đây là điểm cần tiếp tục tối ưu nếu muốn tiến xa hơn khỏi phạm vi ứng dụng thử nghiệm.

Chương này đã kiểm chứng các cơ chế đã đề xuất bằng cả hai con đường: dữ liệu thực nghiệm và kịch bản chạy trên ứng dụng desktop thử nghiệm. Các kịch bản giao diện cho thấy những giả định thiết kế như cấp bundle, người mới gia nhập và quản trị thành viên đã được lập trình thành các usecase sử dụng thật. Các bài kiểm thử và benchmark tiếp tục cho thấy hệ thống có thể đạt hội tụ trạng thái mật mã sau phân vùng mạng, giảm xung đột Commit và giữ được các thuộc tính bảo mật quan trọng của MLS trong phạm vi đã đánh giá.

Những kết quả đó chưa đủ để đưa ra khẳng định tuyệt đối cho mọi môi trường triển khai, nhưng chúng là cơ sở thực nghiệm đáng tin cậy để kết lại phần nghiên cứu. Từ đây, Chương 6 sẽ tổng hợp lại đóng góp chính của đề án, nêu rõ giới hạn còn tồn tại và xác định các hướng phát triển tiếp theo.

CHƯƠNG 6. KẾT LUẬN

Tổng quan chương.

Sau khi đã trình bày phương pháp đề xuất, phân tích cơ sở lý thuyết và kiểm chứng bằng thực nghiệm, đồ án cần khép lại bằng một kết luận trung thực về những gì đã đạt được và những gì vẫn còn bỏ ngỏ. Vì vậy, chương này tổng hợp lại đóng góp chính của đồ án theo đúng trục nghiên cứu, đồng thời nêu rõ giới hạn hiện tại và các hướng phát triển tiếp theo.

Phần đầu của chương nhắc lại kết quả cốt lõi của đồ án ở cả ba mặt: đề xuất lớp điều phối phi tập trung cho MLS, triển khai ứng dụng thử nghiệm để kiểm chứng khả năng triển khai và đánh giá thực nghiệm để đối chiếu với các lập luận lý thuyết. Phần sau của chương tập trung vào giới hạn của nghiên cứu và những hướng cần tiếp tục làm sâu hơn nếu muốn đưa kết quả này tiến thêm một bước.

6.1 Kết luận chung

MLS [1] cung cấp bảo mật mạnh mẽ cho quá trình giao tiếp nhóm, nhưng giao thức này mặc định dựa vào một Delivery Service tập trung để áp đặt thứ tự Commit. Khi đưa MLS vào mạng ngang hàng, không còn thực thể trung tâm nào đảm nhiệm vai trò đó, và hệ quả là các nút dễ rơi vào tình trạng phân nhánh mật mã không thể tự khôi phục. Đồ án này đã giải quyết bài toán trên bằng cách đề xuất một lớp điều phối phi tập trung đặt bao quanh MLS, đồng thời xây dựng một ứng dụng desktop thử nghiệm để kiểm chứng tính khả thi của hướng tiếp cận.

Về mặt thiết kế, đóng góp cốt lõi là việc phối hợp bốn cơ chế thành một giao thức điều phối nhất quán: Single-Writer theo epoch để chỉ một Token Holder được quyền tạo Commit, giảm xung đột ngay từ đầu; kiểm tra epoch để loại bỏ thông điệp cũ trước khi đưa xuống lớp mật mã; phát hiện phân nhánh và Fork Healing để đưa các nút hội tụ trạng thái sau phân vùng mạng; và đồng hồ logic lai HLC để ổn định thứ tự hiển thị thông điệp ứng dụng độc lập với thứ tự mật mã. Điểm quan trọng là toàn bộ thiết kế không sửa đổi lõi mật mã của MLS mà chỉ bổ sung lớp điều phối ở phía ngoài, qua đó giữ nguyên các bảo đảm bảo mật mà RFC 9420 cung cấp.

Một đóng góp đáng chú ý trong Fork Healing là việc dùng External Proposal kết hợp Token Holder thay vì External Commit. Khi nhiều nút cùng cần gia nhập lại sau phân vùng, External Proposal cho phép Token Holder gom tất cả đề nghị vào một Commit duy nhất, thay vì để mỗi nút tự tạo Commit cạnh tranh. So với các thuật toán đồng thuận truyền thống như Raft hay BFT yêu cầu trao đổi chéo

$\mathcal{O}(N^2)$ thông điệp để chốt trạng thái, giao thức đề xuất chỉ cần Token Holder phát sóng một Commit, giữ độ phức tạp truyền thông ở mức $\mathcal{O}(1)$. Hơn nữa, các phân vùng nhỏ vẫn tiếp tục hoạt động độc lập khi mạng bị chia cắt thay vì đóng băng chờ quorum, phù hợp với đặc thù ứng dụng trò chuyện phi tập trung.

Về mặt triển khai, đồ án đã xây dựng một ứng dụng desktop tích hợp đầy đủ mạng P2P libp2p, cơ sở dữ liệu cục bộ SQLite và lõi mật mã MLS/OpenMLS qua kiến trúc sidecar stateless bằng Rust. Các luồng từ khởi tạo định danh, cấp bundle, quản trị tổ chức, tạo nhóm đến trao đổi tin nhắn được triển khai trọn vẹn, qua đó chứng minh rằng thiết kế giao thức không chỉ dừng ở mô tả lý thuyết mà có thể đi vào một hệ thống chạy được.

Về mặt đánh giá thực nghiệm, các kết quả đo đạc đã đối chiếu tích cực với các lập luận lý thuyết. Chaos test với cụm năm nút bị phân vùng lặp lại cho thấy các nút hội tụ về cùng epoch và cùng mã băm cây sau khi mạng được nối lại, không cần Delivery Service trung tâm. Thí nghiệm về cơ chế gom đề xuất cho thấy chiến lược Single-Writer giữ số Commit luôn bằng 1 và tỷ lệ thành công đạt 1,00, trong khi xử lý ngay tạo ra N Commit cạnh tranh với tỷ lệ thành công giảm dần khi số nút tăng. Benchmark mật mã cho thấy thời gian mã hóa MLS tăng theo $\mathcal{O}(\log N)$ trong khi mã hóa từng cặp tăng tuyến tính $\mathcal{O}(N)$ — ở nhóm 4096 thành viên, MLS chỉ tốn 78,95 ms so với 955,18 ms của mã hóa từng cặp, chênh lệch hơn 12 lần. Tỷ trọng chi phí điều phối so với tổng chi phí cục bộ chỉ chiếm dưới 4%, cho thấy lớp điều phối bổ sung không tạo gánh nặng đáng kể. Cuối cùng, kiểm thử với thành phần MLS thật xác nhận thuộc tính Forward Secrecy được giữ nguyên sau khi loại thành viên, chứng tỏ lớp điều phối không phá vỡ các bảo đảm mật mã của MLS.

6.2 Giới hạn của đồ án

Bên cạnh các kết quả đã đạt được, đồ án vẫn còn những giới hạn cần được nhìn nhận thẳng thắn. Trước hết, phần phân tích lý thuyết trong đồ án mới dừng ở mức lập luận theo bất biến và theo giả định vận hành chính, chưa đi tới kiểm chứng hình thức đầy đủ cho toàn bộ giao thức. Vì vậy, các kết luận của đồ án cần được hiểu trong phạm vi giả định đã nêu, không nên suy rộng thành một bảo đảm tuyệt đối cho mọi môi trường phân tán.

Tiếp theo, phần đánh giá thực nghiệm hiện nay vẫn chủ yếu dựa trên mạng giả lập và các bài kiểm thử có kiểm soát. Cách làm này phù hợp với phạm vi một đồ án tốt nghiệp vì nó cho phép cô lập các cơ chế cần kiểm tra. Tuy nhiên, nó chưa phản ánh hết độ phức tạp của mạng libp2p thực tế, đặc biệt khi có độ trễ lớn, thay đổi kết nối liên tục hoặc số lượng thiết bị tăng mạnh.

Một giới hạn khác đến từ chính ứng dụng thử nghiệm hiện tại. Trong nhiều thao

tác, hệ thống vẫn trao đổi toàn bộ trạng thái nhóm giữa phần điều phối và phần MLS. Cách làm này giúp cấu trúc triển khai rõ ràng, nhưng tạo thêm chi phí khi quy mô nhóm tăng. Vì vậy, ứng dụng này hiện nay phù hợp với vai trò kiểm chứng khả thi, chưa phải là một phương án tối ưu cho triển khai quy mô lớn.

Thêm vào đó, cơ chế tự mã hóa và phát lại tin nhắn vẫn tồn tại điểm yếu liên quan đến bài toán trải nghiệm người dùng. Nếu một thành viên gửi tin nhắn trong lúc mạng phân mảnh nhưng lại ngoại tuyến trước khi mạng hồi phục và hợp nhất nhánh, hệ thống sẽ tạm thời mất đi thông điệp đó cho đến khi người gửi trực tuyến trở lại, do không ai khác có được khóa bí mật để mã hóa bản rõ thay cho họ.

Một giới hạn khác liên quan đến độ sâu phân kỳ. Thí nghiệm cho thấy thời gian healing tăng khi hai nhánh đã lệch nhiều epoch, từ 1039 ms ở độ sâu 10 lên 2098 ms ở độ sâu 50. Mặc dù giao thức fork healing luôn thực hiện đúng một chu kỳ bất kể độ sâu phân kỳ và External Proposal giữ số Commits luôn bằng 1, phần chi phí nằm ở kích thước group state phình to qua từng epoch — do OpenMLS tuần tự hóa toàn bộ ratchet tree và lỗi Rust stateless nhận nguyên group state qua gRPC mỗi lần gọi. Đây là hạn chế của kiến trúc triển khai hiện tại, là điểm cần tối ưu thêm nếu muốn hệ thống phục hồi nhanh hơn sau phân vùng kéo dài.

Cuối cùng, cần nhấn mạnh rằng ứng dụng desktop trong đề án chỉ đóng vai trò ứng dụng thử nghiệm. Nó chứng minh rằng các cơ chế được đề xuất có thể đi vào luồng sử dụng thực tế, nhưng chưa đủ để xem như một sản phẩm hoàn thiện. Đóng góp chính của đề án vẫn là lớp điều phối phi tập trung cho MLS trên mạng ngang hàng, còn ứng dụng chỉ là phương tiện triển khai và kiểm chứng hướng tiếp cận đó.

6.3 Hướng phát triển

Hướng phát triển đầu tiên là làm chặt hơn phần cơ sở lý thuyết. Cụ thể, các bất biến như tính an toàn của cơ chế Single-Writer, tính đơn điệu của epoch và điều kiện đạt hội tụ trạng thái mật mã sau Fork Healing (Định nghĩa 3.1) cần được mô hình hóa và kiểm chứng hình thức đầy đủ hơn. Nếu làm được điều này, giá trị học thuật của hướng nghiên cứu sẽ được nâng lên rõ rệt.

Hướng phát triển thứ hai là mở rộng thực nghiệm trên môi trường gần với triển khai thật hơn. Các thí nghiệm nên được chạy trên nhiều tiến trình, nhiều máy và trong các điều kiện mạng đa dạng hơn, chẳng hạn độ trễ cao, mất gói, thay đổi thành viên liên tục hoặc phân vùng mạng kéo dài. Khi đó, đề án có thể cung cấp được bức tranh đầy đủ hơn về độ ổn định và khả năng chịu tải của hệ thống.

Hướng phát triển thứ ba là tối ưu hóa khâu truyền thông giữa phần điều phối và phần MLS. Việc giảm chi phí trao đổi trạng thái, tổ chức lại cách lưu trữ hoặc

áp dụng các kỹ thuật snapshot hiệu quả hơn có thể giúp hệ thống giữ được kiến trúc tách lớp hiện tại nhưng hoạt động tốt hơn khi quy mô nhóm tăng. Đặc biệt, phần khôi phục trạng thái khi độ sâu phân kỳ lớn cần được tối ưu để giảm thời gian healing, chẳng hạn qua việc tăng tốc xử lý Welcome hoặc áp dụng cơ chế checkpoint định kỳ để giới hạn số epoch cần khôi phục.

Hướng phát triển cuối cùng là tiếp tục hoàn thiện ứng dụng thử nghiệm theo hướng phục vụ tốt hơn cho kiểm chứng và cho trình diễn. Các công cụ quan sát trạng thái, kiểm soát thay đổi thành viên và theo dõi quá trình hội tụ nếu được làm rõ hơn sẽ giúp việc đánh giá hệ thống thuận lợi hơn ở các nghiên cứu tiếp theo.

Kết chương. Chương này đã tổng hợp lại những đóng góp chính, giới hạn và hướng phát triển của đề án. Có thể thấy kết quả quan trọng nhất của đề án không nằm ở một ứng dụng đơn lẻ, mà nằm ở việc đề xuất được một lớp điều phối phi tập trung cho MLS trên mạng ngang hàng, rồi kiểm chứng được tính khả thi của hướng tiếp cận đó bằng cả phân tích lý thuyết, thực nghiệm và ứng dụng thử nghiệm chạy được.

Những phần tài liệu tham khảo và phụ lục ở sau chương này đóng vai trò đối chiếu nguồn và bổ sung minh họa cho các nội dung đã trình bày. Chúng giúp người đọc lần lại cơ sở học thuật của đề án, đồng thời quan sát rõ hơn các màn hình và kịch bản của ứng dụng thử nghiệm đã được dùng trong phần kiểm chứng.

TÀI LIỆU THAM KHẢO

- [1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420, July 2023. DOI: 10.17487/RFC9420 visited on June 22, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9420.html>
- [2] B. Beurdouche, E. Rescorla, E. Omara, S. Inguva, and A. Duric, *The Messaging Layer Security (MLS) Architecture*, RFC 9750, Apr. 2025. DOI: 10.17487/RFC9750 visited on June 22, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9750.html>
- [3] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, Ser. Onward! ’19, 2019. DOI: 10.1145/3359591.3359737 visited on June 22, 2026. [Online]. Available: <https://martin.kleppmann.com/2019/10/23/local-first-at-onward.html>
- [4] T. Perrin, M. Marlinspike, and R. Schmidt, *The double ratchet algorithm*, Signal Specifications, Revision 4, Published 2025-11-04, 2025. visited on June 22, 2026. [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>
- [5] J. Alwen, M. Mularczyk, and Y. Tselekounis, “Fork-resilient continuous group key agreement,” in *Advances in Cryptology – CRYPTO 2023*, Cryptology ePrint Archive, Paper 2023/394, Springer, 2023, pp. 396–429. DOI: 10.1007/978-3-031-38551-3_13 visited on June 22, 2026. [Online]. Available: <https://eprint.iacr.org/2023/394>
- [6] K. Kohbrok, *Decentralized Messaging Layer Security*, Internet-Draft draft-kohbrok-mls-dmls-03, Expired Internet-Draft, work in progress, Oct. 2025. visited on June 22, 2026. [Online]. Available: <https://datatracker.ietf.org/doc/draft-kohbrok-mls-dmls/>
- [7] D. Balbás, D. Collins, and P. Gajland, *Analysis and improvements of the sender keys protocol for group messaging*, arXiv preprint arXiv:2301.07045, Appears in RECSI 2022, 2023. DOI: 10.48550/arXiv.2301.07045 visited on June 22, 2026. [Online]. Available: <https://arxiv.org/abs/2301.07045>
- [8] J. Ginesin and C. Nita-Rotaru, *The matrix reloaded: A mechanized formal analysis of the matrix cryptographic suite*, arXiv preprint arXiv:2408.12743,

2024. DOI: 10.48550/arXiv.2408.12743 visited on June 22, 2026. [Online]. Available: <https://arxiv.org/abs/2408.12743>
- [9] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. DOI: 10.1145/564585.564601
- [10] F. B. Schneider, “Implementing fault-tolerant services using the state machine approach: A tutorial,” *ACM Computing Surveys*, vol. 22, no. 4, pp. 299–319, Dec. 1990. DOI: 10.1145/98163.98167 visited on June 26, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/98163.98167>
- [11] M. Castro and B. Liskov, “Practical byzantine fault tolerance,” in *3rd Symposium on Operating Systems Design and Implementation (OSDI 99)*, New Orleans, LA: USENIX Association, Feb. 1999, pp. 173–186. visited on June 26, 2026. [Online]. Available: <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>
- [12] L. Lamport, “Paxos made simple,” *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, Dec. 2001. visited on June 26, 2026. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/paxos-made-simple/>
- [13] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA: USENIX Association, 2014, pp. 305–319. visited on June 26, 2026. [Online]. Available: <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [14] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, Whitepaper, 2008. visited on June 26, 2026. [Online]. Available: <https://bitcoin.org/en/bitcoin-paper>
- [15] R. Barnes, K. Bhargavan, B. Lipp, and C. A. Wood, *Hybrid Public Key Encryption*, RFC 9180, Feb. 2022. DOI: 10.17487/RFC9180 visited on June 22, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9180.html>
- [16] D. A. McGrew, *An Interface and Algorithms for Authenticated Encryption*, RFC 5116, Jan. 2008. DOI: 10.17487/RFC5116 visited on June 26, 2026. [Online]. Available: <https://www.rfc-editor.org/info/rfc5116>

- [17] libp2p, *Libp2p concepts documentation*, Official documentation. visited on June 22, 2026. [Online]. Available: <https://docs.libp2p.io/concepts/>
- [18] D. Vyzovitis, Y. Napora, D. McCormick, D. Dias, and Y. Psaras, *Gossipsub: Attack-resilient message propagation in the filecoin and eth2.0 networks*, arXiv preprint arXiv:2007.02754, July 2020. DOI: 10.48550/arXiv.2007.02754 visited on June 26, 2026. [Online]. Available: <https://arxiv.org/abs/2007.02754>
- [19] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978. DOI: 10.1145/359545.359563
- [20] S. S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva, and M. Leone, “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases,” in *Proceedings of the 18th International Conference on Principles of Distributed Systems (OPODIS 2014)*, Ser. Lecture Notes in Computer Science, vol. 8878, Springer, 2014, pp. 17–32. visited on June 22, 2026. [Online]. Available: <https://cse.buffalo.edu/tech-reports/2014-04.pdf>
- [21] W. Vogels, “Eventually consistent,” *ACM Queue*, vol. 6, no. 6, pp. 14–19, Oct. 2008. DOI: 10.1145/1466443.1466448 visited on June 26, 2026. [Online]. Available: <https://queue.acm.org/detail.cfm?id=1466448>
- [22] OpenMLS Project, *Openmls — a rust implementation of the messaging layer security (mls) protocol*, Official documentation, 2026. visited on June 26, 2026. [Online]. Available: <https://openmls.tech/>
- [23] gRPC Project, *Grpc — a high performance open source universal rpc framework*, Official documentation, 2026. visited on June 26, 2026. [Online]. Available: <https://grpc.io/>
- [24] Wails Project, *Wails — build desktop applications using go & web technologies*, Official documentation, 2026. visited on June 26, 2026. [Online]. Available: <https://wails.io/>
- [25] D. R. Hipp, *Sqlite — a self-contained, serverless, zero-configuration sql database engine*, Official documentation, 2026. visited on June 26, 2026. [Online]. Available: <https://www.sqlite.org/>

PHỤ LỤC

A. ỨNG DỤNG DEMO VÀ KỊCH BẢN MINH HỌA

Tổng quan chương: Phụ lục này trình bày chi tiết về kiến trúc phần mềm, các công nghệ nền tảng, kịch bản sử dụng (use cases) và các hình ảnh minh họa giao diện của ứng dụng desktop thử nghiệm. Ứng dụng này được phát triển nhằm triển khai giao thức điều phối phi tập trung đã đề xuất tại Chương 3, đồng thời đóng vai trò là nền tảng kiểm chứng thực nghiệm trong Chương 5. Thay vì chỉ trình diễn một số giao diện rời rạc, ứng dụng được thiết kế và cài đặt hoàn chỉnh với các luồng nghiệp vụ thực tế từ khởi tạo định danh, kết nối mạng ngang hàng, bảo mật thông điệp, cho đến quản trị hệ thống.

A.1 Kiến trúc phần mềm và công nghệ nền tảng

Ứng dụng thử nghiệm được phát triển dưới dạng một ứng dụng desktop dựa trên framework **Wails** [24], cho phép kết hợp sức mạnh của ngôn ngữ Go ở tầng backend và các công nghệ web hiện đại ở tầng frontend. Kiến trúc hệ thống được chia thành ba lớp độc lập nhằm đảm bảo tính bảo mật, dễ dàng bảo trì và tối ưu hiệu suất.

Lớp Giao diện người dùng (Frontend): Thành phần frontend được xây dựng bằng **React** kết hợp với **TypeScript**, đảm bảo tính an toàn kiểu dữ liệu. Giao diện được thiết kế theo hệ thống thành phần của **Shadcn UI** và **Tailwind CSS**, mang lại trải nghiệm người dùng hiện đại và nhất quán. Quản lý trạng thái nội bộ được thực hiện thông qua **Zustand**, cho phép bóc tách logic hiển thị khỏi logic đồng bộ dữ liệu. Mọi thao tác của người dùng không trực tiếp can thiệp vào tầng giao thức mà được chuyển tiếp qua các lệnh gọi API (Wails Bindings) xuống lớp Runtime.

Lớp Điều phối và Dịch vụ (Go Backend): Thành phần backend được phát triển bằng ngôn ngữ **Go**, đóng vai trò quản lý toàn bộ vòng đời ứng dụng. Lớp này đảm nhiệm ba chức năng cốt lõi:

- *Đồng thuận và mạng ngang hàng:* Tích hợp thư viện **libp2p** để quản lý kết nối phân tán, phụ trách việc khám phá các nút khác trong mạng và GossipSub cho cơ chế truyền thông điệp.
- *Điều phối tiến trình MLS:* Triển khai các cơ chế Single-Writer, phục hồi phân nhánh (fork healing) và quản lý epoch của nhóm chat.
- *Lưu trữ cục bộ:* Toàn bộ trạng thái và khóa mật mã được mã hóa và lưu trữ an toàn trong cơ sở dữ liệu **SQLite** [25] tại thiết bị của người dùng, đảm bảo nguyên tắc phi tập trung tuyệt đối.

Lớp Mật mã (Rust Crypto Engine): Để giải quyết các rào cản về hiệu năng

và độ an toàn bộ nhớ trong việc tính toán mật mã, các tác vụ liên quan đến chuẩn MLS (RFC 9420) được tách riêng thành một tiến trình Sidecar viết bằng ngôn ngữ **Rust**. Lớp này hoạt động hoàn toàn phi trạng thái (stateless), giao tiếp với tiến trình Go thông qua **gRPC** [23]. Tiến trình Rust chỉ nhận yêu cầu và trạng thái hiện tại từ Go, thực hiện các phép tính toán cây khóa MLS (sử dụng OpenMLS [22]), sau đó trả về kết quả để Go lưu trữ vào SQLite. Thiết kế này phân tách hoàn toàn tầng điều phối mạng và tầng tính toán mật mã.

A.2 Các kịch bản sử dụng (Use Cases) cốt lõi

Ứng dụng được thiết kế để hỗ trợ ba nhóm kịch bản sử dụng (Use Cases) chính, tương ứng với vòng đời của một nút mạng trong hệ thống phi tập trung:

1. Kịch bản gia nhập và quản lý định danh (Identity & Onboarding): Thiết bị lần đầu tham gia mạng lưới phải tự sinh cặp khóa mật mã cục bộ và tạo Yêu cầu tham gia (Join Request). Yêu cầu này được gửi tới một Quản trị viên đã có thẩm quyền. Hệ thống không sử dụng máy chủ trung tâm để xác thực, do đó định danh của thiết bị mới chỉ có hiệu lực khi Quản trị viên duyệt và phát hành một *Bundle* (gói thông tin định tuyến) chứa chữ ký hợp lệ.

2. Kịch bản tạo lập và quản trị nhóm (Group Lifecycle & Administration): Bất kỳ định danh hợp lệ nào cũng có thể khởi tạo một nhóm hội thoại mới. Người khởi tạo tự động trở thành quản trị viên của nhóm đó và có quyền phân bổ vai trò, cập nhật chính sách truy cập. Các kịch bản mời thành viên mới (Add/Invite) hay loại bỏ thành viên (Remove) đều trực tiếp kích hoạt các tiến trình *Proposal* và *Commit* theo chuẩn MLS, đảm bảo tính bảo mật tiến (Forward Secrecy) và bảo mật lui (Post-Compromise Security).

3. Kịch bản trao đổi thông tin cộng tác (Collaboration & Messaging): Sau khi danh sách thành viên nhóm được đồng thuận, các nút mạng có thể trao đổi thông điệp an toàn. Ứng dụng hỗ trợ hai luồng giao tiếp chính: Hội thoại tuyến tính (Chat) cho việc nhắn tin trực tiếp theo thời gian thực và Không gian kênh (Channel) theo mô hình bài viết – phản hồi, phù hợp cho việc lưu trữ thông tin có tổ chức và theo dõi luồng công việc (Activity Feed).

4. Kịch bản chia sẻ tệp mã hóa (Encrypted File Sharing): Bên cạnh tin nhắn văn bản, ứng dụng cho phép người dùng chia sẻ tệp tin trong nhóm một cách bảo mật. Khi người dùng chọn một tệp để gửi, ứng dụng tính toán giá trị băm SHA-256 của tệp, sau đó dùng hàm dẫn xuất khóa exporter của MLS theo chuẩn RFC 9420 để tạo ra một cặp khóa AES-256 và nonce cơ sở. Tệp được chia thành các phần nhỏ, mỗi phần mã hóa riêng bằng AES-GCM với nonce khác nhau. Các phần đã mã hóa được lưu tạm trên máy người gửi, sau đó người nhận tự kéo về qua một luồng trực

tiếp giữa hai nút libp2p. Quá trình giải mã diễn ra cục bộ tại máy người nhận, kèm theo việc kiểm tra lại giá trị băm để đảm bảo tệp không bị thay đổi. Khóa mã hóa tệp không bao giờ truyền qua mạng, chỉ những thành viên đang ở cùng kỷ nguyên với người gửi mới có thể dẫn xuất lại khóa và giải mã được tệp.

A.3 Hình ảnh minh họa các thành phần chức năng

Việc ánh xạ các kịch bản sử dụng nêu trên vào giao diện đồ họa đòi hỏi ứng dụng phải tổ chức các màn hình tương tác một cách hợp lý và trực quan. Phần dưới đây sẽ minh họa chi tiết các giao diện đã được triển khai.

A.3.1 Quy trình gia nhập hệ thống và cấp phát Bundle

Giai đoạn đầu tiên của quá trình vận hành là việc thiết bị mới thực hiện gia nhập mạng lưới. Hình A.1 minh họa giao diện tạo định danh trên thiết bị của thành viên mới. Trong khi đó, Hình A.2 thể hiện giao diện khởi tạo tổ chức gốc thuộc về phía quản trị viên. Việc tách biệt hai quy trình này phản ánh nguyên tắc bảo mật của hệ thống: thiết bị thành viên tự chịu trách nhiệm sinh khóa cục bộ. Sau khi hoàn tất khởi tạo định danh cục bộ, thiết bị thành viên chuyển sang trạng thái chờ cấp phát Bundle (Hình A.3). Quản trị viên sử dụng bảng điều khiển quản trị để xác thực và phát hành bundle (Hình A.4). Các giao diện này thể hiện cơ chế kiểm soát truy cập nghiêm ngặt của mạng ngang hàng.

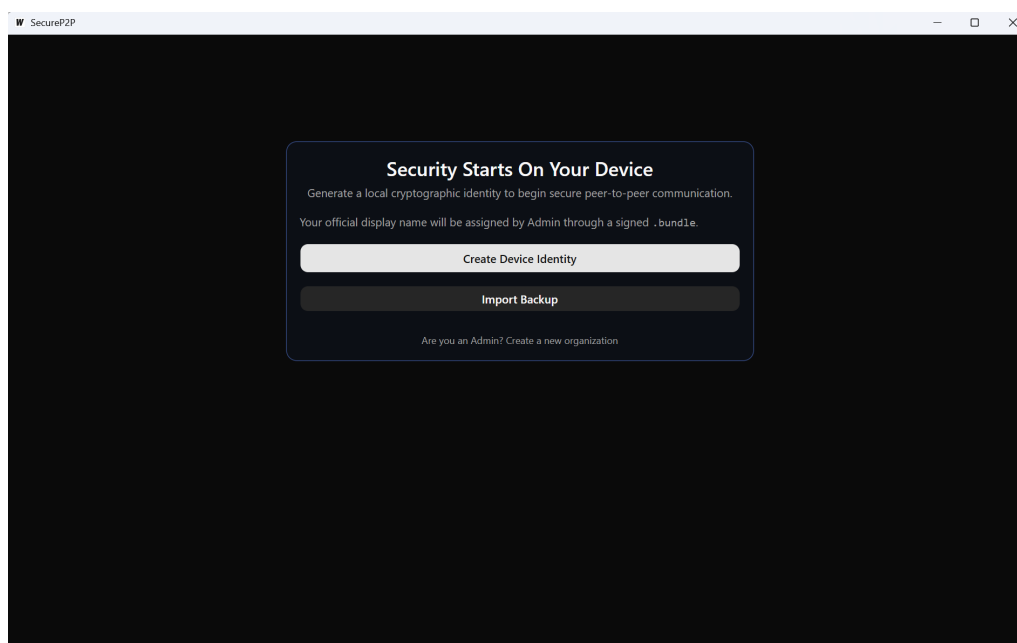


Figure A.1: Màn hình khởi tạo định danh cục bộ trong ứng dụng thử nghiệm

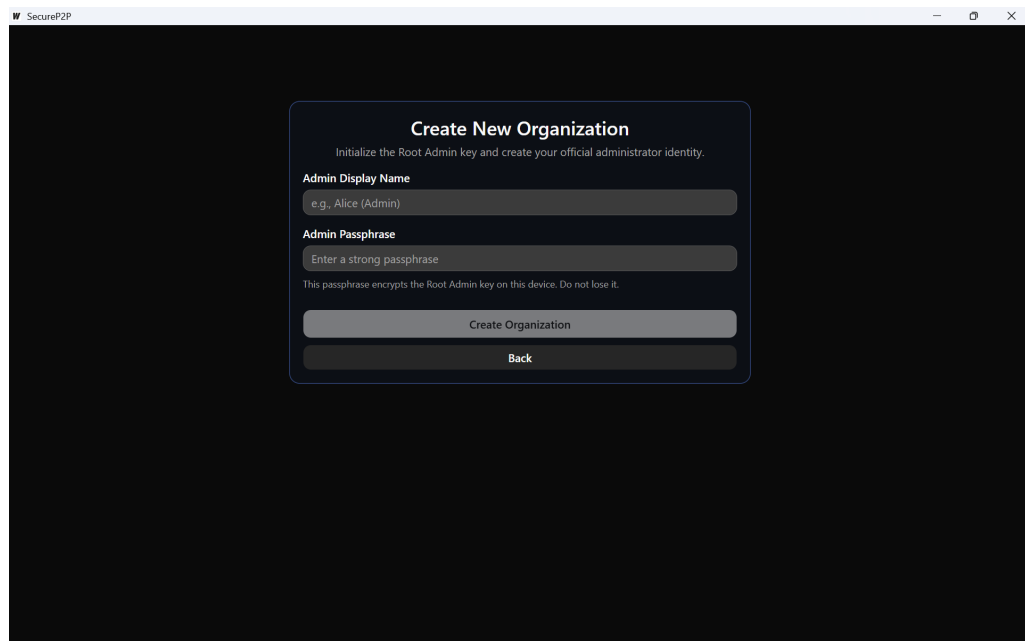


Figure A.2: Màn hình tạo tổ chức gốc của quản trị viên

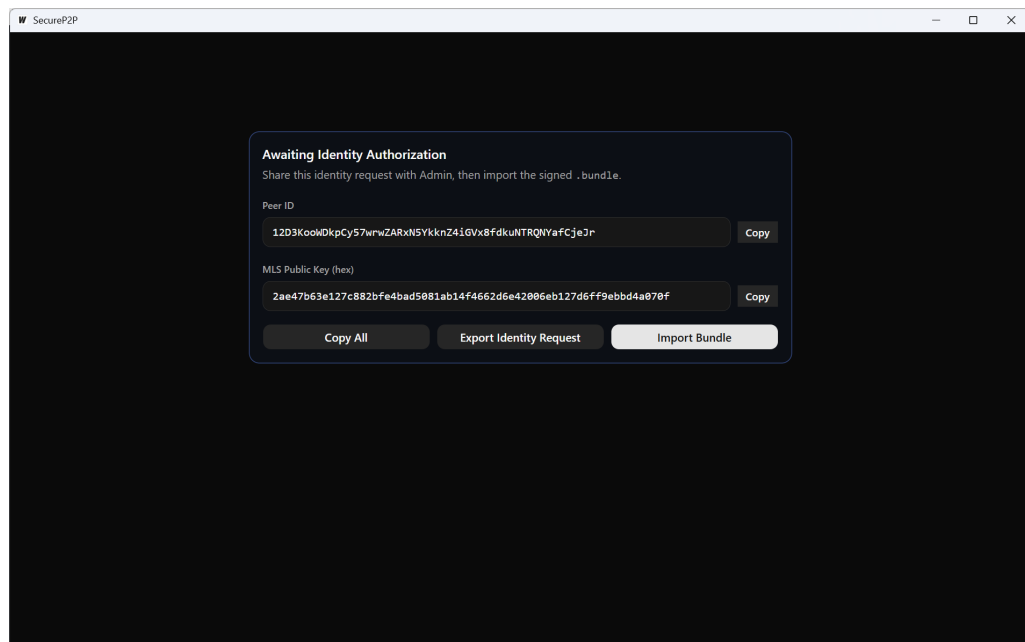


Figure A.3: Thiết bị thành viên ở trạng thái chờ cấp phát bundle

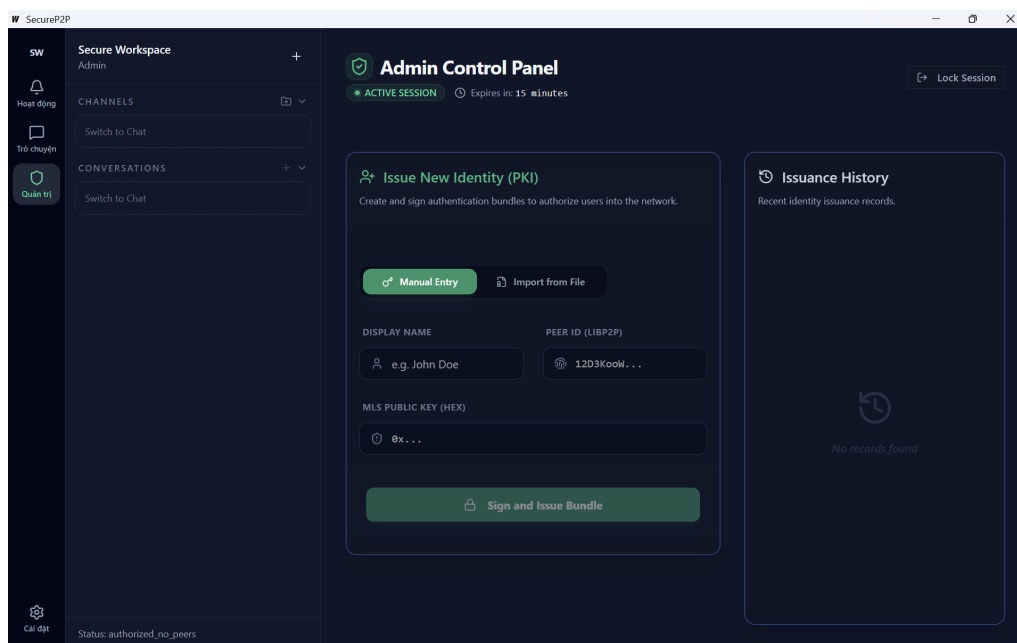


Figure A.4: Bảng điều khiển quản trị phát hành bundle cho thiết bị mới

A.3.2 Không gian làm việc và các chức năng cộng tác

Khi quá trình gia nhập hoàn tất, người dùng được chuyển hướng đến không gian làm việc chính. Hình A.5 mô tả tổng quan bố cục giao diện, tích hợp hài hòa các khu vực chức năng như theo dõi hoạt động, trao đổi thông điệp, quản trị tổ chức và thiết lập hệ thống. Các Hình A.6, Hình A.7 và Hình A.8 minh họa trạng thái đồng bộ của một phiên hội thoại đa điểm, qua góc nhìn của thiết bị Quản trị viên (Admin), người dùng Alice và người dùng Bob. Thông qua kết quả hiển thị, tính toàn vẹn của nội dung tin nhắn và trạng thái thành viên nhóm giữa các nút mạng được khẳng định.

Bên cạnh mô hình hội thoại tuyến tính, ứng dụng hỗ trợ bảng luồng hoạt động tổng hợp (Hình A.9) và không gian kênh thông tin theo cấu trúc bài viết – phản hồi (Hình A.10). Việc phân rã các tính năng cho thấy khả năng mở rộng của giao thức điều phối trong thực tiễn. Cuối cùng, việc khởi tạo nhóm hội thoại mới (Hình A.11) và mời thêm thành viên (Hình A.12) được thực hiện thông qua các hộp thoại chuyên biệt. Sự rõ ràng trong thao tác đồ họa giúp hệ thống dưới nền có thể xử lý mượt mà các tiến trình mật mã phức tạp.

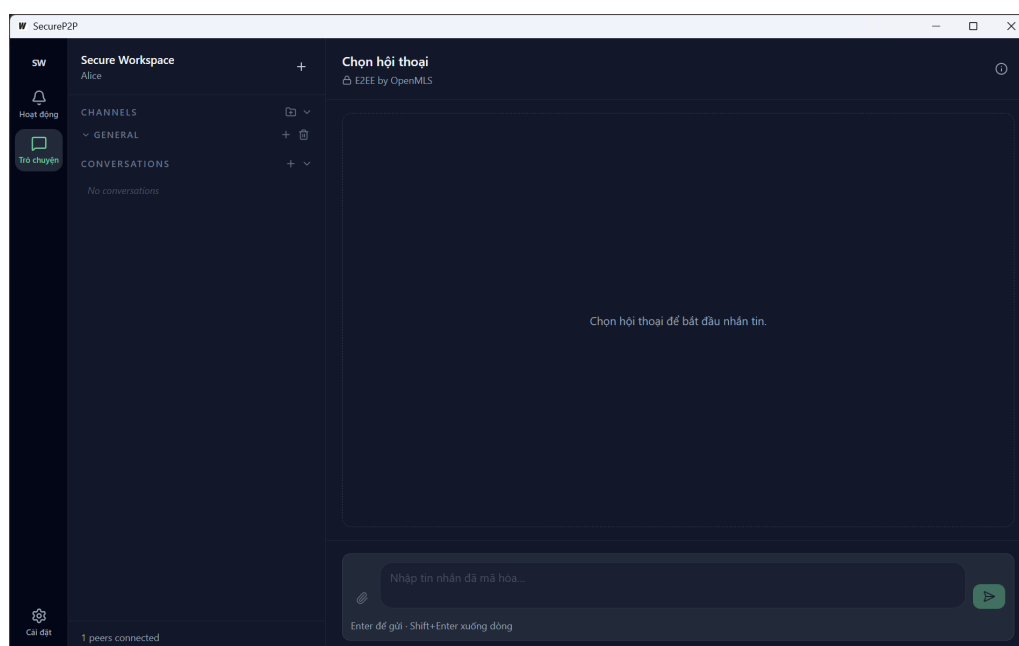


Figure A.5: Tổng quan không gian làm việc của ứng dụng desktop thử nghiệm

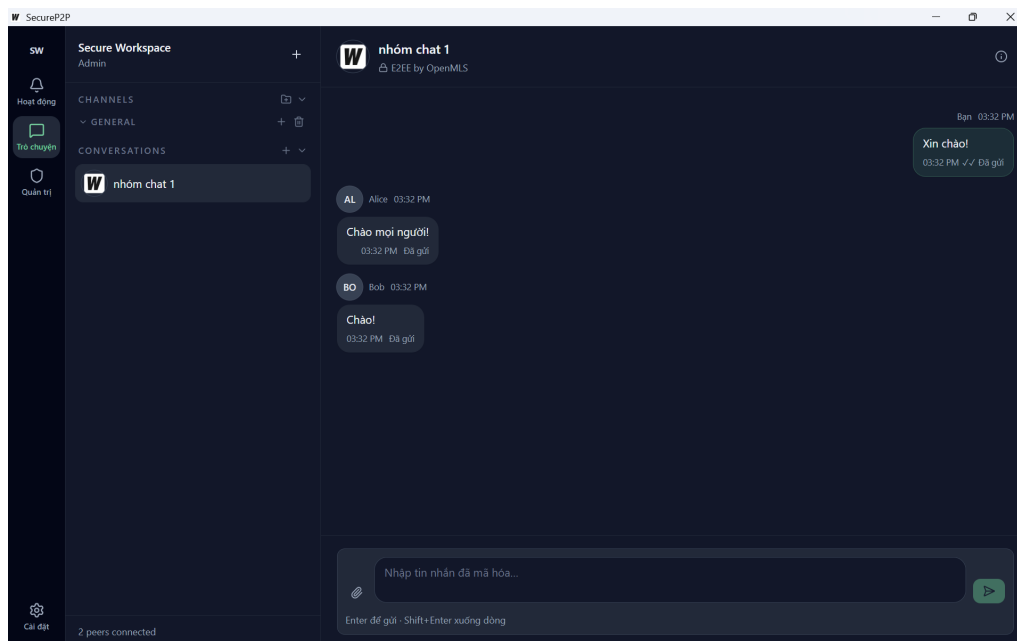


Figure A.6: Giao diện trao đổi tin nhắn trong nhóm trên thiết bị Admin

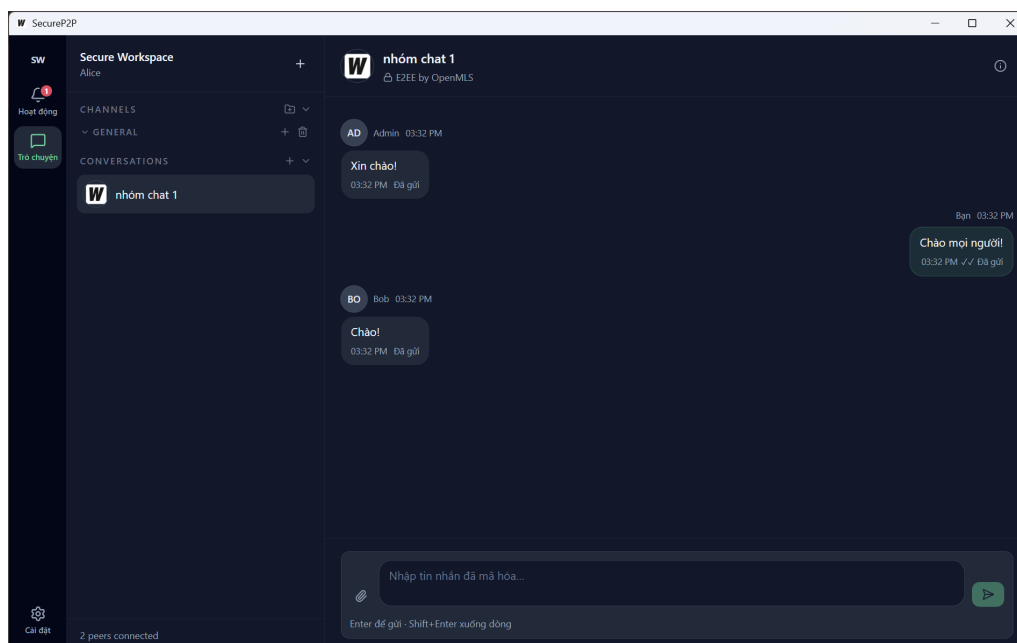


Figure A.7: Giao diện trao đổi tin nhắn trong nhóm trên thiết bị Alice

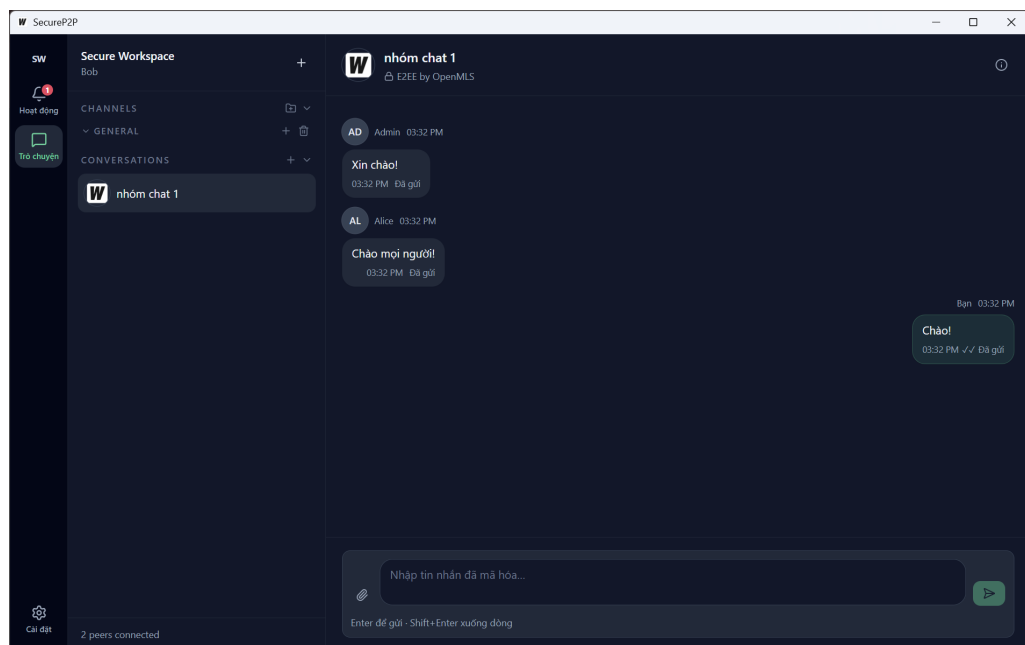


Figure A.8: Giao diện trao đổi tin nhắn trong nhóm trên thiết bị Bob

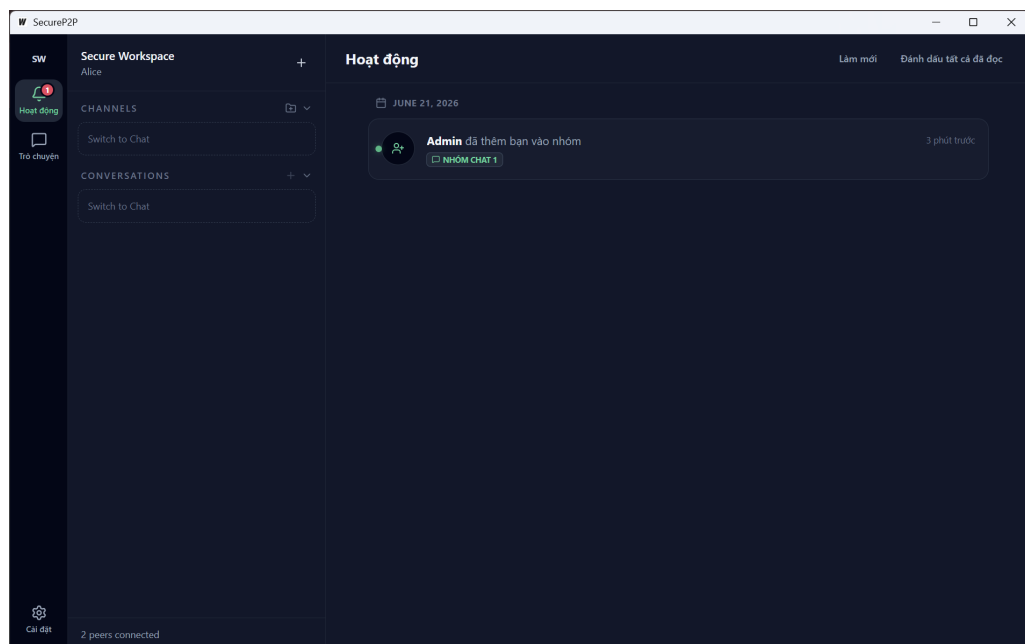


Figure A.9: Màn hình hoạt động và thông báo trong ứng dụng thử nghiệm

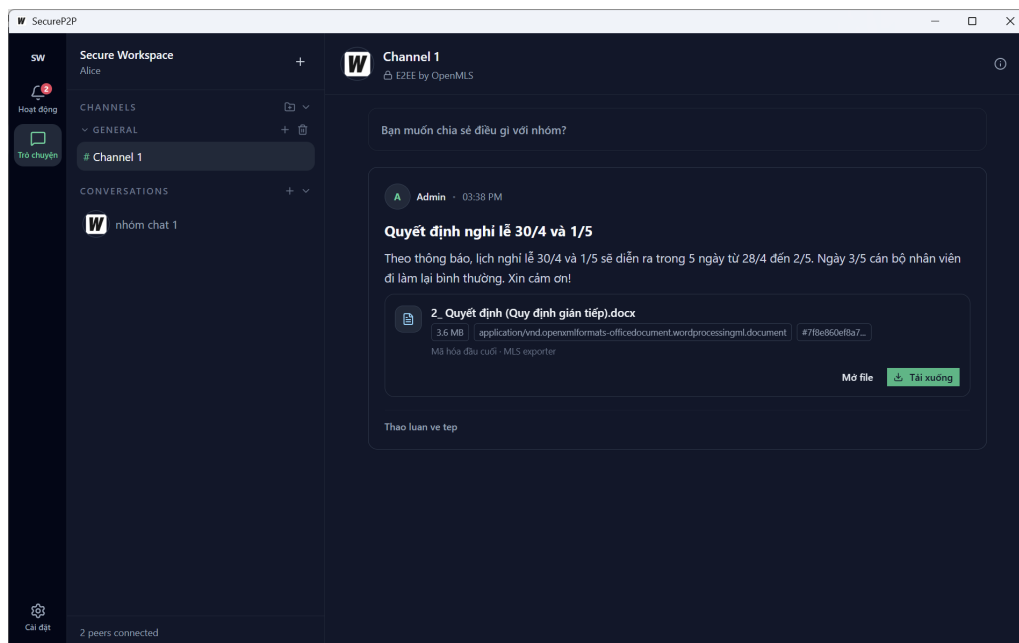


Figure A.10: Không gian kênh giao tiếp theo dạng bài viết và phản hồi

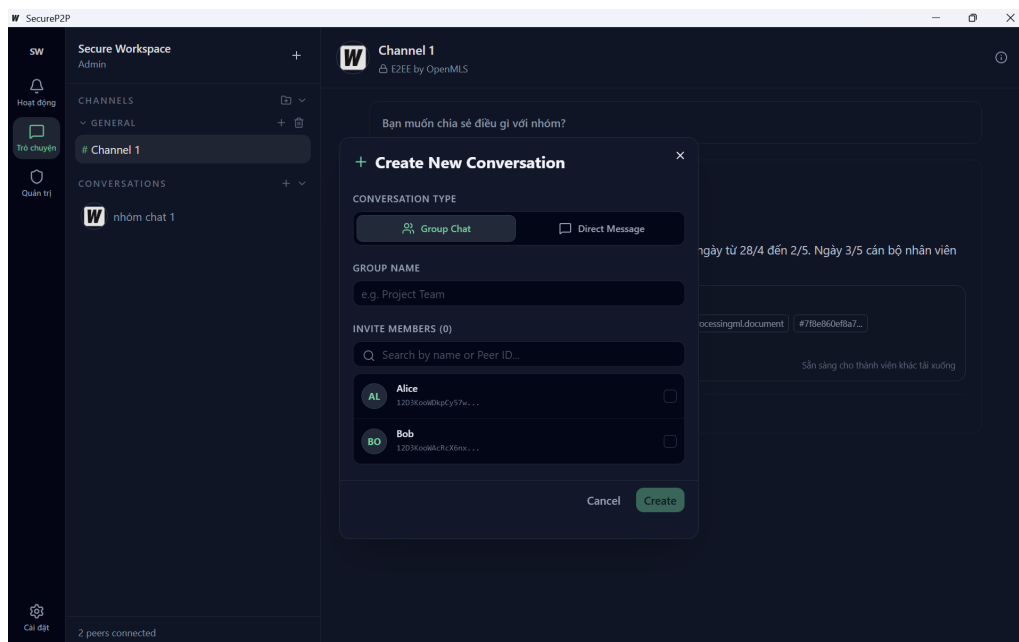


Figure A.11: Hộp thoại tạo nhóm, trò chuyện trực tiếp (DM) hoặc kênh

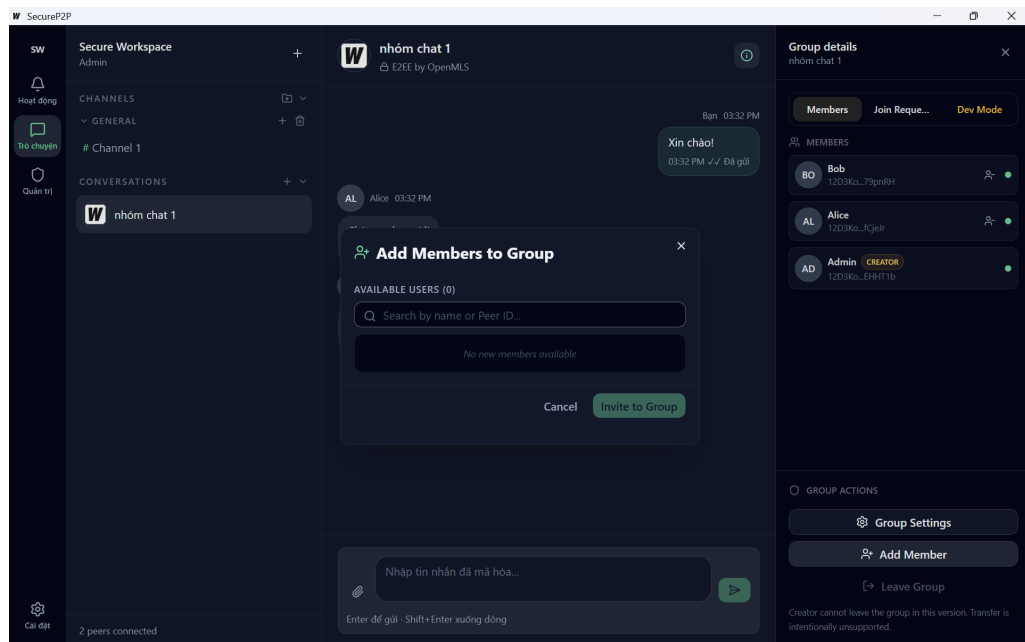


Figure A.12: Hộp thoại mời thêm thành viên vào một nhóm đang có sẵn

A.3.3 Chia sẻ tệp mã hóa trong nhóm

Ứng dụng cho phép người dùng gửi tệp tin trực tiếp trong hội thoại nhóm. Hình A.13 minh họa giao diện khi một tệp được chia sẻ trong dòng tin nhắn. Người gửi chọn tệp từ máy, ứng dụng mã hóa tệp và thông báo đến nhóm. Tệp xuất hiện trong tin nhắn cùng với tên, kích thước và nút tải về. Người nhận nhấn tải về để kéo tệp đã mã hóa từ máy người gửi và giải mã cục bộ. Toàn bộ quá trình diễn ra tự động, người dùng chỉ cần chọn tệp và tải về như các ứng dụng nhắn tin thông thường.

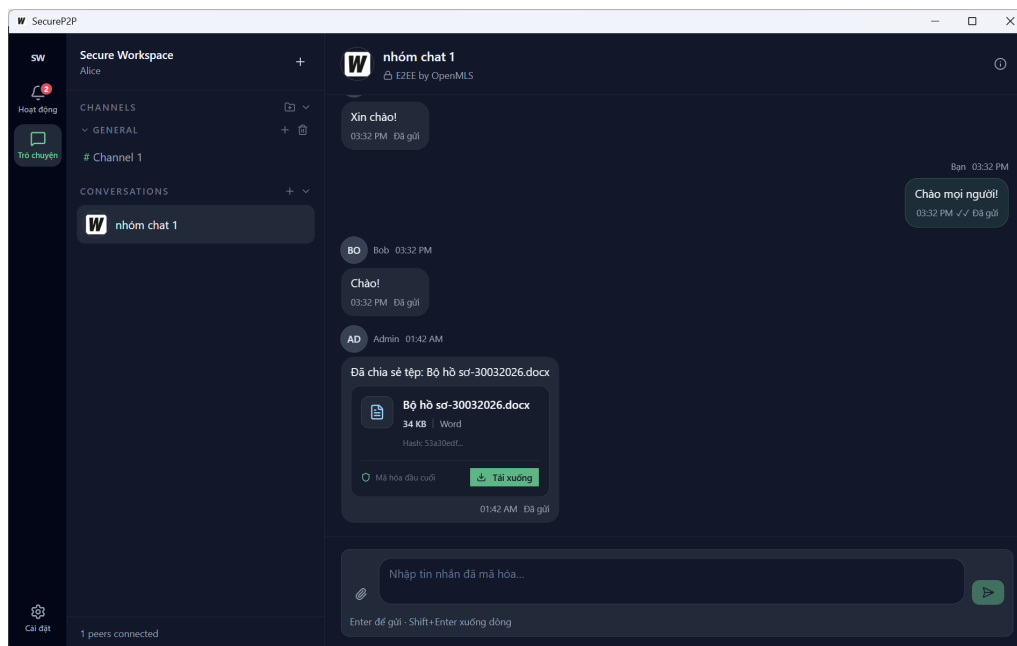


Figure A.13: Tệp mã hóa được hiển thị trong dòng tin nhắn của nhóm

A.3.4 Quản trị hệ thống

Tại màn hình quản trị nhóm (Hình A.14), người dùng có quyền truy xuất danh sách thành viên, kiểm tra vai trò và các chính sách phân quyền hiện hành.

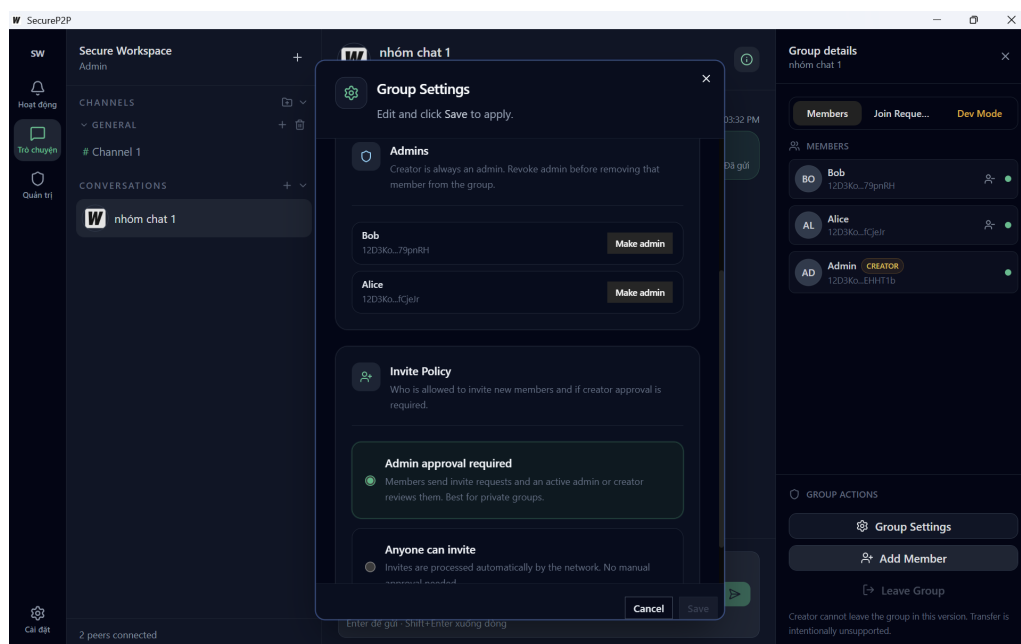


Figure A.14: Màn hình quản trị thành viên và chính sách mời của nhóm

Kết chương: Thông qua việc tích hợp sâu các công nghệ hiện đại như Wails, Go, Rust cùng thư viện mạng ngang hàng libp2p, ứng dụng thử nghiệm không chỉ là một giao diện trực quan đơn thuần mà đã thực sự trở thành một nền tảng phi tập trung hoàn chỉnh. Các kịch bản sử dụng cốt lõi được mô hình hóa sát với thực tế, cùng hệ thống minh họa giao diện chi tiết, đã góp phần khẳng định tính khả thi và đúng đắn của giao thức điều phối được nghiên cứu và đề xuất trong khuôn khổ đề án.